

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Regional Boyacá · Centro Industrial de Mantenimiento y Manufactura – CIMM Tecnólogo
en Análisis y Desarrollo de Software – ADSO

TALLER PRÁCTICO

Pruebas de Software con JUnit 5

Pruebas Unitarias e Integración

Aplicado al Sistema de Gestión de Vacunación

Proyecto base:	Sistema de Gestión de Vacunación
Paquete base:	sena.adso.captcha
Framework de pruebas:	JUnit 5 (junit-jupiter 5.10.2)
BD para integración:	H2 en memoria (no requiere MySQL en pruebas)
Tipo de pruebas:	Unitarias (Unit) + Integración (Integration)
Duración estimada:	12 horas — 3 sesiones de laboratorio
Versión:	1.0 – 2026

1. Introducción — Probar el Sistema de Vacunación

Ya tienes el sistema de vacunación funcionando. El siguiente paso profesional es garantizar que sigue funcionando correctamente cuando se modifique. Las pruebas automatizadas con JUnit 5 permiten verificar cada clase del sistema de forma rápida y repetible, sin necesidad de abrir el navegador y probar manualmente cada módulo.

¿Qué vamos a probar del sistema de vacunación?

Las pruebas se organizan en dos tipos:

PRUEBAS UNITARIAS — prueban una sola clase de forma aislada, sin BD ni red.

- Paciente.java → getNombreCompleto(), validaciones de documento

- Vacuna.java → getInfoCompleta(), estaVencida(), validaciones de lote

- Usuario.java → getNombreCompleto(), esMedico(), esEnfermero()

- ValidadorDocumento.java → clase nueva que crearemos para validar cédulas

PRUEBAS DE INTEGRACIÓN — prueban el DAO conectado a una BD H2 real.

- PacienteDAO.java → insertar, listar, buscar, eliminar

- VacunaDAO.java → insertar, buscar por lote, UNIQUE constraint

- RegistroVacunacionDAO.java → insertar registro, listar por paciente

2. Conceptos Clave de JUnit 5

2.1 Anotaciones que usaremos

Anotación	Cuándo se ejecuta	Ejemplo en vacunación
@Test	Al correr las pruebas	@Test void getNombreCompleto_debeUnirNombresYApellidos()
@BeforeEach	Antes de CADA prueba	Crear un Paciente nuevo para cada test
@AfterEach	Después de CADA prueba	Cerrar la conexión H2 en pruebas de integración
@BeforeAll	Una sola vez antes de TODAS	Crear la tabla pacientes en H2
@AfterAll	Una sola vez después de TODAS	Eliminar la tabla y cerrar H2
Anotación	Cuándo se ejecuta	Ejemplo en vacunación
@DisplayName	—	@DisplayName("Vacuna vencida debe retornar true en estaVencida()")
@Nested	—	Agrupar tests de insertar, actualizar y eliminar de PacienteDAO
@ParameterizedTest	Varias veces con distintos datos	Probar documentos válidos e inválidos
@ValueSource	—	@ValueSource(strings={"1052345678","80456789","CC123"})
@CsvSource	—	Probar múltiples vacunas con distintos datos de entrada

2.2 Assertions más usados



Assertion	Qué verifica — ejemplo con vacunación
<code>assertEquals(esperado, actual)</code>	<code>assertEquals("Carlos Pedraza", paciente.getNombreCompleto())</code>
<code>assertNotNull(objeto)</code>	<code>assertNotNull(vacunaDAO.buscarPorLote("PF001"))</code>
<code>assertNull(objeto)</code>	<code>assertNull(pacienteDAO.buscarPorDocumento("0000000"))</code>
<code>assertTrue(condicion)</code>	<code>assertTrue(vacuna.estaVencida())</code>
<code>assertFalse(condicion)</code>	<code>assertFalse(usuario.esEnfermero())</code> — cuando el rol es MEDICO
<code>assertThrows(Excepcion.class, ...)</code>	<code>assertThrows(IllegalArgumentException.class, () -> new Paciente(null,...))</code>
<code>assertAll(ejecutables...)</code>	Verificar todos los campos de un Paciente a la vez
<code>assertIterableEquals(lista1, lista2)</code>	Verificar que <code>listarTodos()</code> retorna los registros insertados

2.3 Patrón AAA — Arrange, Act, Assert

Todas las pruebas bien escritas siguen este patrón de tres pasos. Separar los bloques con comentarios mejora la legibilidad y facilita el mantenimiento:

```
// Ejemplo real con la clase Vacuna del sistema
@Test
@DisplayName("getInfoCompleta() debe retornar nombre, lote y laboratorio")
void getInfoCompleta debeRetornarDatosFormateados() {

    // ARRANGE — preparar el objeto de prueba
    Vacuna vacuna = new Vacuna(1, "COVID -19 Pfizer", "PF001", "Pfizer", " 2026-1231");

    // ACT — ejecutar el método que queremos probar
    String resultado = vacuna.getInfoCompleta();

    // ASSERT — verificar el resultado esperado
    assertEquals("COVID-19 Pfizer (Lote: PF001, Lab: Pfizer)", resultado);
}
```

3. Configuración del Proyecto para Pruebas

3.1 Agregar dependencias en pom.xml

El pom.xml del sistema de vacunación ya tiene el driver MySQL y la Servlet API. Solo hay que agregar JUnit 5 y H2 como dependencias de prueba:

```
<!-- Agregar en la sección <dependencies> del pom.xml existente -->

<!-- JUnit 5 - framework de pruebas (scope test: no va en el WAR) -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>

<!-- H2 - BD en memoria para pruebas de integración (scope test) -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.2.224</version>
  <scope>test</scope>
</dependency>

<!-- Agregar en la sección <build><plugins> -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
</plugin>
```

3.2 Estructura de carpetas de prueba

```
captcha/src/
├── main/java/sena/adso/captcha/
│   ├── dto/
│   │   ├── Paciente.java           ← ya existe
│   │   ├── Vacuna.java             ← ya existe
│   │   ├── Usuario.java            ← ya existe
│   │   └── RegistroVacunacion.java ← ya existe
│   ├── dao/
│   │   ├── PacienteDAO.java         ← ya existe
│   │   ├── VacunaDAO.java           ← ya existe
│   │   └── RegistroVacunacionDAO.java ← ya existe
│   ├── model/
│   │   └── Conexion.java            ← ya existe
│   └── util/
│       └── ValidadorDocumento.java ← NUEVA clase a crear para probar
└── test/java/sena/adso/captcha/
    ├── dto/
    │   ├── PacienteTest.java        ← Sesión 1
    │   └── VacunaTest.java          ← Sesión 1
    ├── UsuarioTest.java             ← Sesión 1
    ├── util/
    │   └── ValidadorDocumentoTest.java ← Sesión 2
    └── dao/
        ├── ConexionTestHelper.java ← clase auxiliar para H2
        ├── PacienteDAOTest.java     ← Sesión 3
        ├── VacunaDAOTest.java       ← Sesión 3
        └── RegistroVacunacionDAOTest.java ← Sesión 3

captcha/src/test/resources/
└── db-test.properties              ← configuración de H2
```

4. Nueva Clase: ValidadorDocumento.java

Para practicar `@ParameterizedTest` necesitamos una clase con lógica de validación. Crea esta clase en `src/main/java/sena/adso/captcha/util/`:

```

package sena.adso.captcha.util;

/**
 * Valida números de documento colombianos.
 * Reglas: solo dígitos, entre 6 y 10 caracteres, no null ni vacío.
 */ public class
ValidadorDocumento {

    private static final int MIN_DIGITOS = 6;    private
static final int MAX_DIGITOS = 10;

    /**
     * Valida si un número de documento es válido.
     * @param documento Número de documento a validar
     * @return true si es válido, false si no
     */    public boolean esValido(String documento) {        if (documento ==
null || documento.isBlank()) return false;        String limpio =
documento.trim();        if (!limpio.matches("\\d+")) return false;        return
limpio.length() >= MIN_DIGITOS && limpio.length() <= MAX_DIGITOS;    }

    /**
     * Verifica si el documento pertenece a un menor de edad        * (tarjeta de
identidad: 10 dígitos empezando por 1).
     */    public boolean esTarjetaIdentidad(String
documento) {        if (!esValido(documento)) return
false;        return documento.trim().length() == 10
        &&
documento.trim().startsWith("1");    } }

```

También necesitamos agregar el método `estaVencida()` a `Vacuna.java` y los métodos `esMedico()`/`esEnfermero()` a `Usuario.java` para tener más lógica que probar:

```

// Agregar a Vacuna.java (en el paquete sena.adso.captcha.dto) import
java.time.LocalDate;

/**
 * Verifica si la vacuna ya pasó su fecha de vencimiento.
 * @return true si fecha_vencimiento < hoy
 */
public boolean estaVencida() {    if (fechaVencimiento == null ||
fechaVencimiento.isBlank()) return false;    LocalDate vencimiento =
LocalDate.parse(fechaVencimiento); // formato YYYY-MMDD    return
LocalDate.now().isAfter(vencimiento);
}

// Agregar a Usuario.java (en el paquete sena.adso.captcha.dto)
/** Retorna true si el usuario tiene rol MEDICO
 */ public boolean esMedico() {    return
"MEDICO".equalsIgnoreCase(rol);
}

```

```
/** Retorna true si el usuario tiene rol ENFERMERO
 */
public boolean esEnfermero() {    return "ENFERMERO".equalsIgnoreCase(rol);
}
```

5. Sesión 1 — Pruebas Unitarias de los DTOs

Objetivo de la sesión 1

Escribir pruebas unitarias para Paciente.java, Vacuna.java y Usuario.java.

Aprender @BeforeEach, assertAll, assertThrows y @Nested.

NO se usa base de datos en esta sesión — todo en memoria.

5.1 PacienteTest.java


```

    }

    // — getNombreCompleto() —————

    @Test
    @DisplayName("getNombreCompleto() debe unir nombres y apellidos con espacio")
    void getNombreCompleto_debeUnirNombresYApellidos() {
        // ARRANGE — ya listo en @BeforeEach
        // ACT
        String resultado = paciente.getNombreCompleto();
        // ASSERT
        assertEquals("Carlos Ernesto Pedraza Rondón", resultado);
    }

    @Test
    @DisplayName("getNombreCompleto() con nombres vacíos debe retornar solo apellidos")
    void getNombreCompleto_conNombresVacios() {
        paciente.setNombres("");
        // Nota: depende de cómo esté implementado getNombreCompleto()
        // Esta prueba documenta el comportamiento actual del sistema
        String resultado = paciente.getNombreCompleto();
        assertTrue(resultado.contains("Pedraza Rondón"));
    }

    // — assertAll: verificar todos los campos a la vez —————

    @Test
    @DisplayName("Constructor debe inicializar todos los campos correctamente")
    void constructor_debeInicializarTodosLosCampos() {
        assertAll("Campos del Paciente",
            () -> assertEquals(1, paciente.getId()),
            () -> assertEquals("Carlos Ernesto", paciente.getNombres()),
            () -> assertEquals("Pedraza Rondón", paciente.getApellidos()),
            () -> assertEquals("1052345678", paciente.getDocumento()),
            () -> assertNotNull(paciente.getFechaNacimiento())
        );
    }

    // — Getters y Setters —————

    @Test
    @DisplayName("setDocumento() debe actualizar el documento correctamente")
    void setDocumento_debeActualizarDocumento() {
        paciente.setDocumento("80456789");
        assertEquals("80456789", paciente.getDocumento());
    }

    @Test
    @DisplayName("Paciente vacío (constructor sin parámetros) debe tener id=0")
    void constructorVacio_debeCrearPacienteConIdCero() {
        new Paciente();
        assertEquals(0, vacio.getId());
        assertNull(vacio.getNombres());
    }

    // — @Nested: agrupar por contexto —————

    @Nested

```

```

        @DisplayName("Pruebas de nombre completo")        class
NombreCompletoTests {

        @Test
        @DisplayName("Nombre completo de paciente real de Nobsa")
void pacienteNobsa_nombreCompleto() {
        Paciente nobsa = new Paciente(2, "María Eugenia", "Suárez Cely",
        "1052345679", new Date());
assertEquals("María Eugenia Suárez Cely", nobsa.getNombreCompleto());
}

        @Test
        @DisplayName("Nombre completo no debe ser null")        void
nombreCompleto_noDebeSerNull() {
assertNotNull(paciente.getNombreCompleto());
}

        @Test
        @DisplayName("Nombre completo no debe estar vacío")        void
nombreCompleto_noDebeEstarVacio() {
assertFalse(paciente.getNombreCompleto().isBlank());
}
}

```

5.2 VacunaTest.java

```

package sena.adso.captcha.dto;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class VacunaTest {

    private Vacuna vacunaVigente;    private
    Vacuna vacunaVencida;

    @BeforeEach    void
    setUp() {
        // Vacuna con fecha futura (datos del script vacunacion_db.sql)
        vacunaVigente = new Vacuna(1, "COVID-19 Pfizer", "PF001", "Pfizer", "202612-
        31");
        // Vacuna con fecha pasada (vencida)
        vacunaVencida = new Vacuna(3, "Influenza", "IN001", "Sanofi", "2024-01-
        01");
    }

    @Test    @DisplayName("getInfoCompleta() debe retornar formato: Nombre (Lote:
    X, Lab:
    Y)")    void getInfoCompleta_debeRetornarFormatoCorrecto()
    {
        String esperado = "COVID-19 Pfizer (Lote: PF001, Lab: Pfizer)";
        assertEquals(esperado, vacunaVigente.getInfoCompleta());
    }

    @Test

```



```
@DisplayName("estaVencida() debe retornar false para vacuna con fecha futura")
```

```

    void estaVencida_conFechaFutura_debeRetornarFalse()
    {
        assertFalse(vacunaVigente.estaVencida());
    }

    @Test
    @DisplayName("estaVencida() debe retornar true para vacuna con fecha
    pasada")
    void estaVencida_conFechaPasada_debeRetornarTrue() {
        assertTrue(vacunaVigente.estaVencida());
    }

    @Test
    @DisplayName("Constructor debe asignar todos los campos
    correctamente")
    void constructor_debeAsignarTodosLosCampos() {
        assertEquals(1, vacunaVigente.getId()),
        assertEquals("COVID-19 Pfizer", vacunaVigente.getNombre()),
        assertEquals("PF001", vacunaVigente.getLote()),
        assertEquals("Pfizer", vacunaVigente.getLaboratorio()),
        assertEquals("2026-12-31",
        vacunaVigente.getFechaVencimiento())
    };

    @Test
    @DisplayName("Vacuna con fecha_vencimiento null no debe lanzar excepción")
    void estaVencida_conFechaNull_debeRetornarFalse() {
        Vacuna sinFecha = new Vacuna(5, "Triple Viral", "TV001", "Serum",
        null); // No debe lanzar NullPointerException - el método debe manejarlo
        assertDoesNotThrow(() -> sinFecha.estaVencida());
        assertFalse(sinFecha.estaVencida());
    }
}

```

5.3 UsuarioTest.java

```

package sena.adso.captcha.dto;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class UsuarioTest {

    private Usuario medico;    private
    Usuario enfermero;

    @BeforeEach    void
    setUp() {
        // Datos del INSERT de muestra en vacunacion_db.sql
        medico
        = new Usuario(1, "Admin", "Sistema", "1000000000",
        "admin@example.com", "admin", "admin123",
        "MEDICO", "Administración", "Hospital General");
        enfermero = new Usuario(2,
        "María", "López", "52345678",
        "maria@hospital.com", "mlopez", "enf123",
        "ENFERMERO", null, "Hospital General");
    }
}

```

@Test

```

        @DisplayName("esMedico() debe retornar true para usuario con rol MEDICO")
        void esMedico_conRolMedico_debeRetornarTrue() {
            assertTrue(medico.esMedico());
        }

        @Test
        @DisplayName("esMedico() debe retornar false para usuario con rol ENFERMERO")
        void esMedico_conRolEnfermero_debeRetornarFalse() {
            assertFalse(enfermero.esMedico());
        }

        @Test
        @DisplayName("esEnfermero() debe retornar true para usuario con rol ENFERMERO")
        void esEnfermero_conRolEnfermero_debeRetornarTrue() {
            assertTrue(enfermero.esEnfermero());
        }

        @Test
        @DisplayName("esMedico() debe ser insensible a mayúsculas (medico == MEDICO)")
        void esMedico_debeSerInsensibleAMayusculas() {
            medico.setRol("medico"); // minúsculas
            assertTrue(medico.esMedico());
        }

        @Test
        @DisplayName("getNombreCompleto() para el admin del sistema")
        void getNombreCompleto_paraAdmin() {
            assertEquals("Admin Sistema", medico.getNombreCompleto());
        }

        @Test
        @DisplayName("Usuario con rol null NO debe ser médico ni enfermero")
        void usuarioSinRol_noEsMedicoNiEnfermero() {
            medico.setRol(null);
            assertFalse(medico.esMedico());
            assertFalse(medico.esEnfermero());
        }
    }

```

6. Sesión 2 — Pruebas Parametrizadas con @ParameterizedTest

Objetivo de la sesión 2

Usar @ParameterizedTest para probar ValidadorDocumento.java con muchos casos

sin duplicar código. Entender los casos límite del sistema de vacunación.

6.1 ValidadorDocumentoTest.java

```
package sena.adso.captcha.util;

import org.junit.jupiter.api.*; import
org.junit.jupiter.params.ParameterizedTest; import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;
```



```

class ValidadorDocumentoTest {

    private ValidadorDocumento validador;

    @BeforeEach
    void setUp() { validador = new ValidadorDocumento(); }

    // — Documentos válidos — datos reales del script vacunacion_db.sql —

    @ParameterizedTest(name = "Documento válido: [{0}]")
    @ValueSource(strings = {
        "1000000000", // admin del sistema (10 dígitos)
        "1052345678", // paciente de Nobsa
        "52345678",   // 8 dígitos — válido
        "123456",     // 6 dígitos — mínimo válido
        "9999999999"  // 10 dígitos — máximo válido
    })
    @DisplayName("Documentos válidos deben retornar true")    void
    documentoValido_debeRetornarTrue(String doc) {
        assertTrue(validador.esValido(doc),
            "Se esperaba que '" + doc + "' fuera válido");
    }

    // — Documentos inválidos —————

    @ParameterizedTest(name = "Documento inválido: [{0}]")
    @ValueSource(strings = {
        "12345",      // 5 dígitos — menor al mínimo
        "12345678901", // 11 dígitos — mayor al máximo
        "CC-123456",   // tiene letras y guión
        "105 345",     // tiene espacio
        ""             // vacío
    })
    @DisplayName("Documentos inválidos deben retornar
false")    void documentoInvalido_debeRetornarFalse(String
doc) {        assertFalse(validador.esValido(doc));    }

    @Test
    @DisplayName("null debe retornar false sin lanzar
NullPointerException")    void documentoNull_debeRetornarFalse() {
        assertFalse(validador.esValido(null));    }

    // — @CsvSource: probar esTarjetaIdentidad() con input + expected —

    @ParameterizedTest(name = "doc={0}, esperado={1}")
    @CsvSource({
        "1052345678, true",    // 10 dígitos, empieza en 1 → tarjeta
        "1000000000, true",    // admin del sistema → tarjeta
        "52345678, false",     // 8 dígitos → no es tarjeta
        "2052345678, false",   // 10 dígitos pero empieza en 2 → no es
tarjeta    "123456, false",   // 6 dígitos → no es tarjeta    })
    @DisplayName("esTarjetaIdentidad() con varios documentos")    void

```

```
esTarjetaIdentidad_variosDocumentos(String doc, boolean esperado) {  
    assertEquals(esperado, validador.esTarjetaIdentidad(doc.trim()));  
}
```



```
// — Pruebas de casos límite —————

@Test
@DisplayName("Documento con exactamente 6 dígitos (mínimo) debe ser válido") void
documento_6digitos_debeSerValido() {
    assertTrue(validador.esValido("123456")); // valor límite mínimo    }

@Test
@DisplayName("Documento con exactamente 10 dígitos (máximo) debe ser válido")
void documento_10digitos_debeSerValido() {
    assertTrue(validador.esValido("1052345678")); // valor límite máximo    }

@Test
@DisplayName("Documento con 5 dígitos (debajo del mínimo) debe ser inválido")
void documento_5digitos_debeSerInvalido() {
    assertFalse(validador.esValido("12345")); // un dígito por debajo    }

@Test
@DisplayName("Documento con 11 dígitos (encima del máximo) debe ser inválido") void
documento_11digitos_debeSerInvalido() {    assertFalse(validador.esValido("10523456780"));
// un dígito por encima    } }
```

6.2 Tabla de casos límite del sistema de vacunación

Regla del sistema	Valor límite	Caso de prueba	¿Qué puede fallar?
Documento mínimo 6 dígitos	123456 (exacto)	assertTrue(esValido("123456"))	Rechazar el valor mínimo válido
Documento máximo 10 dígitos	1052345678 (exacto)	assertTrue(esValido("1052345678"))	Rechazar el valor máximo válido
Vacuna: no duplicar lote+nombre	Mismo lote PF001 dos veces	assertThrows en VacunaDAO.insertar()	No detectar la violación UNIQUE
Vacuna no vencida	Fecha vencimiento = hoy	assertFalse(estaVencida())	Marcar como vencida el mismo día

getNombreCompleto no null	nombres = null	assertNotNull(getNombreCompleto())	NullPointerException al concatenar
Rol insensible a mayúsculas	"medico" vs "MEDICO"	assertTrue(esMedico()) con minúsculas	Rechazar rol en minúsculas

7. Sesión 3 — Pruebas de Integración con H2

¿Por qué H2 y no MySQL directamente?

Las pruebas de integración necesitan una base de datos REAL, pero usar MySQL de

producción tiene riesgos: datos sucios entre pruebas, conexión compartida con otros,

velocidad lenta. H2 es la solución:

- BD en memoria: se crea al iniciar las pruebas y se borra al terminar.

- Cada prueba empieza con datos limpios — garantiza independencia.

- Soporta MODE=MySQL: el mismo SQL del sistema de vacunación funciona en H2.

- 10x más rápida que MySQL para pruebas.

- scope test: H2 nunca va al WAR de producción.

7.1 db-test.properties — Configuración de H2

```
# src/test/resources/db-test.properties
# Configuración de H2 en memoria para pruebas de integración

db.url=jdbc:h2:mem:vacunacion_test;DB_CLOSE_DELAY=1;MODE=MySQL db.driver=org.h2.Driver
db.user=sa db.password=

# DB_CLOSE_DELAY=-1 → la BD permanece en memoria mientras la JVM corre
# MODE=MySQL → H2 acepta sintaxis MySQL (AUTO_INCREMENT, ENUM, etc.)
```

7.2 ConexionTestHelper.java — Clase auxiliar para pruebas

```
package sena.adso.captcha.dao;

import java.io.InputStream; import java.sql.*;
import java.util.Properties;

/**
 * Clase auxiliar SOLO para pruebas.
 * Provee una conexión H2 en lugar de MySQL.
 */ public class
ConexionTestHelper {

    private static Connection conexion;

    public static Connection getConexion() throws Exception
    {
        if (conexion == null || conexion.isClosed()) {
            Properties props = new Properties();

            try (InputStream in = ConexionTestHelper.class
                .getClassLoader()
                .getResourceAsStream("db-test.properties"))
            {
                props.load(in);
            }
            Class.forName(props.getProperty("db.driver"));
            conexion = DriverManager.getConnection(
                props.getProperty("db.url"),
                props.getProperty("db.user"),
                props.getProperty("db.password")
            );
        }
        return conexion;
    }

    public static void ejecutarSQL(String sql) throws Exception {
        try
        (Statement stmt = getConexion().createStatement()) {
            stmt.execute(sql);
        }
    }
}
```

7.3 PacienteDAOTest.java — Integración completa

```

package sena.adso.captcha.dao;

import sena.adso.captcha.dto.Paciente; import
org.junit.jupiter.api.*; import java.util.*;
import static
org.junit.jupiter.api.Assertions.*;

/**
 * Pruebas de INTEGRACIÓN: PacienteDAO contra H2 en memoria.
 * El mismo SQL del script vacunacion_db.sql funciona en H2 con MODE=MySQL.
 */
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class PacienteDAOTest {

    private static PacienteDAO dao;

    @BeforeAll      static void crearTabla() throws
Exception {
        // Crear la tabla pacientes en H2 – mismo DDL del script vacunacion_db.sql
        ConexionTestHelper.ejecutarSQL(
            "CREATE TABLE IF NOT EXISTS pacientes (" +
            "  id INT AUTO_INCREMENT PRIMARY KEY," +
            "  nombres VARCHAR(100) NOT NULL," +
            "  apellidos VARCHAR(100) NOT NULL," +
            "  documento VARCHAR(20) NOT NULL UNIQUE," +
            "  fecha_nacimiento DATE NOT NULL," +
            "  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)"
        );
        dao = new PacienteDAO(); // usar constructor sin
parámetros
        // NOTA: PacienteDAO necesita ser adaptado para aceptar una
Connection      // externa, o usar una variable de entorno para la URL de
BD.      // Ver sección 7.4 para la adaptación mínima requerida.
    }

```

```

        @BeforeEach      void limpiarDatos() throws
Exception {
        // Limpiar antes de CADA prueba - independencia total
        ConexionTestHelper.ejecutarSQL("DELETE FROM pacientes");
    }

    @AfterAll
    static void eliminarTabla() throws Exception {
        ConexionTestHelper.ejecutarSQL("DROP TABLE IF EXISTS pacientes");
    }

    @Test
    @Order(1)
    @DisplayName("insertar() debe retornar true y guardar el paciente en H2")
    void insertar_debeRetornarTrueYGuardarPaciente() throws Exception {
        Paciente p = new Paciente(0, "Carlos Ernesto", "Pedraza Rondón",
                                   "1052345678",
                                   java.sql.Date.valueOf("1985-03-12"));

        boolean resultado = dao.insertar(p);

        assertTrue(resultado, "insertar() debe retornar true al guardar el
paciente");
    }

    @Test
    @Order(2)
    @DisplayName("listarTodos() debe retornar lista vacía si no hay pacientes")
    void listarTodos_debeRetornarListaVacía_siBDVacía() throws Exception {
        List<Paciente> lista = dao.listarTodos();          assertNotNull(lista, "La lista
nunca debe ser null");          assertTrue(lista.isEmpty(), "Sin datos, la lista
debe estar vacía");    }

    @Test
    @Order(3)
    @DisplayName("listarTodos() debe retornar los pacientes insertados")    void
listarTodos_debeRetornarPacientesInsertados() throws Exception {
        // Insertar dos pacientes de Boyacá          dao.insertar(new
Paciente(0, "Carlos Ernesto", "Pedraza Rondón",
"1052345678",
                                   java.sql.Date.valueOf("1985-03-12")));
        dao.insertar(new Paciente(0, "María Eugenia", "Suárez Cely",
                                   "1052345679",
                                   java.sql.Date.valueOf("1992-07-24")));

        List<Paciente> lista = dao.listarTodos();

        assertEquals(2, lista.size(), "Deben existir exactamente 2 pacientes");
    }

    @Test

```

```
@Order(4) @DisplayName("insertar() con documento duplicado debe fallar  
(UNIQUE constraint)") void insertar_conDocumentoDuplicado_debeFallar()  
throws Exception {  
    Paciente p1 = new Paciente(0, "Carlos", "Pedraza",  
                                "1052345678",
```

```

        java.sql.Date.valueOf("1985-03-12"));
        Paciente p2 = new Paciente(0, "Otro", "Nombre",
        "1052345678", // mismo documento
        java.sql.Date.valueOf("1990-01-01"));

        dao.insertar(p1); // primer insert exitoso
        boolean resultado = dao.insertar(p2); // debe fallar

        assertFalse(resultado,
            "No se debe poder insertar dos pacientes con el mismo documento");
    }

    @Test
    @Order(5)
    @DisplayName("eliminar() debe reducir el conteo de pacientes en 1")
    void eliminar_debeReducirConteo() throws Exception {
        dao.insertar(new Paciente(0, "Carlos", "Pedraza",
        "1052345678",
        java.sql.Date.valueOf("1985-03-12")));
        List<Paciente> antes =
        dao.listarTodos();
        int idEliminar = antes.get(0).getId();

        dao.eliminar(idEliminar);

        List<Paciente> despues = dao.listarTodos();
        assertEquals(antes.size()
        - 1, despues.size());
    }
}

```

7.4 Adaptación mínima de PacienteDAO para pruebas

El PacienteDAO actual usa `Conexion.getConnection()` que apunta a MySQL. Para las pruebas de integración necesita poder usar la conexión H2. La forma más sencilla sin refactorizar el DAO completo es usar variables de entorno:


```
// Opción A – Agregar constructor que acepta Connection (recomendada):
public class PacienteDAO {

    private Connection conexionExterna; // null = usar Conexion.getConnection()

    public PacienteDAO() { this.conexionExterna = null; }

    // Constructor para pruebas de
integración    public PacienteDAO(Connection
conexion) {    this.conexionExterna =
conexion;      }

    private Connection obtenerConexion() throws SQLException {        return
conexionExterna != null
        ? conexionExterna
        : Conexion.getConnection();
    }
    // ... resto del DAO usa obtenerConexion() en lugar de
Conexion.getConnection()
}

```

```
// En la prueba de integración:
dao = new PacienteDAO(ConexionTestHelper.getConexion()); // usa H2

```

```
// En producción (Servlet):
PacienteDAO dao = new PacienteDAO(); // usa MySQL – sin cambios

```

7.5 VacunaDAOTest.java — Prueba de la restricción UNIQUE

```
package sena.adso.captcha.dao;

import sena.adso.captcha.dto.Vacuna; import
org.junit.jupiter.api.*; import java.util.List; import
static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class) class VacunaDAOTest {

    private static VacunaDAO dao;

    @BeforeAll    static void crearTabla() throws
Exception {
        // La tabla vacunas tiene UNIQUE KEY uk_lote_nombre (lote, nombre)
        ConexionTestHelper.ejecutarSQL(
            "CREATE TABLE IF NOT EXISTS vacunas (" +
            " id INT AUTO_INCREMENT PRIMARY KEY," +
            " nombre VARCHAR(100) NOT NULL," +
            " lote VARCHAR(50) NOT NULL," +
            " laboratorio VARCHAR(100) NOT NULL," +
            " fecha_vencimiento DATE NOT NULL," +
            " UNIQUE KEY uk_lote_nombre (lote, nombre))"
        );
        dao = new
VacunaDAO(ConexionTestHelper.getConexion());    }

    @BeforeEach
    void limpiar() throws Exception {
        ConexionTestHelper.ejecutarSQL("DELETE FROM vacunas");
    }

    @AfterAll
    static void destruir() throws Exception {
        ConexionTestHelper.ejecutarSQL("DROP TABLE IF EXISTS vacunas");    }

    @Test
    @Order(1)
    @DisplayName("insertar() debe guardar la vacuna en H2")    void
insertar_debeGuardarVacuna() {
        // Datos del INSERT del script vacunacion_db.sql
        Vacuna v = new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-
31");
        assertTrue(dao.insertar(v));
    }

    @Test
    @Order(2)
```

```

        @DisplayName("insertar() mismo lote+nombre debe fallar (UNIQUE KEY)")        void
insertar_loteDuplicado_debeFallar() {
    Vacuna v1 = new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2026-12-
31");
    Vacuna v2 = new Vacuna(0, "COVID-19 Pfizer", "PF001", "Pfizer", "2027-01-
01");
    dao.insertar(v1);
    boolean resultado = dao.insertar(v2); // mismo lote+nombre
assertFalse(resultado,
    "No se puede insertar dos vacunas con el mismo lote y nombre");
}

@Test
@Order(3)
    @DisplayName("listarTodos() retorna las vacunas del script de datos de
prueba")        void listarTodos_retornaVacunasInsertadas() {        // Insertar las
5 vacunas del script vacunacion_db.sql        dao.insertar(new Vacuna(0, "COVID-
19 Pfizer", "PF001", "Pfizer", "2026-12-
31"));        dao.insertar(new Vacuna(0, "COVID-19 Moderna", "MD001", "Moderna",
"2026-
10-15"));        dao.insertar(new Vacuna(0, "Influenza", "IN001", "Sanofi",
"2025-08-22"));        dao.insertar(new Vacuna(0, "Hepatitis B", "HB001", "GSK",
"2026-05-10"));
        dao.insertar(new Vacuna(0, "Triple Viral (SRP)", "TV001", "Serum
Institute", "2025-11-30"));

    List<Vacuna> lista =
dao.listarTodos();        assertEquals(5,
lista.size());    } }

```

8. Ejecutar las Pruebas

8.1 Desde la terminal

```

# Desde la raíz del proyecto captcha/

# Ejecutar TODAS las pruebas mvn test

# Ejecutar solo PacienteTest (unitarias) mvn test
-Dtest=PacienteTest

# Ejecutar solo las pruebas de integración (DAO)
mvn test -Dtest="*DAOTest"

# Ejecutar solo las pruebas unitarias (excluir DAO)
mvn test -Dexcludes="**/dao/*Test.java"

# Ver reporte detallado en consola mvn test
-Dsurefire.reportFormat=plain

# Resultado exitoso esperado:
[INFO] Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

```

8.2 Interpretar el reporte

```
# Prueba exitosa:
[INFO] PacienteTest > getNombreCompleto_debeUnirNombresYApellidos PASSED

# Prueba fallida — cómo leerla:
[ERROR] VacunaTest > estaVencida_conFechaFutura_debeRetornarFalse
FAILED org.opentest4j.AssertionFailedError: expected: <false> but
was: <true>
    at VacunaTest.estaVencida_conFechaFutura_debeRetornarFalse(VacunaTest.java:31)

# Lectura: esperaba false (no vencida) pero obtuvo true (vencida)
# → la lógica de estaVencida() tiene un error

# Error de integración — BD no accesible:
[ERROR] PacienteDAOTest > insertar_debeRetornarTrue FAILED
    java.lang.NullPointerException: db-test.properties no encontrado
# → verificar que el archivo está en src/test/resources/
```

9. Ejercicios Prácticos por Sesión

Distribución — 3 sesiones de laboratorio

Sesión 1 (4 h): Pruebas unitarias de DTOs — Paciente, Vacuna y Usuario.

Sesión 2 (4 h): @ParameterizedTest con ValidadorDocumento + casos límite.

Sesión 3 (4 h): Pruebas de integración con H2 — PacienteDAO y VacunaDAO.

9.1 Sesión 1 — DTOs del Sistema de Vacunación

#	Actividad
1	Agrega las dependencias de JUnit 5 y H2 al pom.xml del sistema de vacunación. Ejecuta mvn test — debe mostrar 'No tests found' pero sin errores de compilación.

2	Agrega <code>esMedico()</code> y <code>esEnfermero()</code> a <code>Usuario.java</code> y <code>estaVencida()</code> a <code>Vacuna.java</code> (sección 4). Compila con <code>mvn compile</code> .
3	Crea <code>PacienteTest.java</code> con las 6 pruebas de la sección 5.1. Ejecuta y verifica que todas pasan con barra verde.
4	Crea <code>VacunaTest.java</code> con las 5 pruebas de la sección 5.2. Presta especial atención a la prueba de vacuna con fecha null.
5	Crea <code>UsuarioTest.java</code> con las 6 pruebas de la sección 5.3. Verifica que la prueba de rol en minúsculas pasa — si falla, ajusta el método <code>esMedico()</code> .
6	RETO: Agrega a <code>RegistroVacunacion.java</code> un método <code>tieneObservaciones()</code> que retorna true si observaciones no es null ni vacío. Escribe 3 pruebas.
7	ANÁLISIS: Ejecuta las pruebas de <code>PacienteTest</code> y mide el tiempo (<code>mvn test</code>). ¿Cuántos milisegundos tardan? ¿Por qué son tan rápidas si no usan BD?
8	REFLEXIÓN: Escribe intencionalmente una prueba que falle cambiando el valor esperado en <code>assertEquals</code> . Lee el mensaje de error. ¿Qué información te da para corregir el bug?

9.2 Sesión 2 — Pruebas Parametrizadas

#	Actividad
1	Crea <code>ValidadorDocumento.java</code> en <code>src/main/java/sena/adso/captcha/util/</code> (sección 4). Compila con <code>mvn compile</code> .
2	Crea <code>ValidadorDocumentoTest.java</code> con los dos <code>@ParameterizedTest</code> de <code>@ValueSource</code> (documentos válidos e inválidos). Ejecuta — deben aparecer 10 casos individuales en el reporte.
3	Agrega la prueba de <code>@CsvSource</code> para <code>esTarjetaIdentidad()</code> con los 5 escenarios. Verifica que todos pasan.
4	Escribe los 4 casos límite explícitos (6 dígitos mínimo, 10 máximo, 5 dígitos inválido, 11 dígitos inválido). Explica en tu bitácora por qué los bordes son los más propensos a errores.

5	RETO: Agrega a ValidadorDocumento un método esFormatoCedula(String doc) que acepta formatos como '1.052.345.678' (con puntos). Escribe pruebas con @ParameterizedTest.
6	Completa la tabla de casos límite (sección 6.2) con 2 casos propios del sistema de vacunación que aún no están cubiertos.
7	COMPARACIÓN: ¿Cuántas líneas de código tendrías si no usaras @ParameterizedTest y escribieras las 10 pruebas de documentos como @Test individuales? Calcula y anota.

9.3 Sesión 3 — Pruebas de Integración

#	Actividad
1	Crea db-test.properties en src/test/resources/ y ConexionTestHelper.java (sección 7.2). Escribe una prueba mínima que solo verifica la conexión H2.
2	Adapta PacienteDAO.java para aceptar una Connection externa por constructor (sección 7.4). Verifica que el sistema sigue funcionando en Tomcat.
3	Crea PacienteDAOTest.java con las 5 pruebas de la sección 7.3. Ejecuta y verifica que todas pasan. Observa el @BeforeEach limpiando los datos.
4	Crea VacunaDAOTest.java con las 3 pruebas de la sección 7.5. La prueba de UNIQUE KEY es la más importante — verifica que el DAO maneja correctamente el error SQL.
5	RETO: Crea RegistroVacunacionDAOTest con una prueba de integración que: inserta un paciente, inserta una vacuna, inserta un registro de vacunación y verifica que listarTodos() retorna 1 registro.
6	COMPARACIÓN: Ejecuta mvn test -Dtest='*DAOTest' y anota el tiempo. Luego ejecuta solo los tests unitarios y anota el tiempo. ¿Cuántas veces más lentos son los de integración?
7	RETO AVANZADO: Agrega a PacienteDAOTest una prueba con assertTimeout() que verifique que listarTodos() con 50 pacientes responde en menos de 500ms en H2.

10. Preguntas de Reflexión

Responde en tu bitácora — mínimo 5 líneas por pregunta

Las respuestas se socializan en plenaria al cierre de la sesión 3.

10.1 Sobre Pruebas Unitarias

- En VacunaTest.java usamos @BeforeEach para crear vacunaVigente y vacunaVencida antes de cada prueba. ¿Qué pasaría si modificaras vacunaVigente en una prueba y la siguiente prueba dependiera de ese estado? ¿Cómo garantiza @BeforeEach que esto no ocurra?

Si una prueba modificara vacunaVigente y otra prueba dependiera de sus valores originales, la segunda prueba podría fallar porque estaría usando un objeto alterado por la prueba anterior. Esto generaría dependencia entre pruebas y resultados inconsistentes.

@BeforeEach evita este problema porque ejecuta nuevamente el método de inicialización antes de cada prueba, creando objetos nuevos y limpios. Así, cada prueba trabaja con su propio estado independiente y no afecta a las demás.

- La prueba estaVencida_conFechaNull_debeRetornarFalse() usa assertDoesNotThrow(). ¿Por qué es importante probar que el método no lanza una excepción inesperada? En el sistema de vacunación, ¿dónde podría ocurrir un NullPointerException no controlado si no se prueban estos casos?

Es importante porque un sistema no solo debe funcionar con datos correctos, sino también manejar datos nulos o incompletos sin caerse.

assertDoesNotThrow() verifica que el método maneje correctamente esos casos especiales y no produzca errores inesperados.

En el sistema de vacunación podría ocurrir un NullPointerException si, por ejemplo:

- una vacuna no tiene fecha de vencimiento,
- el paciente no tiene fecha de nacimiento,
- un registro llega incompleto desde la base de datos,
- o se intenta usar .isBefore() o .equals() sobre una fecha nula.

Si no se prueban estos casos, el sistema podría cerrarse al consultar vacunas o generar reportes.

- assertAll() en el constructor de Paciente ejecuta todos los assertions aunque alguno falle. Si usaras assertEquals() individual y el primer campo falla, ¿cuántos campos no verías en el reporte? ¿Cuándo usarías assertAll() en el sistema de vacunación?

Con assertAll(), aunque un assertion falle, los demás también se ejecutan. Así se muestran todos los errores encontrados en el objeto.

Si se usaran assertEquals() individuales y el primero fallara, la prueba se detendría inmediatamente y no se verían los errores de los demás campos.

Por ejemplo, si un paciente tiene errores en:

- nombre,
- documento,
- correo,
- y teléfono, solo se reportaría el primero.

`assertAll()` es útil cuando se validan muchos atributos relacionados, como:

- creación de pacientes,
- datos de vacunas,
- información de citas,
- o respuestas completas del DAO

10.2 Sobre Pruebas Parametrizadas

- `@ParameterizedTest` con 10 documentos genera 10 casos de prueba desde un solo método. Imagina que el registrador de Nobsa te informa que ahora deben aceptar documentos de 11 dígitos para extranjeros. ¿Cuántas líneas de código cambiarías en `ValidadorDocumentoTest`? ¿Y si hubieran sido 10 métodos `@Test` separados?

Con `@ParameterizedTest` solo habría que agregar un nuevo caso de prueba, por ejemplo:

`"12345678901"`

Prácticamente sería cambiar una sola línea.

En cambio, si existieran 10 métodos `@Test` separados, probablemente habría que:

- crear otro método,
- copiar código,
- agregar assertions,
- y mantener más código repetido.

Las pruebas parametrizadas facilitan muchísimo el mantenimiento.

- Los casos límite (6 dígitos mínimo, 10 máximo, etc.) son donde más errores se esconden. En el sistema de vacunación, ¿cuál es el caso límite más crítico en términos de impacto para el paciente? (Pista: piensa en fechas de vencimiento de vacunas).

El caso límite más crítico es la fecha de vencimiento de una vacuna.

Por ejemplo:

- una vacuna que vence exactamente hoy,
- o una vacuna vencida por un día.

Un error en ese cálculo podría provocar:

- aplicar vacunas vencidas,
- rechazar vacunas válidas,
- afectar tratamientos,
- o poner en riesgo la salud del paciente.

Los errores de fechas son especialmente peligrosos en sistemas médicos.

10.3 Sobre Pruebas de Integración

- La prueba `insertar_conDocumentoDuplicado_debeFallar()` verifica que el DAO retorna false cuando viola la restricción UNIQUE. En el sistema de vacunación en producción, ¿qué mostraría el usuario si este error no estuviera manejado? ¿Cómo mejorarías el mensaje de error al usuario?

Si el error no estuviera manejado, el usuario probablemente vería:

- un error técnico,
- una excepción SQL,
- o incluso una página blanca del servidor. Algo como:

Duplicate entry '1234567890' for key 'documento'

Eso es confuso para un usuario normal.

Un mejor mensaje sería:

El documento ya se encuentra registrado en el sistema. Por favor verifique la información ingresada.

Así el usuario entiende claramente qué ocurrió y cómo solucionarlo.

- Usamos H2 en lugar de MySQL para las pruebas de integración. Tu compañero propone conectar las pruebas directamente a la BD MySQL del sistema de vacunación en producción. Menciona exactamente 3 problemas que podrían ocurrir en la clase de hoy si hiciera eso.
 - 1.Pérdida o alteración de datos reales**
Las pruebas podrían borrar, modificar o insertar pacientes reales accidentalmente.
 - 2.Resultados inconsistentes**
Si otros usuarios usan el sistema mientras corren las pruebas, los datos cambiarían constantemente y las pruebas podrían fallar aleatoriamente.
 - 3.Riesgo de rendimiento y bloqueos**
Las pruebas podrían consumir recursos de la base de datos y afectar el funcionamiento normal del sistema en producción.
- `@BeforeEach` ejecuta `DELETE FROM pacientes` antes de cada prueba. ¿Por qué es crítico este paso en `PacienteDAOTest`? Escribe el escenario exacto donde la prueba `listarTodos_debeRetornarPacientesInsertados()` fallaría si no limpiaras antes.

Es crítico porque garantiza que cada prueba empiece con una base de datos limpia y controlada.

Escenario exacto donde fallaría `listarTodos_debeRetornarPacientesInsertados()`

Supongamos que una prueba anterior insertó 3 pacientes y no los eliminó.

Luego `listarTodos_debeRetornarPacientesInsertados()` inserta 1 paciente esperando obtener:

```
assertEquals(1, lista.size());
```

Pero realmente la tabla tendría:

3 pacientes anteriores + 1 nuevo = 4

Entonces la prueba fallaría porque recibiría 4 registros en vez de 1.

Por eso limpiar la tabla antes de cada prueba asegura resultados confiables y repetibles.

10.4 Prueba Final

Aplicación de Citas Médicas

Aplique todo lo aprendido en la aplicación construida.

11. Errores Frecuentes y Soluciones

Error / Síntoma	Causa y Solución
No tests were found en el proyecto de vacunación	Maven no detecta las pruebas. El archivo de prueba debe terminar en Test.java y estar en src/test/java/ (no en src/main/).
estaVencida() NullPointerException con fecha null	LocalDate.parse(null) lanza excepción. Agregar: if (fechaVencimiento == null fechaVencimiento.isBlank()) return false; al inicio del método.
db-test.properties not found (H2 no conecta)	El archivo está en la ruta incorrecta. Debe estar en src/test/resources/ (no en src/main/resources/). Verifica con: ls src/test/resources/
Table 'pacientes' not found en la prueba de integración	El @BeforeAll que crea la tabla no se ejecutó. Verifica que sea static void y que la anotación sea de org.junit.jupiter.api.
La prueba de UNIQUE KEY pasa aunque inserte el duplicado	El PacienteDAO captura la excepción y retorna false — ese es el comportamiento correcto. Si retorna true con duplicado, hay un bug real en el DAO.

Pruebas pasan solas pero fallan en conjunto	Dependencia de estado entre pruebas: una prueba modifica datos que otra necesita limpios. Usar <code>@BeforeEach</code> para resetear.
<code>esMedico()</code> retorna false con rol en minúsculas	La comparación usa <code>==</code> en lugar de <code>equalsIgnoreCase()</code> . Cambiar: <code>return "MEDICO".equalsIgnoreCase(rol);</code>
<code>@ParameterizedTest</code> no ejecuta todos los casos	Falta el import: <code>import org.junit.jupiter.params.ParameterizedTest;</code> y <code>import org.junit.jupiter.params.provider.ValueSource;</code>

12. Criterios de Evaluación

#	Criterio de Desempeño	Puntos	Instrumento
1	PacienteTest.java: mínimo 5 pruebas incluyendo <code>assertAll</code> y <code>@Nested</code> — todas pasan	20 %	mvn test
2	VacunaTest.java: prueba de <code>estaVencida()</code> con fecha futura, pasada y null — todas pasan	15 %	mvn test
3	UsuarioTest.java: <code>esMedico()</code> y <code>esEnfermero()</code> incluyendo prueba con rol en minúsculas	10 %	mvn test
4	ValidadorDocumentoTest.java: <code>@ParameterizedTest</code> con mínimo 5 documentos válidos y 5 inválidos	15 %	Revisión código
5	PacienteDAOTest.java: 5 pruebas de integración con H2 incluyendo UNIQUE — todas pasan	20 %	mvn test
6	VacunaDAOTest.java: 3 pruebas incluyendo la de lote duplicado — todas pasan	10 %	mvn test
7	Preguntas de reflexión respondidas (sección 10)	10 %	Lista de chequeo
#	Criterio de Desempeño	Puntos	Instrumento
BONUS A	RegistroVacunacionDAOTest: flujo completo paciente+vacuna+registro	+0,3	mvn test
BONUS B	<code>assertTimeout()</code> verificando rendimiento de <code>listarTodos()</code> con 50 registros	+0,2	Demostración

Condiciones de no aprobación automática

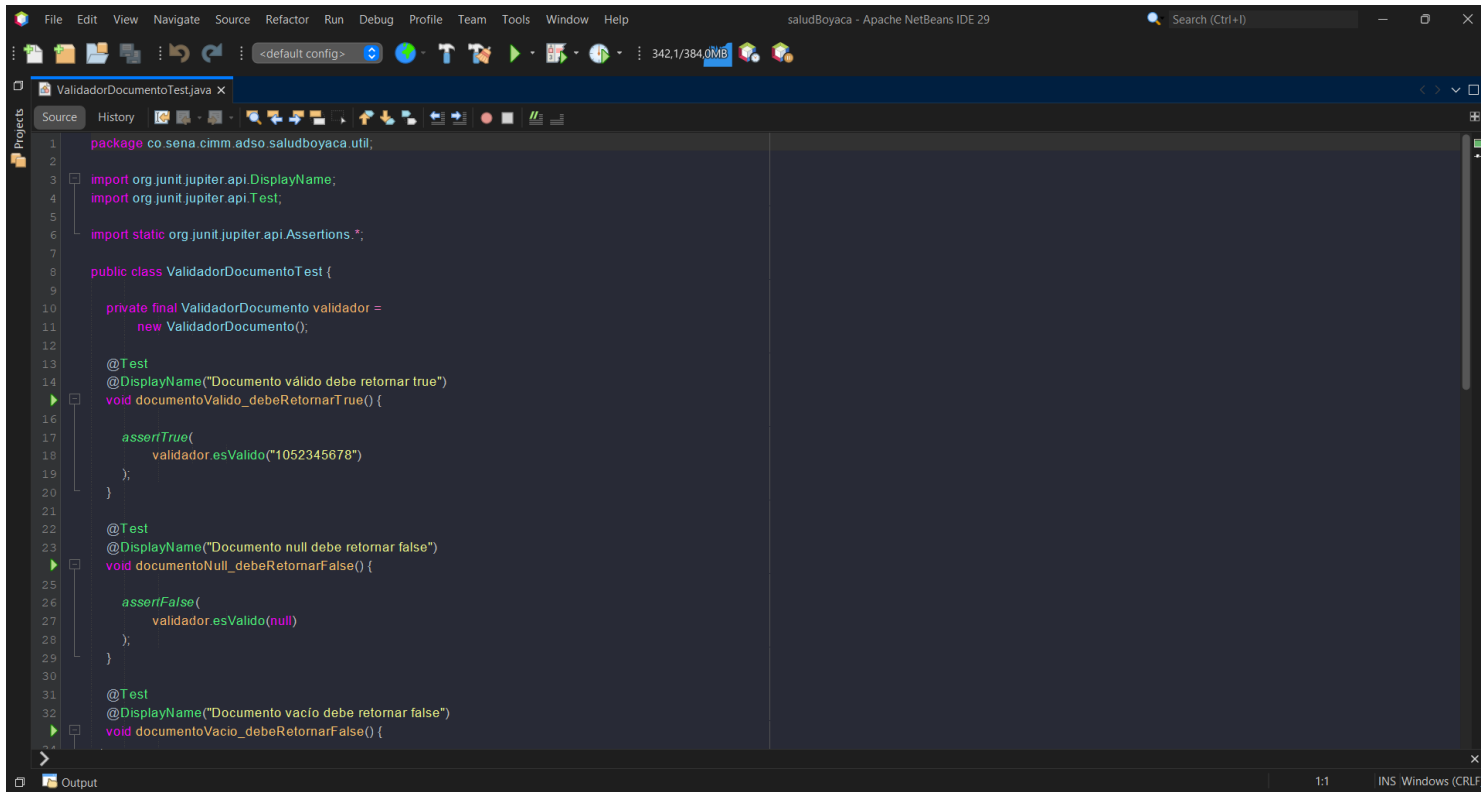
- Pruebas que siempre pasan sin importar el código (assertions vacíos o triviales).

- Pruebas de integración que apuntan a MySQL de producción (no a H2).

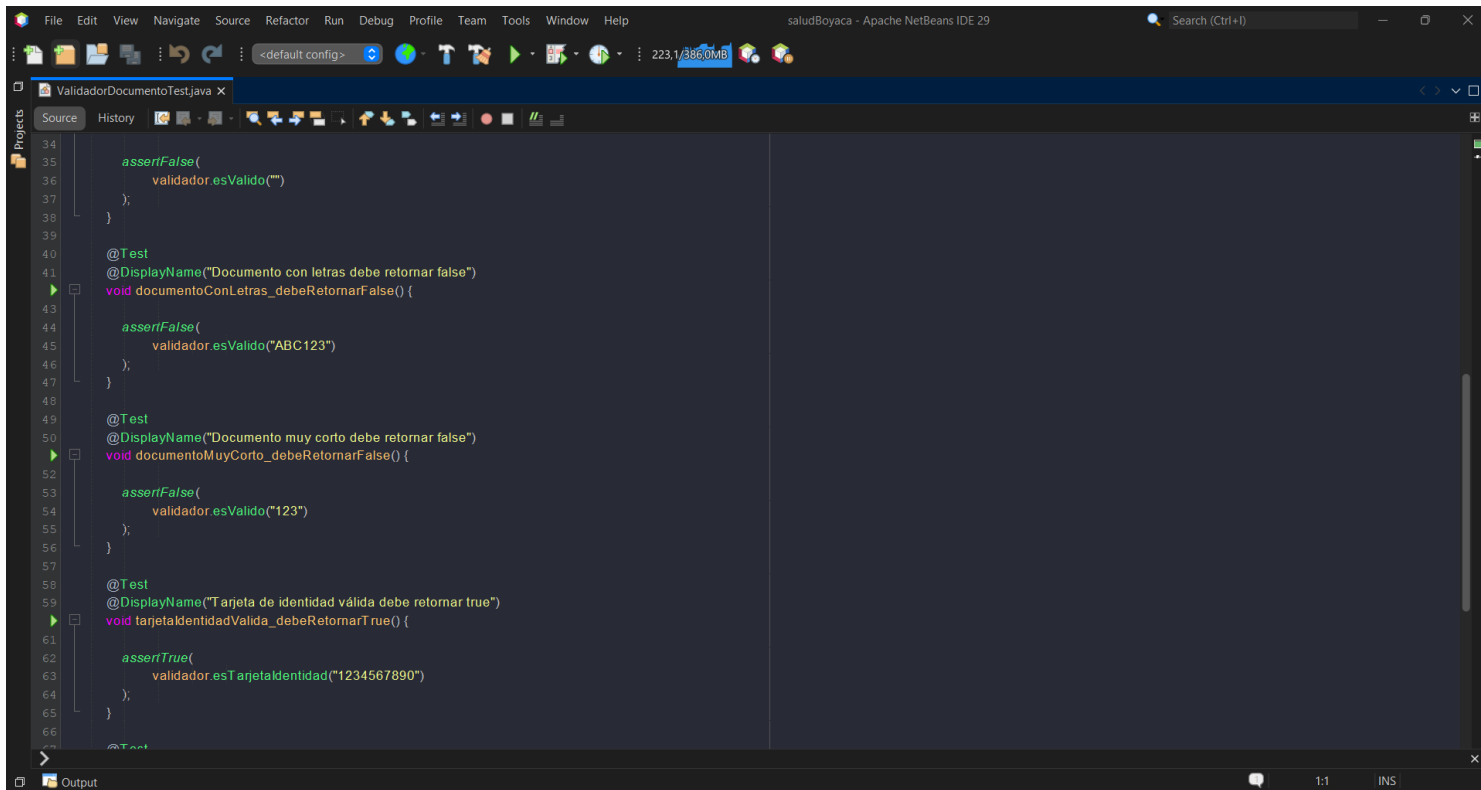
- Pruebas con nombres genéricos test1(), test2() sin @DisplayName descriptivo.

- Pruebas que dependen del orden de ejecución sin @TestMethodOrder.

- Menos de 5 pruebas en cualquier clase marcada como obligatoria.



```
1 package co.sena.cimm.adso.saludboyaca.util;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 public class ValidatorDocumentoTest {
9
10     private final ValidatorDocumento validator =
11         new ValidatorDocumento();
12
13     @Test
14     @DisplayName("Documento válido debe retornar true")
15     void documentoValido_debeRetornarTrue() {
16
17         assertTrue(
18             validator.esValido("1052345678")
19         );
20     }
21
22     @Test
23     @DisplayName("Documento null debe retornar false")
24     void documentoNull_debeRetornarFalse() {
25
26         assertFalse(
27             validator.esValido(null)
28         );
29     }
30
31     @Test
32     @DisplayName("Documento vacío debe retornar false")
33     void documentoVacio_debeRetornarFalse() {
34
35         assertFalse(
36             validator.esValido("")
37         );
38     }
39
40     @Test
41     @DisplayName("Documento con letras debe retornar false")
42     void documentoConLetras_debeRetornarFalse() {
43
44         assertFalse(
45             validator.esValido("ABC123")
46         );
47     }
48
49     @Test
50     @DisplayName("Documento muy corto debe retornar false")
51     void documentoMuyCorto_debeRetornarFalse() {
52
53         assertFalse(
54             validator.esValido("123")
55         );
56     }
57
58     @Test
59     @DisplayName("Tarjeta de identidad válida debe retornar true")
60     void tarjetaIdentidadValida_debeRetornarTrue() {
61
62         assertTrue(
63             validator.esTarjetaIdentidad("1234567890")
64         );
65     }
66
67     @Test
68     @DisplayName("Tarjeta de identidad inválida debe retornar false")
69     void tarjetaIdentidadInvalida_debeRetornarFalse() {
70
71         assertFalse(
72             validator.esTarjetaIdentidad("12345678901")
73         );
74     }
75 }
```



```
34
35     assertFalse(
36         validator.esValido("")
37     );
38 }
39
40 @Test
41 @DisplayName("Documento con letras debe retornar false")
42 void documentoConLetras_debeRetornarFalse() {
43
44     assertFalse(
45         validator.esValido("ABC123")
46     );
47 }
48
49 @Test
50 @DisplayName("Documento muy corto debe retornar false")
51 void documentoMuyCorto_debeRetornarFalse() {
52
53     assertFalse(
54         validator.esValido("123")
55     );
56 }
57
58 @Test
59 @DisplayName("Tarjeta de identidad válida debe retornar true")
60 void tarjetaIdentidadValida_debeRetornarTrue() {
61
62     assertTrue(
63         validator.esTarjetaIdentidad("1234567890")
64     );
65 }
66
67 @Test
68 @DisplayName("Tarjeta de identidad inválida debe retornar false")
69 void tarjetaIdentidadInvalida_debeRetornarFalse() {
70
71     assertFalse(
72         validator.esTarjetaIdentidad("12345678901")
73     );
74 }
75 }
```

saludBoyaca - Apache NetBeans IDE 29

Search (Ctrl+I)

ValidadorDocumentoTest.java

```
54     validador.esValido("123")
55     };
56 }
57
58 @Test
59 @DisplayName("Tarjeta de identidad válida debe retornar true")
60 void tarjetaIdentidadValida_debeRetornarTrue() {
61
62     assertTrue(
63         validador.esTarjetaIdentidad("1234567890")
64     );
65 }
66
67 @Test
68 @DisplayName("Documento normal no debe ser tarjeta identidad")
69 void documentoNormal_noDebeSerTarjetaIdentidad() {
70
71     assertFalse(
72         validador.esTarjetaIdentidad("987654321")
73     );
74 }
75 }
```

saludBoyaca - Apache NetBeans IDE 29

Search (Ctrl+I)

UsuarioTest.java

```
1 package co.sena.cimm.adso.saludboyaca.model;
2
3 import org.junit.jupiter.api.BeforeEach;
4 import org.junit.jupiter.api.DisplayName;
5 import org.junit.jupiter.api.Test;
6
7 import static org.junit.jupiter.api.Assertions.*;
8
9 public class UsuarioTest {
10
11     private Usuarios medico;
12     private Usuarios enfermero;
13
14     @BeforeEach
15     void setup() {
16
17         medico = new Usuarios(
18             1,
19             "Admin",
20             "Sistema",
21             "1000000000",
22             "admin@example.com",
23             "admin",
24             "admin123",
25             "MEDICO",
26             "Cardiología",
27             "es",
28             1
29         );
30
31         enfermero = new Usuarios(
32             2,
33             "María",
34             "Enfermería",
35             "2000000000",
36             "maria@example.com",
37             "maria",
38             "maria123",
39             "ENFERMERO",
40             "Enfermería",
41             "es",
42             2
43         );
44     }
45 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
34 "López",
35 "52345678",
36 "maria@hospital.com",
37 "mlopez",
38 "enf123",
39 "ENFERMERO",
40 null,
41 "es",
42 1
43 };
44 }
45
46 @Test
47 @DisplayName("esMedico() debe retornar true")
48 void esMedico_conRolMedico_debeRetomarTrue() {
49
50     assertTrue(
51         medico.esMedico()
52     );
53 }
54
55 @Test
56 @DisplayName("esMedico() debe retornar false")
57 void esMedico_conRolEnfermero_debeRetomarFalse() {
58
59     assertFalse(
60         enfermero.esMedico()
61     );
62 }
63
64 @Test
65 @DisplayName("esEnfermero() debe retornar true")
66 void esEnfermero_conRolEnfermero_debeRetomarTrue() {
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
67     assertTrue(
68         enfermero.esEnfermero()
69     );
70 }
71
72
73 @Test
74 @DisplayName("esMedico() debe ignorar mayúsculas")
75 void esMedico_debeSerInsensibleAMayusculas() {
76
77     medico.setRol("medico");
78
79     assertTrue(
80         medico.esMedico()
81     );
82 }
83
84 @Test
85 @DisplayName("getNombreCompleto() debe unir nombres y apellidos")
86 void getNombreCompleto_paraAdmin() {
87
88     assertEquals(
89         "Admin Sistema",
90         medico.getNombreCompleto()
91     );
92 }
93
94 @Test
95 @DisplayName("Usuario sin rol no debe ser médico ni enfermero")
96 void usuarioSinRol_noEsMedicoNiEnfermero() {
97
98     medico.setRol(null);
99
100     assertFalse(
```

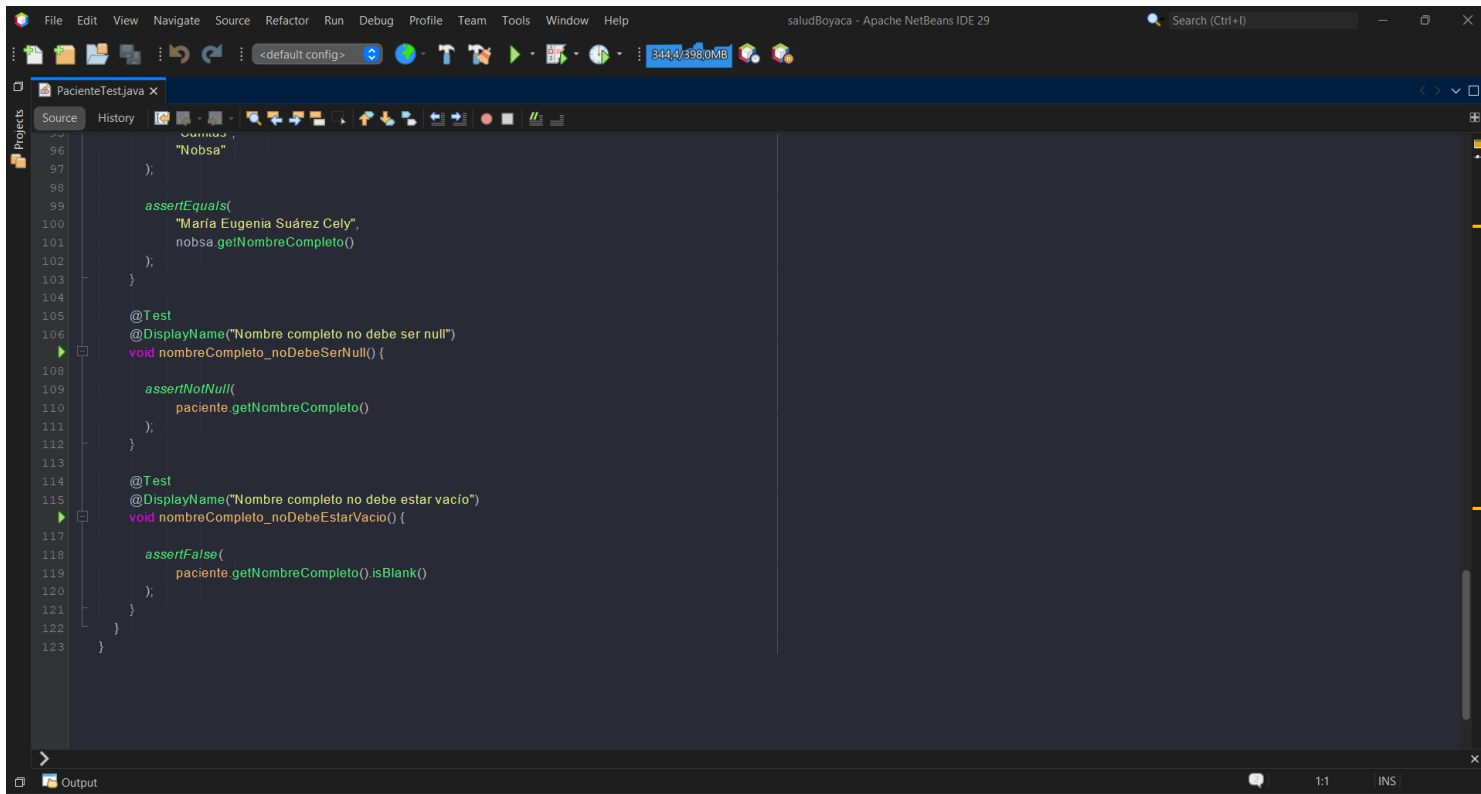
```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 350.4/390.0MB
UsuarioTest.java X
Source History
87
88
89 assertEquals(
90     "Admin Sistema",
91     medico.getNombreCompleto()
92 );
93 }
94
95 @Test
96 @DisplayName("Usuario sin rol no debe ser médico ni enfermero")
97 void usuarioSinRol_noEsMedicoNiEnfermero() {
98
99     medico.setRol(null);
100
101     assertFalse(
102         medico.esMedico()
103     );
104
105     assertFalse(
106         medico.esEnfermero()
107     );
108 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 250.8/390.0MB
PacienteTest.java X
Source History
1 package co.sena.cimm.adso.saludboyaca.model;
2
3 import org.junit.jupiter.api.BeforeEach;
4 import org.junit.jupiter.api.DisplayName;
5 import org.junit.jupiter.api.Nested;
6 import org.junit.jupiter.api.Test;
7
8 import java.time.LocalDate;
9
10 import static org.junit.jupiter.api.Assertions.*;
11
12 public class PacienteTest {
13
14     private Pacientes paciente;
15
16     @BeforeEach
17     void setUp() {
18
19         paciente = new Pacientes(
20             1,
21             "Carlos Ernesto",
22             "Pedraza Rondón",
23             "1052345678",
24             LocalDate.of(1985, 3, 12),
25             "3123456789",
26             "carlos@gmail.com",
27             "Nueva EPS",
28             "Nobsa"
29         );
30     }
31
32     @Test
33     @DisplayName("getNombreCompleto() debe unir nombres y apellidos")
34     void getNombreCompleto_debeUnirNombresYApellidos() {
```



```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
286.8/398.0MB
PacienteTest.java X
Source History
32 @Test
33 @DisplayName("getNombreCompleto() debe unir nombres y apellidos")
34 void getNombreCompleto_debeUnirNombresYApellidos() {
35
36     String resultado = paciente.getNombreCompleto();
37
38     assertEquals(
39         "Carlos Ernesto Pedraza Rondón",
40         resultado
41     );
42 }
43
44 @Test
45 @DisplayName("Constructor debe inicializar todos los campos")
46 void constructor_debeInicializarTodosLosCampos() {
47
48     assertEquals(
49         () -> assertEquals(1, paciente.getId()),
50         () -> assertEquals("Carlos Ernesto", paciente.getNombres()),
51         () -> assertEquals("Pedraza Rondón", paciente.getApellidos()),
52         () -> assertEquals("1052345678", paciente.getDocumento()),
53         () -> assertNotNull(paciente.getFechaNacimiento())
54     );
55 }
56
57 @Test
58 @DisplayName("setDocumento() debe actualizar documento")
59 void setDocumento_debeActualizarDocumento() {
60
61     paciente.setDocumento("80456789");
62
63     assertEquals(
64         "80456789",
65         paciente.getDocumento()
66     );
67 }
68
69 @Test
70 @DisplayName("Constructor vacío debe crear paciente con id 0")
71 void constructorVacio_debeCrearPacienteConIdCero() {
72
73     Pacientes vacio = new Pacientes();
74
75     assertEquals(0, vacio.getId());
76     assertNull(vacio.getNombres());
77 }
78
79 @Nested
80 @DisplayName("Pruebas de nombre completo")
81 class NombreCompletoTests {
82
83     @Test
84     @DisplayName("Nombre completo de paciente de Nobsa")
85     void pacienteNobsa_nombreCompleto() {
86
87         Pacientes nobsa = new Pacientes(
88             2,
89             "María Eugenia",
90             "Suárez Cely",
91             "1052345679",
92             LocalDate.of(1992, 7, 24),
93             "30000000000",
94             "maria@gmail.com",
95             "Sanitas",
96             "Nobsa"
97         );
98     }
99 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
266.3/398.0MB
PacienteTest.java X
Source History
65 paciente.getDocumento()
66 );
67 }
68
69 @Test
70 @DisplayName("Constructor vacío debe crear paciente con id 0")
71 void constructorVacio_debeCrearPacienteConIdCero() {
72
73     Pacientes vacio = new Pacientes();
74
75     assertEquals(0, vacio.getId());
76     assertNull(vacio.getNombres());
77 }
78
79 @Nested
80 @DisplayName("Pruebas de nombre completo")
81 class NombreCompletoTests {
82
83     @Test
84     @DisplayName("Nombre completo de paciente de Nobsa")
85     void pacienteNobsa_nombreCompleto() {
86
87         Pacientes nobsa = new Pacientes(
88             2,
89             "María Eugenia",
90             "Suárez Cely",
91             "1052345679",
92             LocalDate.of(1992, 7, 24),
93             "30000000000",
94             "maria@gmail.com",
95             "Sanitas",
96             "Nobsa"
97         );
98     }
99 }
```



```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 231.5/398.0MB

OTP_TokensTest.java x
Source History
1 /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5 package co.sena.cimm.adso.saludboyaca.model;
6
7 import java.time.LocalDateTime;
8 import java.time.LocalDateTime;
9 import org.junit.jupiter.api.DisplayName;
10 import org.junit.jupiter.api.Test;
11 import static org.junit.jupiter.api.Assertions.*;
12
13 /**
14  *
15  * @author julia
16  */
17 public class OTP_TokensTest {
18
19     @Test
20     @DisplayName("Constructor vacio debe crear objeto")
21     void constructorVacio_debeCrearObjeto() {
22
23         OTP_Tokens token = new OTP_Tokens();
24
25         assertNotNull(token);
26     }
27
28     @Test
29     @DisplayName("Constructor debe inicializar todos los campos")
30     void constructor_debeInicializarCampos() {
31
32         LocalDateTime fechaGeneracion
33             = LocalDateTime.of(2026, 5, 11, 10, 0);
34     }
35 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 358.3/398.0MB

OTP_TokensTest.java x
Source History
34
35     LocalDateTime fechaExpiracion
36         = LocalDateTime.of(2026, 5, 11, 10, 5);
37
38     OTP_Tokens token = new OTP_Tokens(
39         1,
40         10,
41         "ABC123",
42         fechaGeneracion,
43         fechaExpiracion,
44         0
45     );
46
47     assertEquals(
48         () -> assertEquals(1, token.getId()),
49         () -> assertEquals(
50             10,
51             token.getId_usuario()
52         ),
53         () -> assertEquals(
54             "ABC123",
55             token.getCodigo()
56         ),
57         () -> assertEquals(
58             fechaGeneracion,
59             token.getFecha_gen()
60         ),
61         () -> assertEquals(
62             fechaExpiracion,
63             token.getExpira_en()
64         ),
65         () -> assertEquals(
66             0,
67             token.getEstado()
68         )
69     );
70 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 257.3/398.0MB

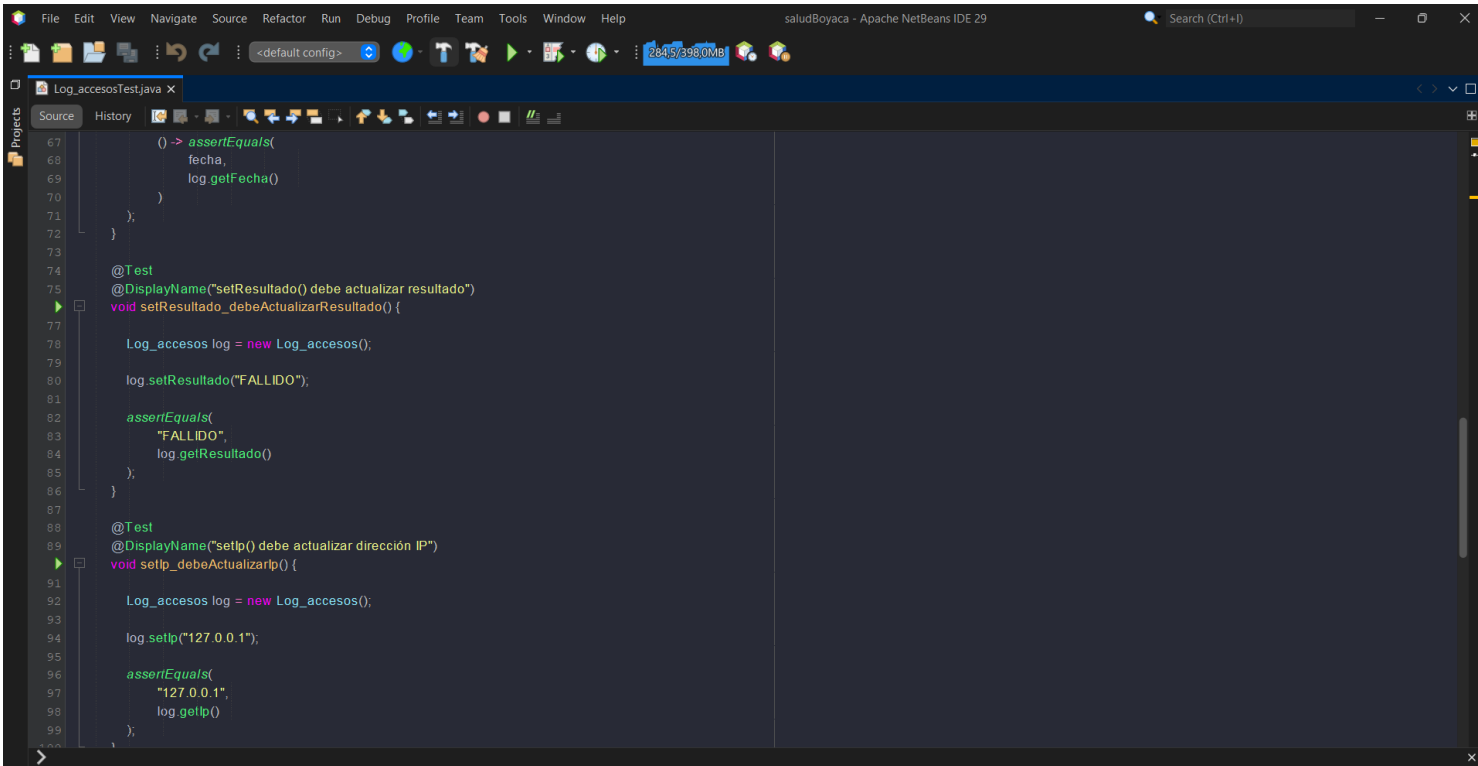
OTP_TokensTest.java x
Source History
67 token.getUsado()
68 }
69 };
70 }
71 }
72
73 @Test
74 @DisplayName("setCodigo() debe actualizar código")
75 void setCodigo_debeActualizarCodigo(){
76
77     OTP_Tokens token = new OTP_Tokens();
78
79     token.setCodigo("999999");
80
81     assertEquals(
82         "999999",
83         token.getCodigo()
84     );
85 }
86
87 @Test
88 @DisplayName("Token no debe estar expirado")
89 void token_noDebeEstarExpirado(){
90
91     OTP_Tokens token = new OTP_Tokens();
92
93     token.setExpira_en(
94         LocalDateTime.now().plusMinutes(5)
95     );
96
97     assertTrue(
98         token.getExpira_en().isAfter(LocalDateTime.now())
99     );
100 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 329.9/398.0MB

OTP_TokensTest.java x
Source History
97 assertTrue(
98     token.getExpira_en().isAfter(LocalDateTime.now())
99 );
100 }
101
102 @Test
103 @DisplayName("Token expirado debe ser anterior a fecha actual")
104 void tokenExpirado_debeSerAnteriorALaFechaActual(){
105
106     OTP_Tokens token = new OTP_Tokens();
107
108     token.setExpira_en(
109         LocalDateTime.now().minusMinutes(5)
110     );
111
112     assertTrue(
113         token.getExpira_en().isBefore(LocalDateTime.now())
114     );
115 }
116
117 @Test
118 @DisplayName("setUsado() debe actualizar estado")
119 void setUsado_debeActualizarEstado(){
120
121     OTP_Tokens token = new OTP_Tokens();
122
123     token.setUsado(1);
124
125     assertEquals(
126         1,
127         token.getUsado()
128     );
129 }
130 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
Log_accesosTest.java x
Source History
1 /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5 package co.sena.cimm.adso.saludboyaca.model;
6
7 /**
8  *
9  * @author julia
10 */
11 import java.time.LocalDateTime;
12 import java.time.LocalDate;
13 import static org.junit.jupiter.api.Assertions.*;
14 import org.junit.jupiter.api.DisplayName;
15 import org.junit.jupiter.api.Test;
16
17 public class Log_accesosTest {
18
19     @Test
20     @DisplayName("Constructor vacío debe crear objeto")
21     void constructorVacío_debeCrearObjeto() {
22
23         Log_accesos log = new Log_accesos();
24
25         assertNotNull(log);
26     }
27
28     @Test
29     @DisplayName("Constructor debe inicializar todos los campos")
30     void constructor_debeInicializarCampos() {
31
32         LocalDateTime fecha
33             = LocalDateTime.of(2026, 5, 11, 14, 30);
34     }
35 }
```

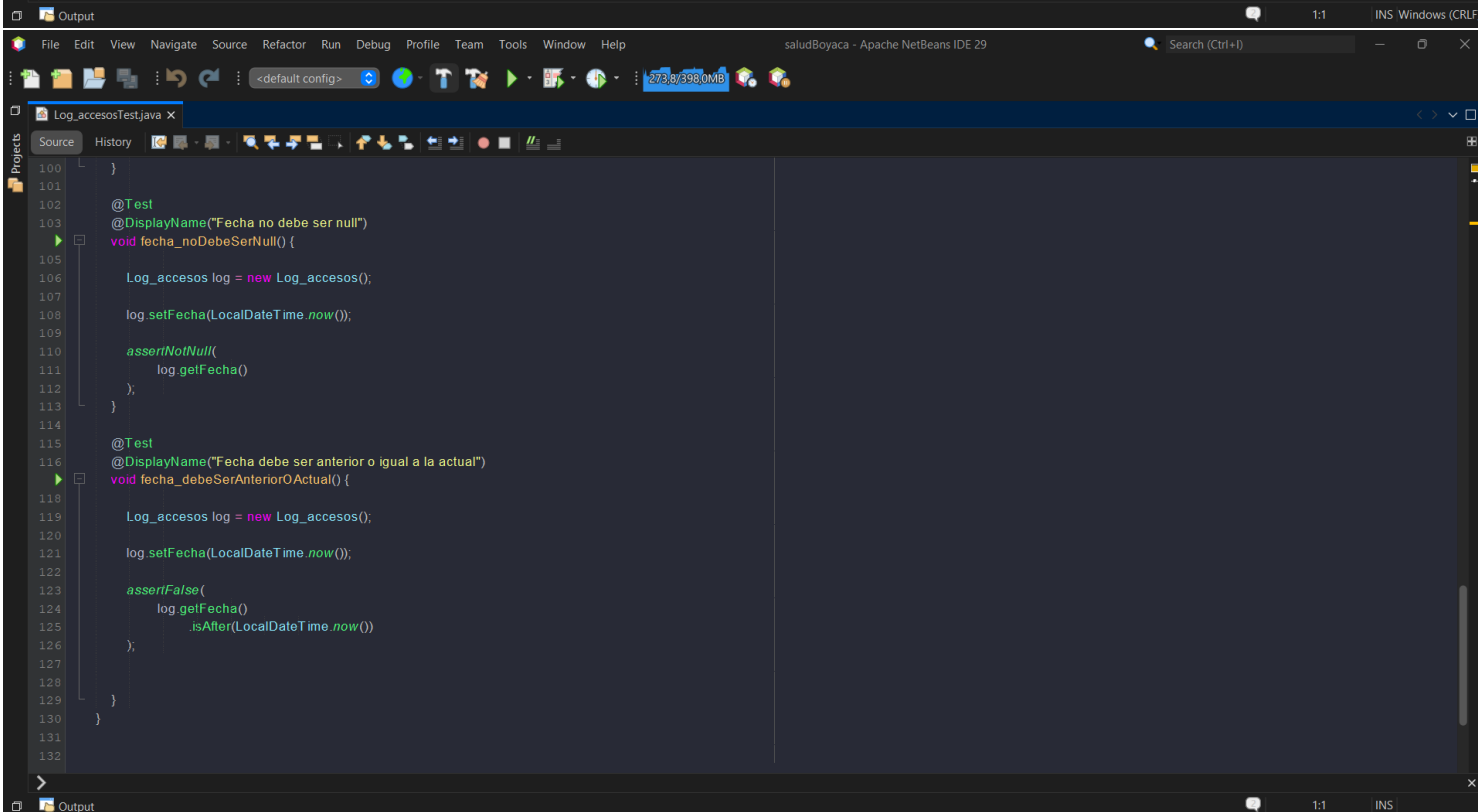
```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
Log_accesosTest.java x
Source History
34
35     Log_accesos log = new Log_accesos(
36         1,
37         10,
38         "admin",
39         "LOGIN",
40         "192.168.1.18",
41         "EXITOSO",
42         fecha
43     );
44
45     assertEquals(
46         () -> assertEquals(1, log.getId()),
47         () -> assertEquals(
48             10,
49             log.getId_usuario()
50         ),
51         () -> assertEquals(
52             "admin",
53             log.getUsername()
54         ),
55         () -> assertEquals(
56             "LOGIN",
57             log.getAccion()
58         ),
59         () -> assertEquals(
60             "192.168.1.18",
61             log.getIp()
62         ),
63         () -> assertEquals(
64             "EXITOSO",
65             log.getResultado()
66         ),
67         () -> assertEquals(
68             fecha,
69             log.getFecha()
70         )
71     );
72 }
```



SaludBoyaca - Apache NetBeans IDE 29

Log_accesosTest.java

```
67    () -> assertEquals(
68        fecha,
69        log.getFecha()
70    );
71    };
72    }
73
74    @Test
75    @DisplayName("setResultado() debe actualizar resultado")
76    void setResultado_debeActualizarResultado() {
77
78        Log_accesos log = new Log_accesos();
79
80        log.setResultado("FALLIDO");
81
82        assertEquals(
83            "FALLIDO",
84            log.getResultado()
85        );
86    }
87
88    @Test
89    @DisplayName("setIp() debe actualizar dirección IP")
90    void setIp_debeActualizarIp() {
91
92        Log_accesos log = new Log_accesos();
93
94        log.setIp("127.0.0.1");
95
96        assertEquals(
97            "127.0.0.1",
98            log.getIp()
99        );
100    }
```



SaludBoyaca - Apache NetBeans IDE 29

Log_accesosTest.java

```
100    }
101
102    @Test
103    @DisplayName("Fecha no debe ser null")
104    void fecha_noDebeSerNull() {
105
106        Log_accesos log = new Log_accesos();
107
108        log.setFecha(LocalDate.now());
109
110        assertNotNull(
111            log.getFecha()
112        );
113    }
114
115    @Test
116    @DisplayName("Fecha debe ser anterior o igual a la actual")
117    void fecha_debeSerAnteriorOActual() {
118
119        Log_accesos log = new Log_accesos();
120
121        log.setFecha(LocalDate.now());
122
123        assertFalse(
124            log.getFecha()
125                .isAfter(LocalDate.now())
126        );
127    }
128
129    }
130
131    }
132
133    }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 292.8/398.0MB

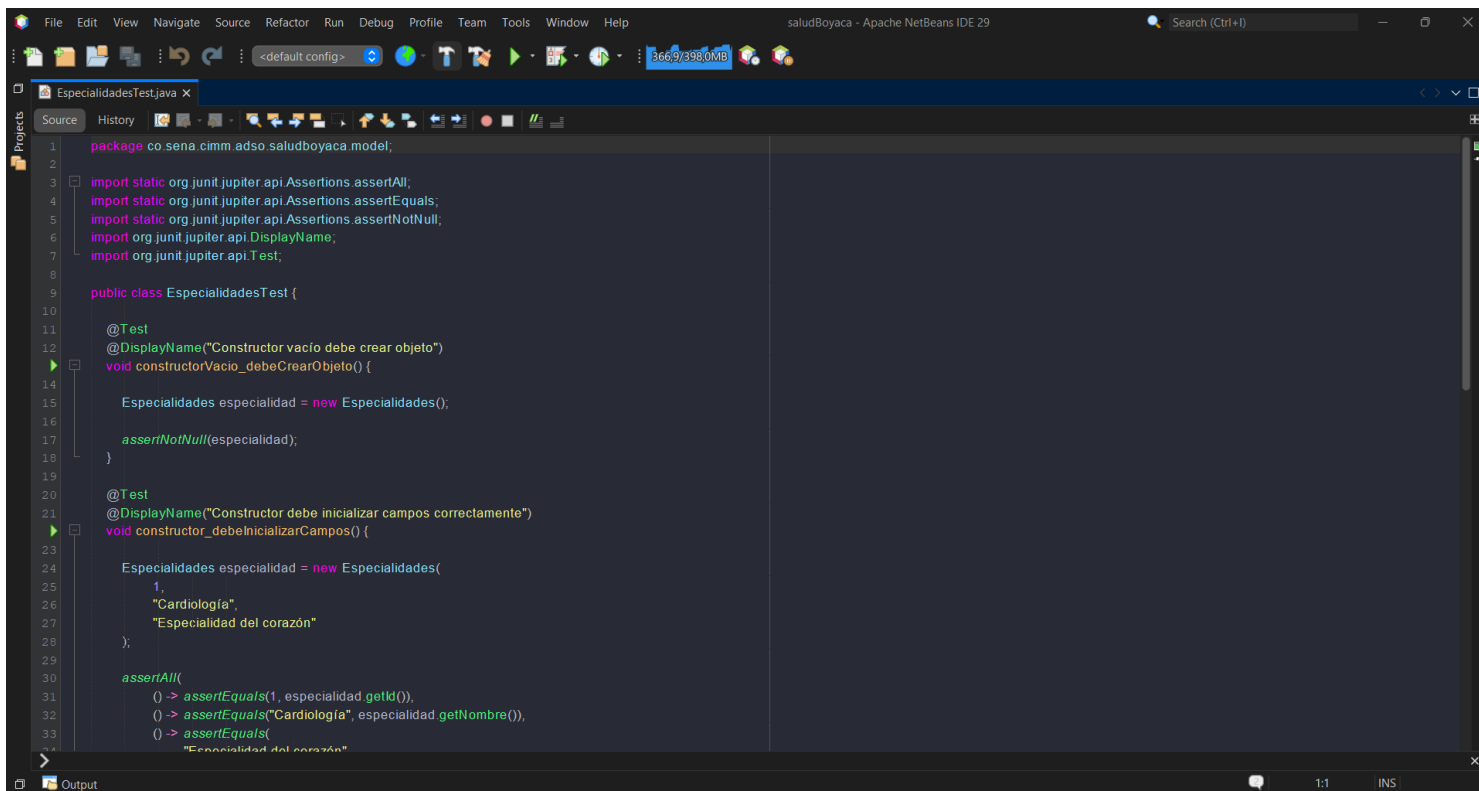
HorariosTest.java x
Source History
1 2
3  * Click pbfs://nhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
4  * Click pbfs://nhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
5  */
6  package co.sena.cimm.adso.saludboyaca.model;
7
8  import java.sql.Time;
9  import org.junit.jupiter.api.DisplayName;
10 import org.junit.jupiter.api.Test;
11
12 import static org.junit.jupiter.api.Assertions.*;
13
14 /**
15  * @author julia
16  */
17 public class HorariosTest {
18
19     @Test
20     @DisplayName("Constructor vacío debe crear objeto")
21     void constructorVacío_debeCrearObjeto() {
22
23         Horarios horario = new Horarios();
24
25         assertNotNull(horario);
26     }
27
28     @Test
29     @DisplayName("Constructor debe inicializar todos los campos")
30     void constructor_debeInicializarCampos() {
31
32         Horarios horario = new Horarios(
33             1,
34             10,
35             1,
36             Time.valueOf("08:00:00"),
37             Time.valueOf("12:00:00"),
38             20
39         );
40
41         assertEquals(1, horario.getId());
42         assertEquals(10, horario.getId_medico());
43         assertEquals(1, horario.getId_semana());
44         assertEquals(
45             Time.valueOf("08:00:00"),
46             horario.getHora_inicio()
47         );
48         assertEquals(
49             Time.valueOf("12:00:00"),
50             horario.getHora_fin()
51         );
52         assertEquals(
53             20,
54             horario.getMax_citas()
55         );
56     }
57
58     @Test
59     @DisplayName("setHora_inicio() debe actualizar hora")
60     void setHoraInicio_debeActualizarHora() {
61
62         Horarios horario = new Horarios();
63         horario.setHora_inicio(
64             Time.valueOf("08:00:00")
65         );
66     }
67 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 398.4/398.0MB

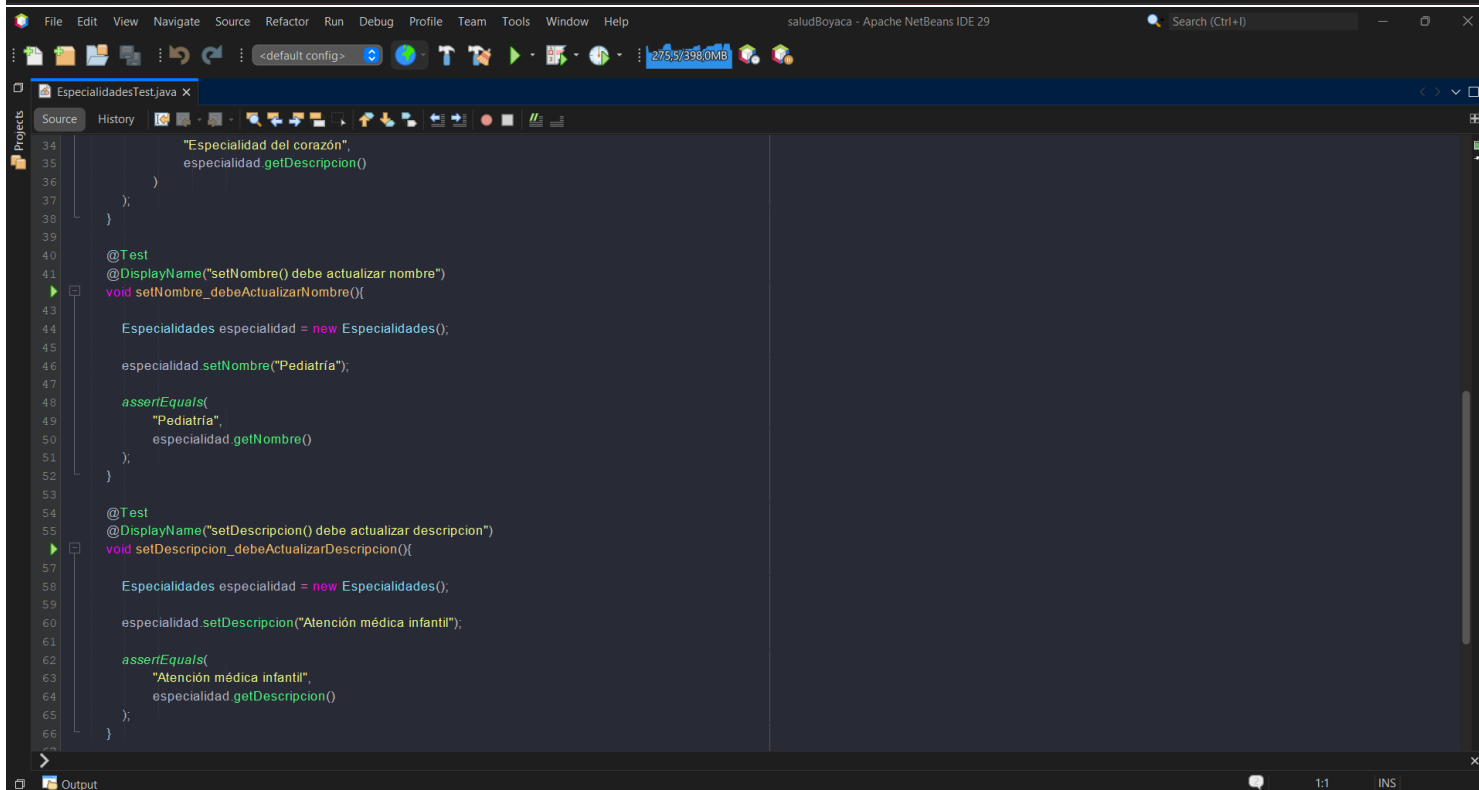
HorariosTest.java x
Source History
34 10,
35 1,
36 Time.valueOf("08:00:00"),
37 Time.valueOf("12:00:00"),
38 20
39 );
40
41 assertEquals(
42     () -> assertEquals(1, horario.getId()),
43     () -> assertEquals(10, horario.getId_medico()),
44     () -> assertEquals(1, horario.getId_semana()),
45     () -> assertEquals(
46         Time.valueOf("08:00:00"),
47         horario.getHora_inicio()
48     ),
49     () -> assertEquals(
50         Time.valueOf("12:00:00"),
51         horario.getHora_fin()
52     ),
53     () -> assertEquals(
54         20,
55         horario.getMax_citas()
56     )
57 );
58
59 }
60
61 @Test
62 @DisplayName("setHora_inicio() debe actualizar hora")
63 void setHoraInicio_debeActualizarHora() {
64
65     Horarios horario = new Horarios();
66     horario.setHora_inicio(
67         Time.valueOf("08:00:00")
68     );
69 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 367.2/398.0MB
Projects
Source History
67 horario.setHora_inicio(
68     Time.valueOf("09:00:00")
69 );
70
71 assertEquals(
72     Time.valueOf("09:00:00"),
73     horario.getHora_inicio()
74 );
75 }
76
77 @Test
78 @DisplayName("setMax_citas() debe actualizar cantidad máxima")
79 void setMaxCitas_debeActualizarCantidad(){
80
81     Horarios horario = new Horarios();
82
83     horario.setMax_citas(15);
84
85     assertEquals(
86         15,
87         horario.getMax_citas()
88     );
89 }
90
91 @Test
92 @DisplayName("Hora fin debe ser posterior a hora inicio")
93 void horaFin_debeSerPosteriorAHoraInicio(){
94
95     Horarios horario = new Horarios();
96
97     horario.setHora_inicio(
98         Time.valueOf("08:00:00")
99 );
100
101
102
103
104
105
106
107
108
109
110
111
Output 1:1 INS
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 367.2/398.0MB
Projects
Source History
90
91 @Test
92 @DisplayName("Hora fin debe ser posterior a hora inicio")
93 void horaFin_debeSerPosteriorAHoraInicio(){
94
95     Horarios horario = new Horarios();
96
97     horario.setHora_inicio(
98         Time.valueOf("08:00:00")
99 );
100
101     horario.setHora_fin(
102         Time.valueOf("10:00:00")
103     );
104
105     assertTrue(
106         horario.getHora_fin().after(horario.getHora_inicio())
107     );
108 }
109
110
111
Output 1:1 INS
```

```
1 package co.sena.cimm.adso.saludboyaca.model;
2
3 import static org.junit.jupiter.api.Assertions.assertAll;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5 import static org.junit.jupiter.api.Assertions.assertNotNull;
6 import org.junit.jupiter.api.DisplayName;
7 import org.junit.jupiter.api.Test;
8
9 public class EspecialidadesTest {
10
11     @Test
12     @DisplayName("Constructor vacío debe crear objeto")
13     void constructorVacio_debeCrearObjeto() {
14
15         Especialidades especialidad = new Especialidades();
16
17         assertNotNull(especialidad);
18     }
19
20     @Test
21     @DisplayName("Constructor debe inicializar campos correctamente")
22     void constructor_debeInicializarCampos() {
23
24         Especialidades especialidad = new Especialidades(
25             1,
26             "Cardiología",
27             "Especialidad del corazón"
28         );
29
30         assertAll(
31             () -> assertEquals(1, especialidad.getId()),
32             () -> assertEquals("Cardiología", especialidad.getNombre()),
33             () -> assertEquals(
34                 "Especialidad del corazón",
35                 especialidad.getDescripcion()
36             )
37         );
38     }
39
40     @Test
41     @DisplayName("setNombre() debe actualizar nombre")
42     void setNombre_debeActualizarNombre() {
43
44         Especialidades especialidad = new Especialidades();
45
46         especialidad.setNombre("Pediatria");
47
48         assertEquals(
49             "Pediatria",
50             especialidad.getNombre()
51         );
52     }
53
54     @Test
55     @DisplayName("setDescripcion() debe actualizar descripcion")
56     void setDescripcion_debeActualizarDescripcion() {
57
58         Especialidades especialidad = new Especialidades();
59
60         especialidad.setDescripcion("Atención médica infantil");
61
62         assertEquals(
63             "Atención médica infantil",
64             especialidad.getDescripcion()
65         );
66     }
67 }
```



```
34         "Especialidad del corazón",
35         especialidad.getDescripcion()
36     );
37 }
38
39
40 @Test
41 @DisplayName("setNombre() debe actualizar nombre")
42 void setNombre_debeActualizarNombre(){
43
44     Especialidades especialidad = new Especialidades();
45
46     especialidad.setNombre("Pediatria");
47
48     assertEquals(
49         "Pediatria",
50         especialidad.getNombre()
51     );
52 }
53
54 @Test
55 @DisplayName("setDescripcion() debe actualizar descripcion")
56 void setDescripcion_debeActualizarDescripcion(){
57
58     Especialidades especialidad = new Especialidades();
59
60     especialidad.setDescripcion("Atención médica infantil");
61
62     assertEquals(
63         "Atención médica infantil",
64         especialidad.getDescripcion()
65     );
66 }
67 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 3177/398.0MB

CitasTest.java X
Source History
1 /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5 package co.sena.cimm.adso.saludboyaca.model;
6
7 import java.sql.Time;
8 import java.time.LocalDate;
9 import org.junit.jupiter.api.DisplayName;
10 import org.junit.jupiter.api.Test;
11 import static org.junit.jupiter.api.Assertions.*;
12
13 /**
14  *
15  * @author julia
16  */
17 public class CitasTest {
18
19     @Test
20     @DisplayName("Constructor vacío debe crear objeto")
21     void constructorVacío_debeCrearObjeto() {
22
23         Citas cita = new Citas();
24
25         assertNotNull(cita);
26     }
27
28     @Test
29     @DisplayName("Constructor debe inicializar todos los campos")
30     void constructor_debeInicializarCampos() {
31
32         LocalDate fechaCita
33             = LocalDate.of(2026, 5, 20, 10, 30);
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 2815/398.0MB

CitasTest.java X
Source History
34
35     LocalDateTime fechaRegistro
36         = LocalDateTime.of(2026, 5, 10, 8, 0);
37
38     Time hora
39         = Time.valueOf("10:30:00");
40
41     Citas cita = new Citas(
42         1,
43         100,
44         200,
45         3,
46         fechaCita,
47         hora,
48         "Control médico",
49         "PENDIENTE",
50         "Primera consulta",
51         fechaRegistro,
52         1
53     );
54
55     assertEquals(
56         () -> assertEquals(1, cita.getId()),
57         () -> assertEquals(100, cita.getId_paciente()),
58         () -> assertEquals(200, cita.getId_medico()),
59         () -> assertEquals(3, cita.getId_especialidad()),
60         () -> assertEquals(fechaCita, cita.getFecha_cita()),
61         () -> assertEquals(hora, cita.getHora_cita()),
62         () -> assertEquals(
63             "Control médico",
64             cita.getMotivo()
65         ),
66         () -> assertEquals(
67             "PENDIENTE"
```

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

CitasTest.java X

```
67     "PENDIENTE",
68     cita.getEstado()
69     ),
70     () -> assertEquals(
71         "Primera consulta",
72         cita.getObservaciones()
73     ),
74     () -> assertEquals(
75         fechaRegistro,
76         cita.getFecha_registro()
77     ),
78     () -> assertEquals(
79         1,
80         cita.getId_registro_por()
81     )
82     );
83 }
84
85 @Test
86 @DisplayName("setEstado() debe actualizar estado")
87 void setEstado_debeActualizarEstado(){
88
89     Citas cita = new Citas();
90
91     cita.setEstado("CONFIRMADA");
92
93     assertEquals(
94         "CONFIRMADA",
95         cita.getEstado()
96     );
97 }
98
99 @Test
```

Output 1:1 INS

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

CitasTest.java X

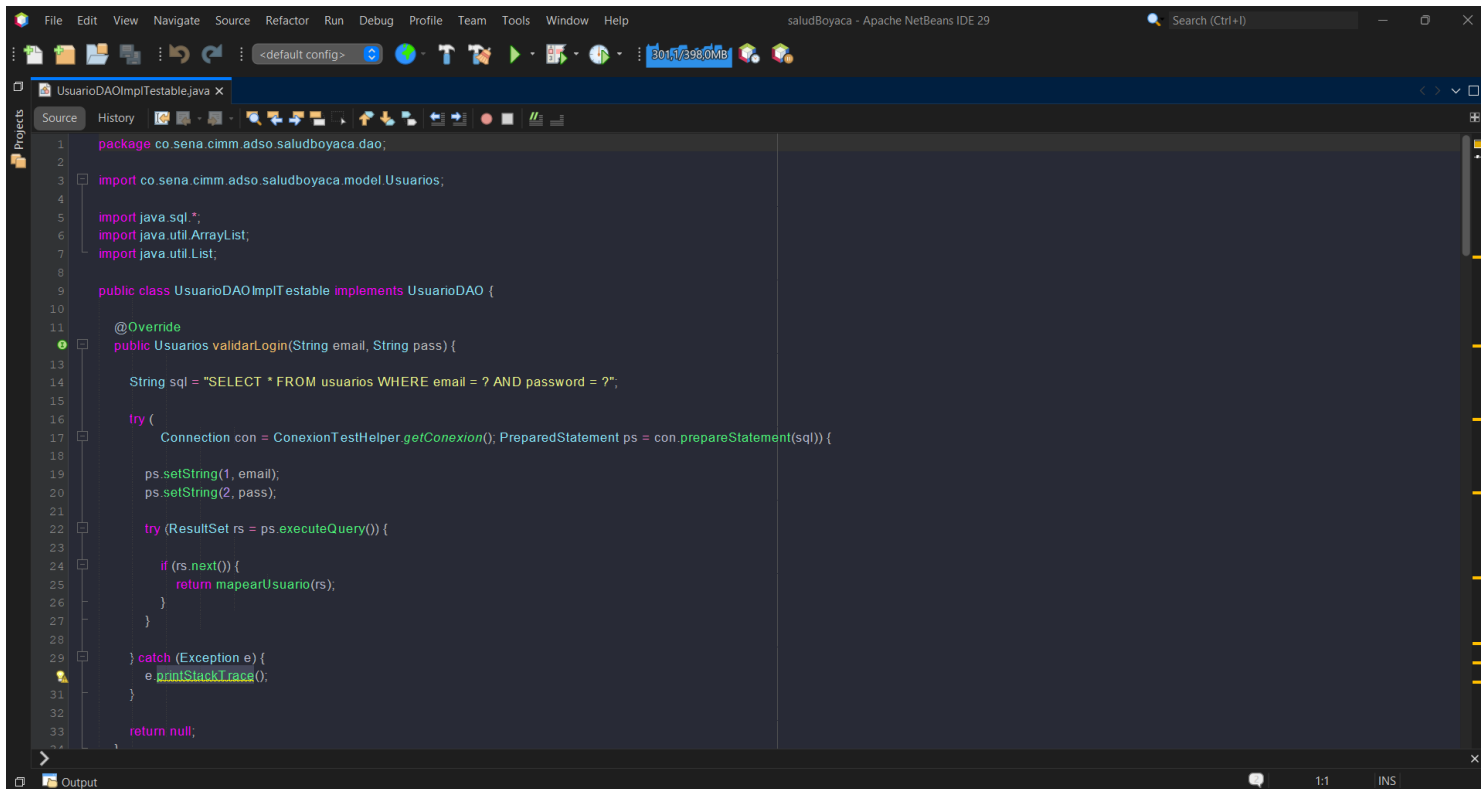
```
100 @Test
101 @DisplayName("setNombre_paciente() debe actualizar nombre")
102 void setNombre_paciente_debeActualizarNombre(){
103
104     Citas cita = new Citas();
105
106     cita.setNombre_paciente(
107         "Carlos Pérez"
108     );
109
110     assertEquals(
111         "Carlos Pérez",
112         cita.getNombre_paciente()
113     );
114 }
115
116 @Test
117 @DisplayName("Fecha de cita debe ser futura")
118 void fechaCita_debeSerPosteriorALAActual(){
119
120     Citas cita = new Citas();
121
122     cita.setFecha_cita(
123         LocalDateTime.now().plusDays(2)
124     );
125
126     assertTrue(
127         cita.getFecha_cita().isAfter(LocalDateTime.now())
128     );
129 }
130
131 @Test
132 @DisplayName("Hora de cita no debe ser null")
133 void horaCita_noDebeSerNull()
```

```
125
126     assertTrue(
127         cita.getFecha_cita().isAfter(LocalDate.now())
128     );
129 }
130
131 @Test
132 @DisplayName("Hora de cita no debe ser null")
133 void horaCita_noDebeSerNull(){
134
135     Citas cita = new Citas();
136
137     cita.setHora_cita(
138         Time.valueOf("09:00:00")
139     );
140
141     assertNotNull(
142         cita.getHora_cita()
143     );
144 }
145 }
146 }
```

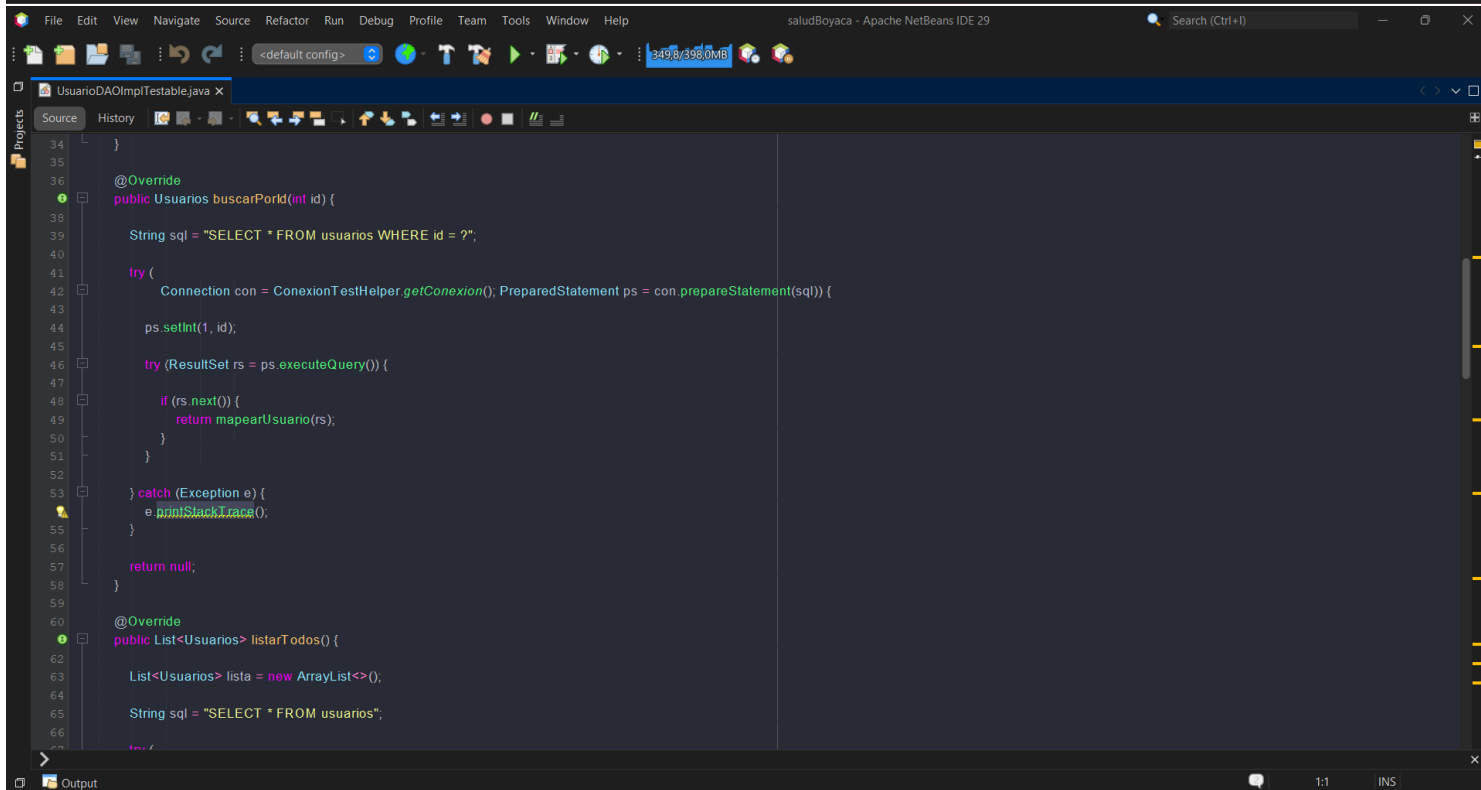
```
1 # src/test/resources/db-test.properties
2 # Configuración de H2 en memoria para pruebas de integración
3 db.url=jdbc:h2:mem:saludboyaca_test;DB_CLOSE_DELAY=-1;MODE=MySQL
4 db.driver=org.h2.Driver
5 db.user=sa
6 db.password=
7 # DB_CLOSE_DELAY=-1 → la BD permanece en memoria mientras la JVM corre
8 # MODE=MySQL → H2 acepta sintaxis MySQL (AUTO_INCREMENT, ENUM, etc.)
```

```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3
4 import java.io.InputStream;
5 import java.sql.Connection;
6 import java.sql.DriverManager;
7 import java.sql.Statement;
8 import java.util.Properties;
9
10 public class ConexionTestHelper {
11
12     private static Connection conexion;
13
14     public static Connection getConnection() throws Exception {
15
16         if (conexion == null || conexion.isClosed()) {
17
18             Properties props = new Properties();
19
20             try {
21                 InputStream in
22                     = ConexionTestHelper.class
23                     .getClassLoader()
24                     .getResourceAsStream("db-test.properties");
25                 props.load(in);
26             }
27
28             Class.forName("org.h2.Driver");
29
30             conexion = DriverManager.getConnection(
31                 props.getProperty("db.url"),
32                 props.getProperty("db.user"),
33                 props.getProperty("db.password")
34             );
35         }
36     }
37
38     return conexion;
39 }
40
41 public static void ejecutarSQL(String sql)
42     throws Exception {
43
44     try {
45         Statement stmt
46             = getConnection().createStatement();
47         stmt.execute(sql);
48     }
49 }
50
51 }
52
```

```
31     conexion = DriverManager.getConnection(
32         props.getProperty("db.url"),
33         props.getProperty("db.user"),
34         props.getProperty("db.password")
35     );
36 }
37
38 return conexion;
39 }
40
41 public static void ejecutarSQL(String sql)
42     throws Exception {
43
44     try {
45         Statement stmt
46             = getConnection().createStatement();
47         stmt.execute(sql);
48     }
49 }
50
51 }
52
```



```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Usuarios;
4
5 import java.sql.*;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class UsuarioDAOImplTestable implements UsuarioDAO {
10
11     @Override
12     public Usuarios validarLogin(String email, String pass) {
13
14         String sql = "SELECT * FROM usuarios WHERE email = ? AND password = ?";
15
16         try {
17             Connection con = ConexionTestHelper.getConexion();
18             PreparedStatement ps = con.prepareStatement(sql);
19
20             ps.setString(1, email);
21             ps.setString(2, pass);
22
23             try (ResultSet rs = ps.executeQuery()) {
24                 if (rs.next()) {
25                     return mapearUsuario(rs);
26                 }
27             }
28
29         } catch (Exception e) {
30             e.printStackTrace();
31         }
32
33         return null;
34     }
35 }
```



```
34 }
35
36 @Override
37 public Usuarios buscarPorId(int id) {
38
39     String sql = "SELECT * FROM usuarios WHERE id = ?";
40
41     try {
42         Connection con = ConexionTestHelper.getConexion();
43         PreparedStatement ps = con.prepareStatement(sql);
44
45         ps.setInt(1, id);
46
47         try (ResultSet rs = ps.executeQuery()) {
48             if (rs.next()) {
49                 return mapearUsuario(rs);
50             }
51         }
52
53     } catch (Exception e) {
54         e.printStackTrace();
55     }
56
57     return null;
58 }
59
60 @Override
61 public List<Usuarios> listarTodos() {
62
63     List<Usuarios> lista = new ArrayList<>();
64
65     String sql = "SELECT * FROM usuarios";
66
67     try {
68         Connection con = ConexionTestHelper.getConexion();
69         PreparedStatement ps = con.prepareStatement(sql);
70
71         try (ResultSet rs = ps.executeQuery()) {
72             while (rs.next()) {
73                 Usuarios usuario = mapearUsuario(rs);
74                 lista.add(usuario);
75             }
76         }
77
78     } catch (Exception e) {
79         e.printStackTrace();
80     }
81
82     return lista;
83 }
84 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
<default config> 262.7/398.0MB

UsuarioDAOImplTestable.java X
Source History
62 public List<Usuarios> listarTodos() {
63     List<Usuarios> lista = new ArrayList<>();
64
65     String sql = "SELECT * FROM usuarios";
66
67     try {
68         Connection con = ConexionTestHelper.getConexion(); Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql); {
69
70         while (rs.next()) {
71             lista.add(mapearUsuario(rs));
72         }
73     } catch (Exception e) {
74         e.printStackTrace();
75     }
76
77     return lista;
78 }
79
80
81 @Override
82 public boolean actualizarPreferenciaIdioma(int idUsuario, String lang) {
83
84     String sql = "UPDATE usuarios SET lang_preferido = ? WHERE id = ?";
85
86     try {
87         Connection con = ConexionTestHelper.getConexion(); PreparedStatement ps = con.prepareStatement(sql); {
88
89         ps.setString(1, lang);
90         ps.setInt(2, idUsuario);
91
92         return ps.executeUpdate() > 0;
93     } catch (Exception e) {
94         e.printStackTrace();
95     }
96 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
<default config> 310.9/398.0MB

UsuarioDAOImplTestable.java X
Source History
94 } catch (Exception e) {
95     e.printStackTrace();
96 }
97
98 return false;
99 }
100
101 @Override
102 public Usuarios buscarPorUsername(String username) {
103
104     String sql = "SELECT * FROM usuarios WHERE username = ?";
105
106     try {
107         Connection con = ConexionTestHelper.getConexion(); PreparedStatement ps = con.prepareStatement(sql); {
108
109         ps.setString(1, username);
110
111         try (ResultSet rs = ps.executeQuery()) {
112             if (rs.next()) {
113                 return mapearUsuario(rs);
114             }
115         }
116     } catch (Exception e) {
117         e.printStackTrace();
118     }
119
120     return null;
121 }
122
123
124 private Usuarios mapearUsuario(ResultSet rs) throws SQLException {
125     Usuarios u = new Usuarios();
126 }
```

The screenshot shows the NetBeans IDE with the file `UsuarioDAOImplTestable.java` open. The code is in the `Source` tab. The method `private Usuarios mapUsuario(ResultSet rs) throws SQLException` is being implemented. It creates a new `Usuarios` object and sets its attributes from the `ResultSet` using `rs.getString` and `rs.getInt`. There are three try-catch blocks for setting `username`, `password`, and `activo`, each catching `Exception`. The method returns the `Usuarios` object.

```
124
125 private Usuarios mapUsuario(ResultSet rs) throws SQLException {
126
127     Usuarios u = new Usuarios();
128
129     u.setId(rs.getInt("id"));
130     u.setNombres(rs.getString("nombres"));
131     u.setApellidos(rs.getString("apellidos"));
132     u.setEmail(rs.getString("email"));
133     u.setRol(rs.getString("rol"));
134     u.setLang_preferido(rs.getString("lang_preferido"));
135
136     try {
137         u.setUsername(rs.getString("username"));
138     } catch (Exception e) {
139     }
140
141     try {
142         u.setPassword(rs.getString("password"));
143     } catch (Exception e) {
144     }
145
146     try {
147         u.setActivo(rs.getInt("activo"));
148     } catch (Exception e) {
149     }
150
151     return u;
152 }
153
154
```

The screenshot shows the NetBeans IDE with the file `UsuarioDAOImplTest.java` open. The code is in the `Source` tab. It starts with a package declaration and imports for `co.sena.cimm.adso.saludboyaca.dao`, `co.sena.cimm.adso.saludboyaca.model.Usuarios`, and JUnit/JDBC classes. It then defines the `UsuarioDAOImplTest` class with a static `dao` field and a `@BeforeAll` `setUp()` method. The `setUp` method initializes the `dao` and executes a SQL script to create a table named `usuarios` with columns for `id`, `nombres`, `apellidos`, `username`, `email`, `password`, `rol`, `lang_preferido`, and `activo`.

```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Usuarios;
4
5 import org.junit.jupiter.api.BeforeAll;
6 import org.junit.jupiter.api.DisplayName;
7 import org.junit.jupiter.api.Test;
8
9 import java.sql.Connection;
10 import java.sql.PreparedStatement;
11 import java.util.List;
12
13 import static org.junit.jupiter.api.Assertions.*;
14
15 public class UsuarioDAOImplTest {
16
17     private static UsuarioDAOImplTestable dao;
18
19     @BeforeAll
20     static void setUp() throws Exception {
21         dao = new UsuarioDAOImplTestable();
22         ConexionTestHelper.ejecutarSQL("
23 CREATE TABLE IF NOT EXISTS usuarios (
24     id INT AUTO_INCREMENT PRIMARY KEY,
25     nombres VARCHAR(100),
26     apellidos VARCHAR(100),
27     username VARCHAR(100),
28     email VARCHAR(100),
29     password VARCHAR(100),
30     rol VARCHAR(50),
31     lang_preferido VARCHAR(10),
32     activo INT
33 );
34 ");
35     }
36 }
```

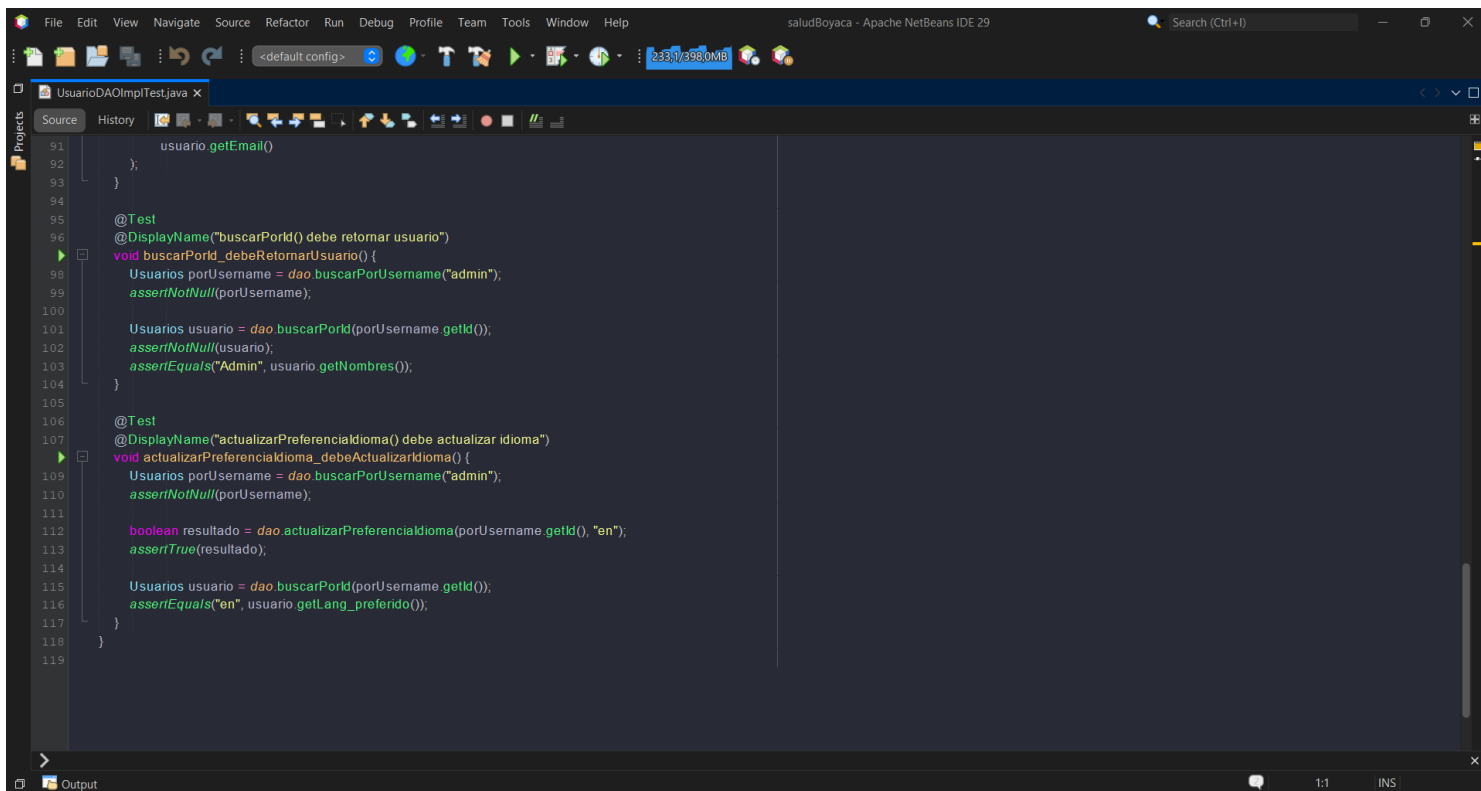


```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
<default config> 284.2/398.0MB

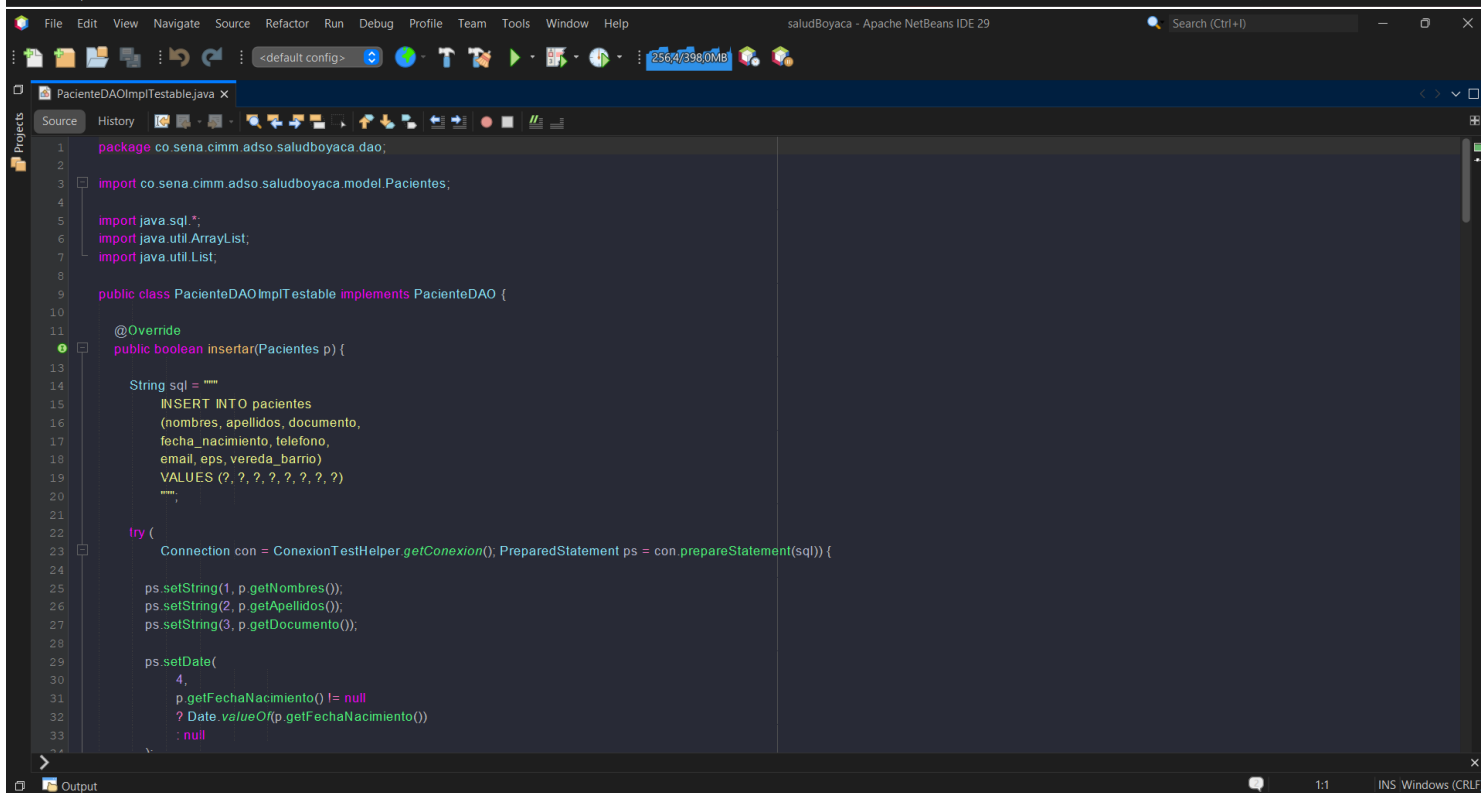
UsuarioDAOImplTest.java X
Source History
34
35 ConexionTestHelper.ejecutarSQL("DELETE FROM usuarios");
36 Connection con = ConexionTestHelper.getConnection();
37 PreparedStatement ps = con.prepareStatement("");
38 INSERT INTO usuarios
39 (nombres, apellidos, username, email, password, rol, lang_preferido, activo)
40 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
41
42 ps.setString(1, "Admin");
43 ps.setString(2, "Principal");
44 ps.setString(3, "admin");
45 ps.setString(4, "admin@gmail.com");
46 ps.setString(5, "123456");
47 ps.setString(6, "ADMINISTRADOR");
48 ps.setString(7, "es");
49 ps.setInt(8, 1);
50 ps.executeUpdate();
51
52
53 @Test
54 @DisplayName("listarTodos() debe retornar usuarios")
55 void listarTodos_debeRetornarUsuarios() {
56
57     List<Usuarios> lista = dao.listarTodos();
58
59     assertFalse(lista.isEmpty());
60 }
61
62 @Test
63 @DisplayName("validarLogin() debe encontrar usuario")
64 void validarLogin_debeEncontrarUsuario() {
65
66     Usuarios usuario
67     = dao.validarLogin(
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
<default config> 325.6/398.0MB

UsuarioDAOImplTest.java X
Source History
65 void validarLogin_debeEncontrarUsuario() {
66
67     Usuarios usuario
68     = dao.validarLogin(
69         "admin@gmail.com",
70         "123456"
71     );
72
73     assertNotNull(usuario);
74
75     assertEquals(
76         "Admin",
77         usuario.getNombres()
78     );
79 }
80
81 @Test
82 @DisplayName("buscarPorUsername() debe encontrar usuario")
83 void buscarPorUsername_debeEncontrarUsuario() {
84
85     Usuarios usuario
86     = dao.buscarPorUsername("admin");
87
88     assertNotNull(usuario);
89
90     assertEquals(
91         "admin@gmail.com",
92         usuario.getEmail()
93     );
94 }
95
96 @Test
97 @DisplayName("buscarPorId() debe retornar usuario")
98 void buscarPorId_debeRetornarUsuario() {
```



```
91     usuario.getEmail()
92   };
93 }
94
95 @Test
96 @DisplayName("buscarPorId() debe retomar usuario")
97 void buscarPorId_debeRetomarUsuario() {
98     Usuarios porUsername = dao.buscarPorUsername("admin");
99     assertNotNull(porUsername);
100
101     Usuarios usuario = dao.buscarPorId(porUsername.getId());
102     assertNotNull(usuario);
103     assertEquals("Admin", usuario.getNombres());
104 }
105
106 @Test
107 @DisplayName("actualizarPreferenciadioma() debe actualizar idioma")
108 void actualizarPreferenciadioma_debeActualizarIdioma() {
109     Usuarios porUsername = dao.buscarPorUsername("admin");
110     assertNotNull(porUsername);
111
112     boolean resultado = dao.actualizarPreferenciadioma(porUsername.getId(), "en");
113     assertTrue(resultado);
114
115     Usuarios usuario = dao.buscarPorId(porUsername.getId());
116     assertEquals("en", usuario.getLang_preferido());
117 }
118 }
119 }
```



```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Pacientes;
4
5 import java.sql.*;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class PacienteDAOImplTestable implements PacienteDAO {
10
11     @Override
12     public boolean insertar(Pacientes p) {
13
14         String sql = ""
15             INSERT INTO pacientes
16             (nombres, apellidos, documento,
17             fecha_nacimiento, telefono,
18             email, eps, vereda_barrio)
19             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
20             ""
21
22         try {
23             Connection con = ConexionTestHelper.getConnection();
24             PreparedStatement ps = con.prepareStatement(sql);
25
26             ps.setString(1, p.getNombres());
27             ps.setString(2, p.getApellidos());
28             ps.setString(3, p.getDocumento());
29
30             ps.setDate(
31                 4,
32                 p.getFechaNacimiento() != null
33                     ? Date.valueOf(p.getFechaNacimiento())
34                     : null
35             );
36
37             ps.executeUpdate();
38         } catch (SQLException e) {
39             e.printStackTrace();
40         }
41
42         return true;
43     }
44 }
```

This screenshot shows the Apache NetBeans IDE with the file `PacienteDAOImplTestable.java` open. The editor displays the `actualizar` method, which is annotated with `@Override`. The method's logic involves constructing an SQL `UPDATE` statement for the `pacientes` table, setting various fields like `nombres`, `apellidos`, `documento`, `fecha_nacimiento`, `telefono`, `email`, `eps`, and `vereda_barrio` based on the parameters of the `Pacientes` object. It then attempts to execute the update using `ps.executeUpdate()` and returns `true` if successful, or `false` if an exception occurs. The IDE interface includes a menu bar, a toolbar, and a status bar at the bottom showing the output window.

```
34  
35  
36     ps.setString(5, p.getTelefono());  
37     ps.setString(6, p.getEmail());  
38     ps.setString(7, p.getEps());  
39     ps.setString(8, p.getVeredaBarrio());  
40  
41     return ps.executeUpdate() > 0;  
42  
43 } catch (Exception e) {  
44  
45     return false;  
46 }  
47  
48  
49 @Override  
50 public boolean actualizar(Pacientes p) {  
51  
52     String sql = ""  
53         UPDATE pacientes  
54         SET nombres=?,  
55         apellidos=?,  
56         documento=?,  
57         fecha_nacimiento=?,  
58         telefono=?,  
59         email=?,  
60         eps=?,  
61         vereda_barrio=?  
62         WHERE id=?  
63     "";  
64  
65     try {  
66         Connection con = ConexionTestHelper.getConexion(); PreparedStatement ps = con.prepareStatement(sql);
```

This screenshot shows the same Apache NetBeans IDE environment, but now displaying the `eliminar` method in `PacienteDAOImplTestable.java`. This method is also annotated with `@Override` and takes an `id` parameter. It constructs an SQL `DELETE` statement: `"DELETE FROM pacientes WHERE id = ?"`. The method then attempts to execute the delete operation using `ps.executeUpdate()` and returns `true` if successful, or `false` if an exception occurs. The IDE interface remains consistent with the previous screenshot, showing the menu bar, toolbar, and status bar.

```
67  
68     ps.setString(1, p.getNombres());  
69     ps.setString(2, p.getApellidos());  
70     ps.setString(3, p.getDocumento());  
71  
72     ps.setDate(  
73         4,  
74         p.getFechaNacimiento() != null  
75         ? Date.valueOf(p.getFechaNacimiento())  
76         : null  
77     );  
78  
79     ps.setString(5, p.getTelefono());  
80     ps.setString(6, p.getEmail());  
81     ps.setString(7, p.getEps());  
82     ps.setString(8, p.getVeredaBarrio());  
83  
84     ps.setInt(9, p.getId());  
85  
86     return ps.executeUpdate() > 0;  
87  
88 } catch (Exception e) {  
89  
90     return false;  
91 }  
92  
93  
94 @Override  
95 public boolean eliminar(int id) {  
96  
97     String sql = "DELETE FROM pacientes WHERE id = ?";  
98  
99     try {  
100         Connection con = ConexionTestHelper.getConexion(); PreparedStatement ps = con.prepareStatement(sql);
```

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

PacienteDAOImplTestable.java X

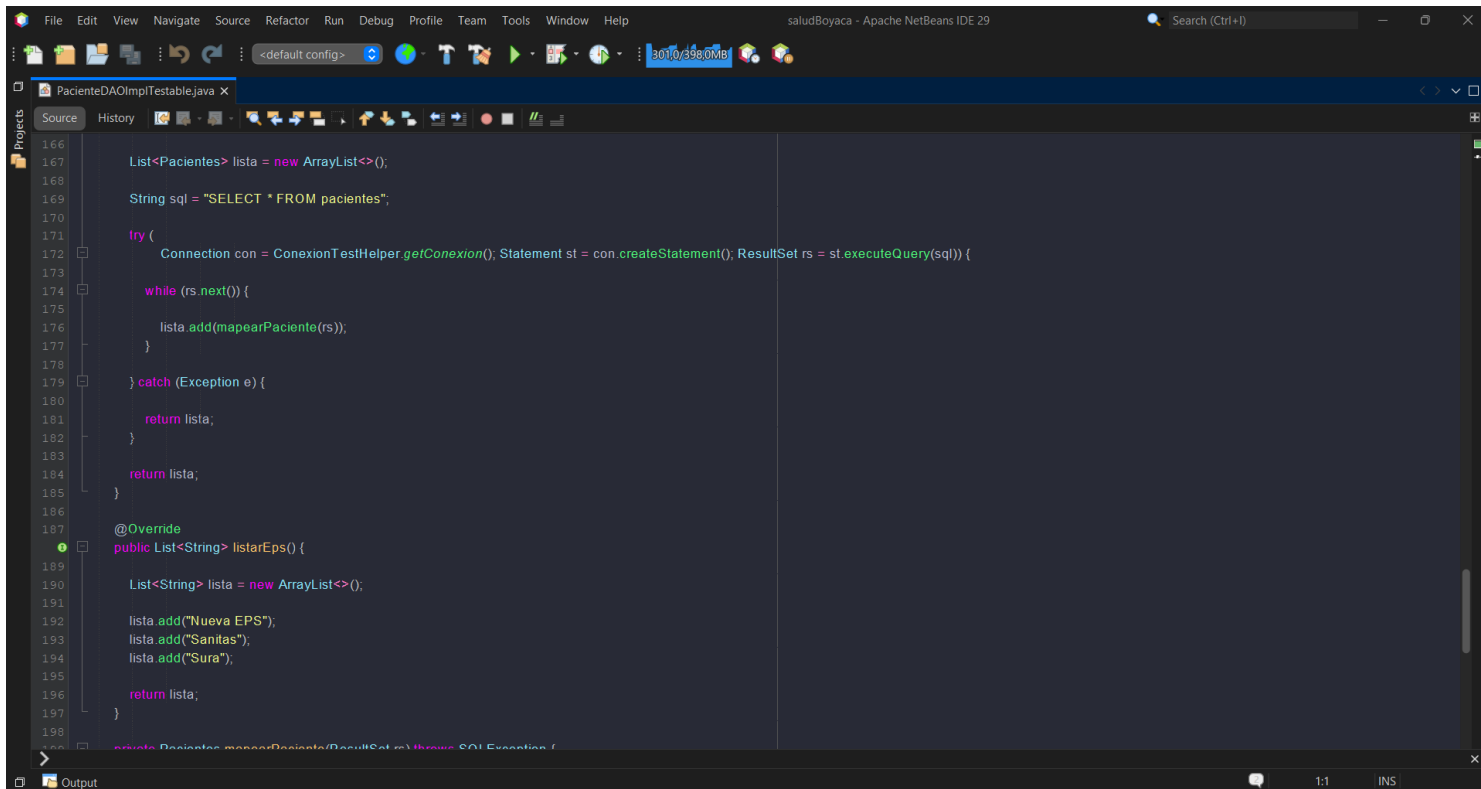
```
100 Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql) {
101
102     ps.setInt(1, id);
103
104     return ps.executeUpdate() > 0;
105
106 } catch (Exception e) {
107
108     return false;
109 }
110 }
111
112 @Override
113 public Pacientes buscarPorDocumento(String documento) {
114
115     String sql = "SELECT * FROM pacientes WHERE documento = ?";
116
117     try {
118         Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql) {
119
120             ps.setString(1, documento);
121
122             try (ResultSet rs = ps.executeQuery()) {
123
124                 if (rs.next()) {
125
126                     return mapearPaciente(rs);
127                 }
128             }
129
130         } catch (Exception e) {
131
132             return null;
133         }
134     }
```

Output 1:1 INS

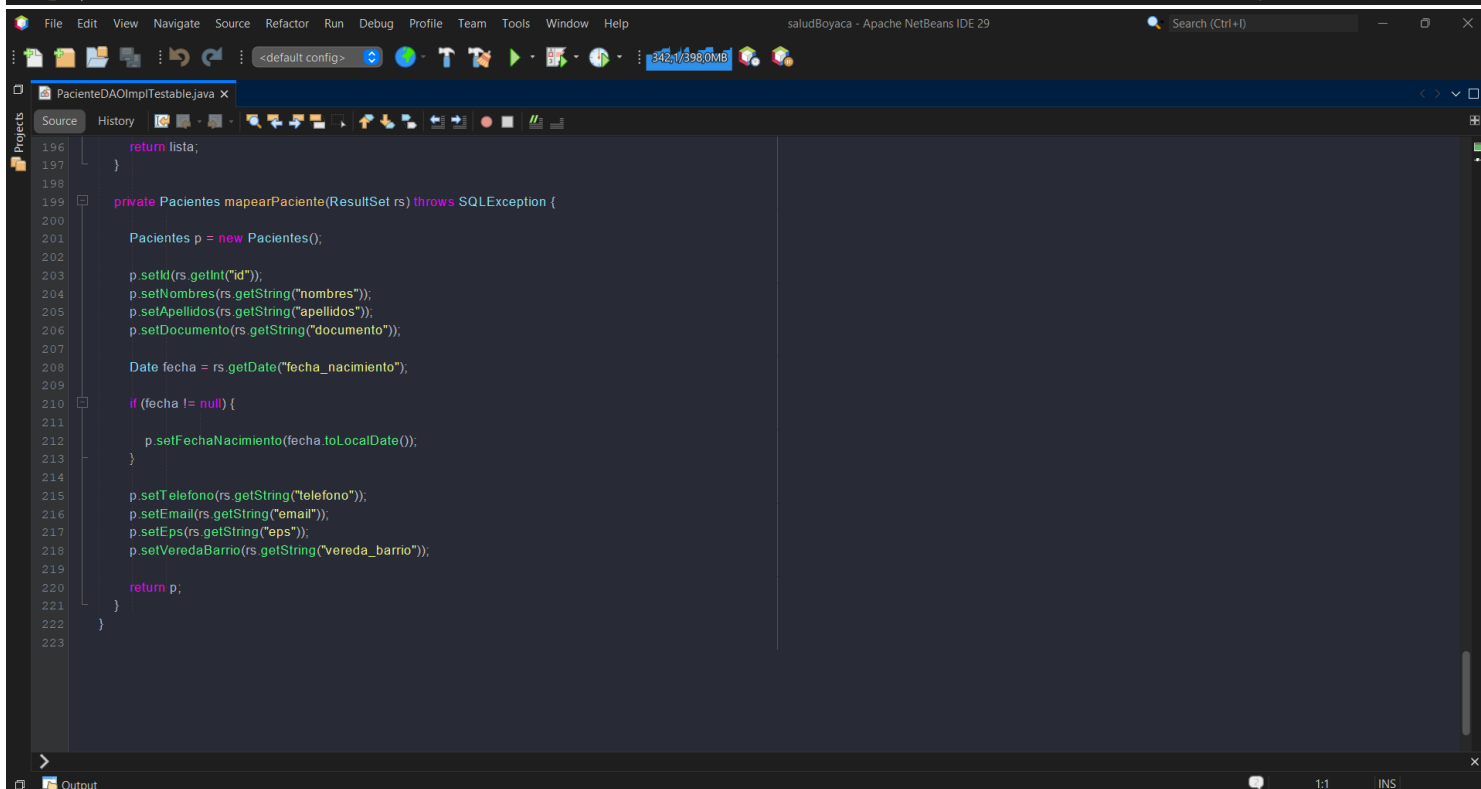
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

PacienteDAOImplTestable.java X

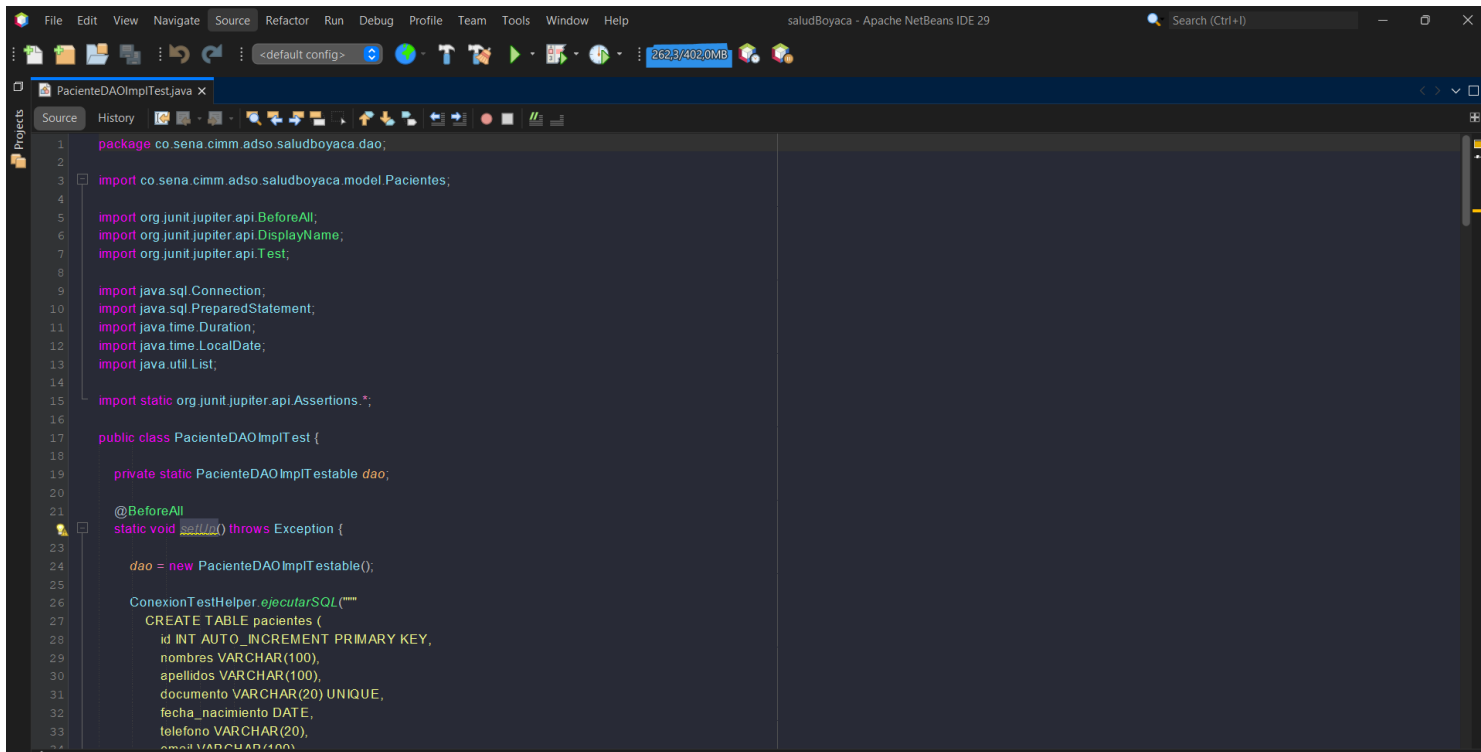
```
133 }
134
135 return null;
136 }
137
138 @Override
139 public Pacientes buscarPorId(int id) {
140
141     String sql = "SELECT * FROM pacientes WHERE id = ?";
142
143     try {
144         Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql) {
145
146             ps.setInt(1, id);
147
148             try (ResultSet rs = ps.executeQuery()) {
149
150                 if (rs.next()) {
151
152                     return mapearPaciente(rs);
153                 }
154             }
155
156         } catch (Exception e) {
157
158             return null;
159         }
160
161         return null;
162     }
163
164 @Override
165 public List<Pacientes> listarTodos() {
```



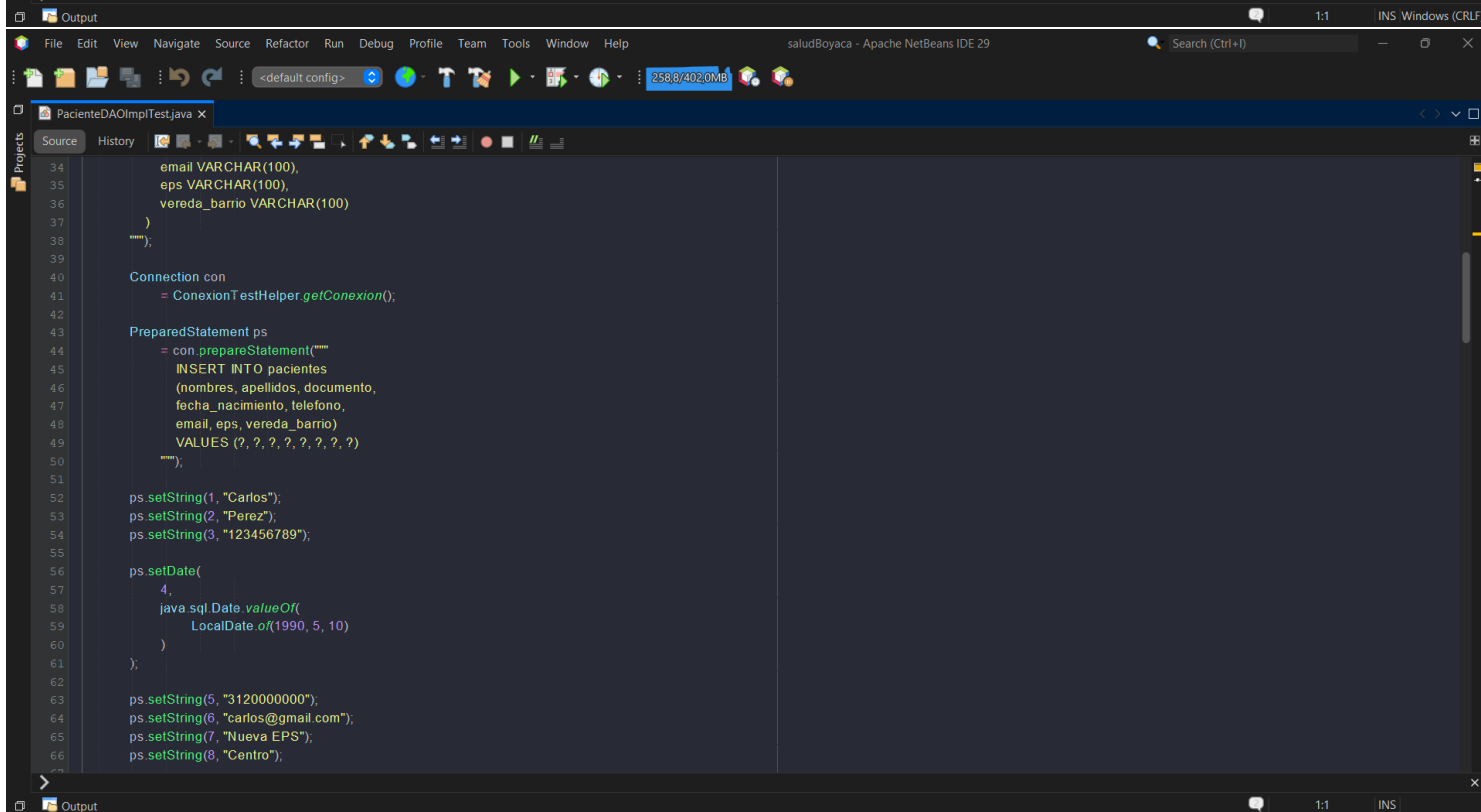
```
166
167
168     List<Pacientes> lista = new ArrayList<>();
169
170     String sql = "SELECT * FROM pacientes";
171
172     try {
173         Connection con = ConexionTestHelper.getConexion(); Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql); {
174
175         while (rs.next()) {
176             lista.add(mapearPaciente(rs));
177         }
178     } catch (Exception e) {
179
180         return lista;
181     }
182
183     return lista;
184 }
185
186
187 @Override
188 public List<String> listarEps() {
189
190     List<String> lista = new ArrayList<>();
191
192     lista.add("Nueva EPS");
193     lista.add("Sanitas");
194     lista.add("Sura");
195
196     return lista;
197 }
198
199 private Pacientes mapearPaciente(ResultSet rs) throws SQLException {
```



```
196
197
198     return lista;
199 }
200
201 private Pacientes mapearPaciente(ResultSet rs) throws SQLException {
202
203     Pacientes p = new Pacientes();
204
205     p.setId(rs.getInt("id"));
206     p.setNombres(rs.getString("nombres"));
207     p.setApellidos(rs.getString("apellidos"));
208     p.setDocumento(rs.getString("documento"));
209
210     Date fecha = rs.getDate("fecha_nacimiento");
211
212     if (fecha != null) {
213         p.setFechaNacimiento(fecha.toLocalDate());
214     }
215
216     p.setTelefono(rs.getString("telefono"));
217     p.setEmail(rs.getString("email"));
218     p.setEps(rs.getString("eps"));
219     p.setVeredaBarrio(rs.getString("vereda_barrio"));
220
221     return p;
222 }
223
```



```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Pacientes;
4
5 import org.junit.jupiter.api.BeforeAll;
6 import org.junit.jupiter.api.DisplayName;
7 import org.junit.jupiter.api.Test;
8
9 import java.sql.Connection;
10 import java.sql.PreparedStatement;
11 import java.time.Duration;
12 import java.time.LocalDate;
13 import java.util.List;
14
15 import static org.junit.jupiter.api.Assertions.*;
16
17 public class PacienteDAOImplTest {
18
19     private static PacienteDAOImplTestable dao;
20
21     @BeforeAll
22     static void setUp() throws Exception {
23
24         dao = new PacienteDAOImplTestable();
25
26         ConexionTestHelper.ejecutarSQL("""
27         CREATE TABLE pacientes (
28             id INT AUTO_INCREMENT PRIMARY KEY,
29             nombres VARCHAR(100),
30             apellidos VARCHAR(100),
31             documento VARCHAR(20) UNIQUE,
32             fecha_nacimiento DATE,
33             telefono VARCHAR(20),
34             email VARCHAR(100),
35             eps VARCHAR(100),
36             vereda_barrio VARCHAR(100)
37         );
38
39         Connection con
40             = ConexionTestHelper.getConexion();
41
42         PreparedStatement ps
43             = con.prepareStatement("""
44             INSERT INTO pacientes
45             (nombres, apellidos, documento,
46             fecha_nacimiento, telefono,
47             email, eps, vereda_barrio)
48             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
49             """);
50
51         ps.setString(1, "Carlos");
52         ps.setString(2, "Perez");
53         ps.setString(3, "123456789");
54
55         ps.setDate(
56             4,
57             java.sql.Date.valueOf(
58                 LocalDate.of(1990, 5, 10)
59             )
60         );
61
62         ps.setString(5, "3120000000");
63         ps.setString(6, "carlos@gmail.com");
64         ps.setString(7, "Nueva EPS");
65         ps.setString(8, "Centro");
```



```
34         email VARCHAR(100),
35         eps VARCHAR(100),
36         vereda_barrio VARCHAR(100)
37     );
38
39     Connection con
40         = ConexionTestHelper.getConexion();
41
42     PreparedStatement ps
43         = con.prepareStatement("""
44         INSERT INTO pacientes
45         (nombres, apellidos, documento,
46         fecha_nacimiento, telefono,
47         email, eps, vereda_barrio)
48         VALUES (?, ?, ?, ?, ?, ?, ?, ?)
49         """);
50
51     ps.setString(1, "Carlos");
52     ps.setString(2, "Perez");
53     ps.setString(3, "123456789");
54
55     ps.setDate(
56         4,
57         java.sql.Date.valueOf(
58             LocalDate.of(1990, 5, 10)
59         )
60     );
61
62     ps.setString(5, "3120000000");
63     ps.setString(6, "carlos@gmail.com");
64     ps.setString(7, "Nueva EPS");
65     ps.setString(8, "Centro");
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
307.9/402.0MB

PacienteDAOImplTest.java x
Source History
68 ps.executeUpdate();
69 }
70
71 @Test
72 @DisplayName("listarTodos() debe retornar pacientes")
73 void listarTodos_debeRetornarPacientes() {
74
75     List<Pacientes> lista
76         = dao.listarTodos();
77
78     assertFalse(lista.isEmpty());
79 }
80
81 @Test
82 @DisplayName("buscarPorDocumento() debe encontrar paciente")
83 void buscarPorDocumento_debeEncontrarPaciente() {
84
85     Pacientes paciente
86         = dao.buscarPorDocumento(
87         "123456789"
88     );
89
90     assertNotNull(paciente);
91
92     assertEquals(
93         "Carlos",
94         paciente.getNombres()
95     );
96 }
97
98 @Test
99 @DisplayName("buscarPorId() debe retornar paciente")
100 void buscarPorId_debeRetornarPaciente() {
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
351.0/402.0MB

PacienteDAOImplTest.java x
Source History
101
102     Pacientes nuevo = new Pacientes(
103         0,
104         "Pedro",
105         "Gomez",
106         "555555",
107         LocalDate.of(2000, 1, 1),
108         "3000000000",
109         "pedro@gmail.com",
110         "Sura",
111         "Centro"
112     );
113
114     dao.insertar(nuevo);
115
116     Pacientes paciente
117         = dao.buscarPorDocumento("555555");
118
119     assertNotNull(paciente);
120
121     Pacientes encontrado
122         = dao.buscarPorId(paciente.getId());
123
124     assertNotNull(encontrado);
125 }
126
127 @Test
128 @DisplayName("insertar() debe guardar paciente")
129 void insertar_debeGuardarPaciente() {
130
131     Pacientes nuevo
132         = new Pacientes(
133         0,
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 255.4/402.0MB
PacienteDAOImplTest.java x
Source History
134         "Maria",
135         "Lopez",
136         "999999",
137         LocalDate.of(1995, 8, 15),
138         "3001234567",
139         "maria@gmail.com",
140         "Sanitas",
141         "Nobsa"
142     );
143
144     boolean resultado
145         = dao.insertar(nuevo);
146
147     assertTrue(resultado);
148 }
149
150 @Test
151 @DisplayName("No debe permitir documento duplicado")
152 void insertarDocumentoDuplicado_debeFallar() {
153
154     Pacientes duplicado
155         = new Pacientes(
156         0,
157         "Juan",
158         "Ramirez",
159         "123456789",
160         LocalDate.of(1998, 2, 20),
161         "3111111111",
162         "juan@gmail.com",
163         "Compensar",
164         "Duitama"
165     );
166 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 301.4/402.0MB
PacienteDAOImplTest.java x
Source History
167     boolean resultado
168         = dao.insertar(duplicado);
169
170     assertFalse(resultado);
171 }
172
173 @Test
174 @DisplayName("listarTodos() debe ejecutarse rápidamente con 50 registros")
175 void listarTodos_debeSerRapido() {
176
177     for (int i = 1; i <= 50; i++) {
178
179         Pacientes paciente
180             = new Pacientes(
181             0,
182             "Paciente" + i,
183             "Test",
184             "DOC" + i,
185             LocalDate.of(2000, 1, 1),
186             "30000000000",
187             "test" + i + "@gmail.com",
188             "EPS Test",
189             "Sogamoso"
190         );
191
192         dao.insertar(paciente);
193     }
194
195     assertTimeout(
196         Duration.ofSeconds(2),
197         () -> {
198             List<Pacientes> lista
199                 = dao.listarTodos();
200         }
201     );
202 }
```


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

PacienteDAOImplTest.java x

```
185         LocalDate.of(2000, 1, 1),
186         "30000000000",
187         "test" + i + "@gmail.com",
188         "EPS Test",
189         "Sogamoso"
190     );
191
192     dao.insertar(paciente);
193 }
194
195 @Override
196 public void Duration.ofSeconds(2),
197 () -> {
198     List<Pacientes> lista
199     = dao.listarTodos();
200
201     assertTrue(lista.size() >= 50);
202 }
203 }
204 }
205 }
206 }
```

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

HorarioDAOImplTestable.java x

```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Horarios;
4
5 import java.sql.*;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class HorarioDAOImplTestable implements HorarioDAO {
10
11     @Override
12     public List<Horarios> listarPorMedico(int idMedico) {
13
14         List<Horarios> lista = new ArrayList<>();
15
16         String sql = "SELECT * FROM horarios WHERE id_medico = ?";
17
18         try (Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
19
20             ps.setInt(1, idMedico);
21
22             try (ResultSet rs = ps.executeQuery()) {
23                 while (rs.next()) {
24                     lista.add(mapear(rs));
25                 }
26             }
27
28         } catch (Exception e) {
29             e.printStackTrace();
30         }
31
32         return lista;
33     }
34 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
<default config> 34217/4040MB

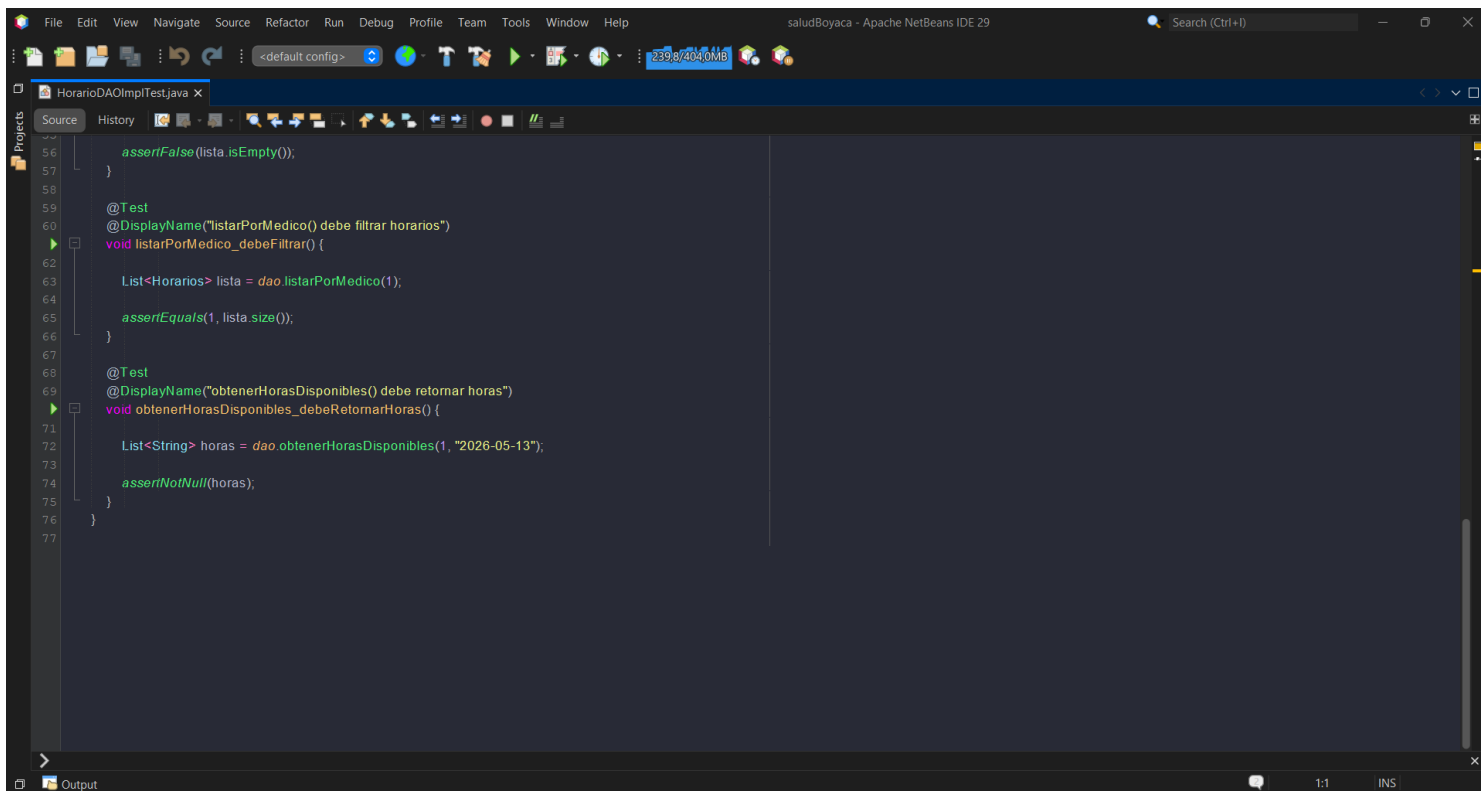
HorarioDAOImpleTestable.java X
Source History
34
35 @Override
36 public List<Horarios> listarTodos() {
37
38     List<Horarios> lista = new ArrayList<>();
39
40     String sql = "SELECT * FROM horarios";
41
42     try (Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql); ResultSet rs = ps.executeQuery()) {
43
44         while (rs.next()) {
45             lista.add(mapear(rs));
46         }
47     } catch (Exception e) {
48         e.printStackTrace();
49     }
50
51     return lista;
52 }
53
54
55 @Override
56 public List<String> obtenerHorasDisponibles(int idMedico, String fecha) {
57     List<String> horas = new ArrayList<>();
58     String sql = "SELECT hora_inicio, hora_fin FROM horarios WHERE id_medico = ? AND dia_semana = ?";
59     try (Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql)) {
60         ps.setInt(1, idMedico);
61         ps.setInt(2, obtenerDiaSemana(fecha));
62         try (ResultSet rs = ps.executeQuery()) {
63             while (rs.next()) {
64                 horas.add(rs.getString("hora_inicio") + " - " + rs.getString("hora_fin"));
65             }
66         }
67     } catch (Exception e) {
68         e.printStackTrace();
69     }
70     return horas;
71 }
72
73 private int obtenerDiaSemana(String fecha) {
74     return java.time.LocalDate.parse(fecha).getDayOfWeek().getValue();
75 }
76
77 private Horarios mapear(ResultSet rs) throws SQLException {
78     Horarios h = new Horarios();
79
80     h.setId(rs.getInt("id"));
81     h.setId_medico(rs.getInt("id_medico"));
82     h.setDia_semana(rs.getInt("dia_semana"));
83     h.setHora_inicio(rs.getTime("hora_inicio"));
84     h.setHora_fin(rs.getTime("hora_fin"));
85     h.setMax_citas(rs.getInt("max_citas"));
86
87     return h;
88 }
89
90
91
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
<default config> 26117/4040MB

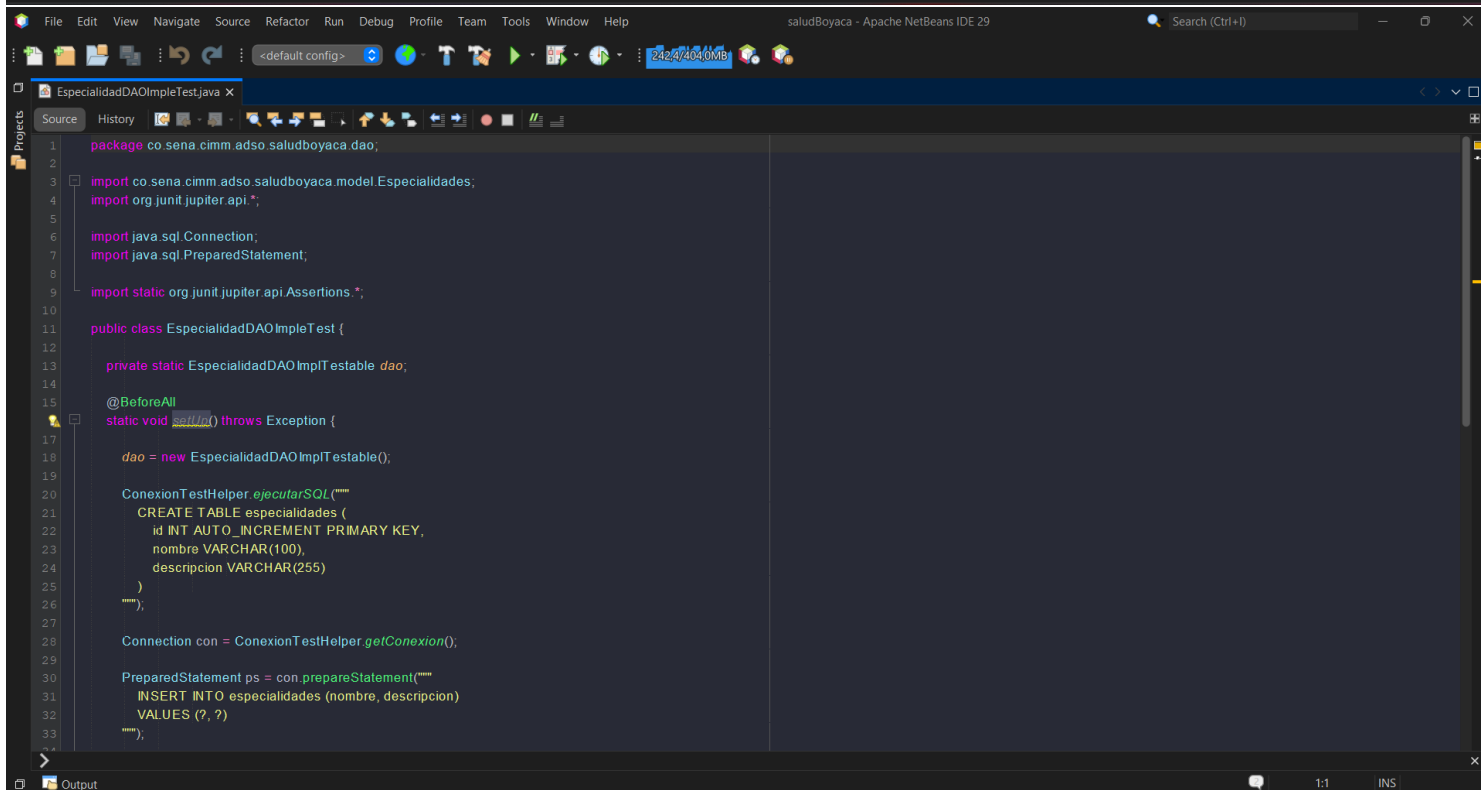
HorarioDAOImpleTestable.java X
Source History
67
68     } catch (Exception e) {
69         e.printStackTrace();
70     }
71     return horas;
72 }
73
74 private int obtenerDiaSemana(String fecha) {
75     return java.time.LocalDate.parse(fecha).getDayOfWeek().getValue();
76 }
77
78 private Horarios mapear(ResultSet rs) throws SQLException {
79     Horarios h = new Horarios();
80
81     h.setId(rs.getInt("id"));
82     h.setId_medico(rs.getInt("id_medico"));
83     h.setDia_semana(rs.getInt("dia_semana"));
84     h.setHora_inicio(rs.getTime("hora_inicio"));
85     h.setHora_fin(rs.getTime("hora_fin"));
86     h.setMax_citas(rs.getInt("max_citas"));
87
88     return h;
89 }
90
91
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 343.6/404.0MB
HorarioDAOImpTest.java X
Source History
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Horarios;
4 import org.junit.jupiter.api.*;
5
6 import java.sql.Connection;
7 import java.sql.PreparedStatement;
8 import java.sql.Time;
9 import java.util.List;
10
11 import static org.junit.jupiter.api.Assertions.*;
12
13 public class HorarioDAOImpTest {
14
15     private static HorarioDAOImpleTestable dao;
16
17     @BeforeAll
18     static void setUp() throws Exception {
19
20         dao = new HorarioDAOImpleTestable();
21
22         ConexionTestHelper.ejecutarSQL("""
23         CREATE TABLE horarios (
24             id INT AUTO_INCREMENT PRIMARY KEY,
25             id_medico INT,
26             dia_semana INT,
27             hora_inicio TIME,
28             hora_fin TIME,
29             max_citas INT
30         )
31         """);
32
33         Connection con = ConexionTestHelper.getConnection();
34     }
35 }
```

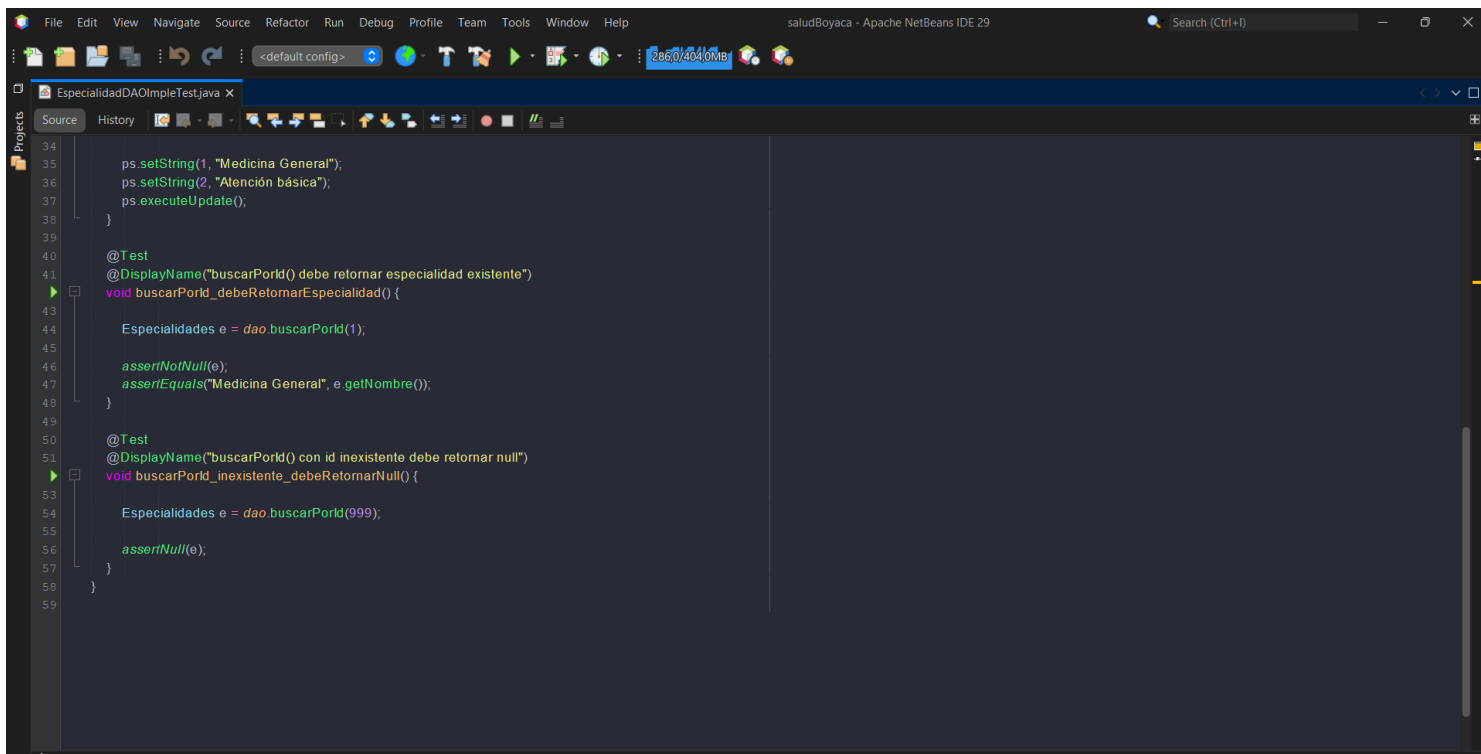
```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
... <default config> 343.6/404.0MB
HorarioDAOImpTest.java X
Source History
34
35 PreparedStatement ps = con.prepareStatement("""
36     INSERT INTO horarios
37     (id_medico, dia_semana, hora_inicio, hora_fin, max_citas)
38     VALUES (?, ?, ?, ?, ?)
39     """);
40
41 ps.setInt(1, 1);
42 ps.setInt(2, 1);
43 ps.setTime(3, Time.valueOf("08:00:00"));
44 ps.setTime(4, Time.valueOf("12:00:00"));
45 ps.setInt(5, 10);
46
47 ps.executeUpdate();
48
49
50 @Test
51 @DisplayName("listarTodos() debe retornar horarios")
52 void listarTodos_debeRetornarLista() {
53
54     List<Horarios> lista = dao.listarTodos();
55
56     assertFalse(lista.isEmpty());
57 }
58
59 @Test
60 @DisplayName("listarPorMedico() debe filtrar horarios")
61 void listarPorMedico_debeFiltrar() {
62
63     List<Horarios> lista = dao.listarPorMedico(1);
64
65     assertEquals(1, lista.size());
66 }
67 }
```



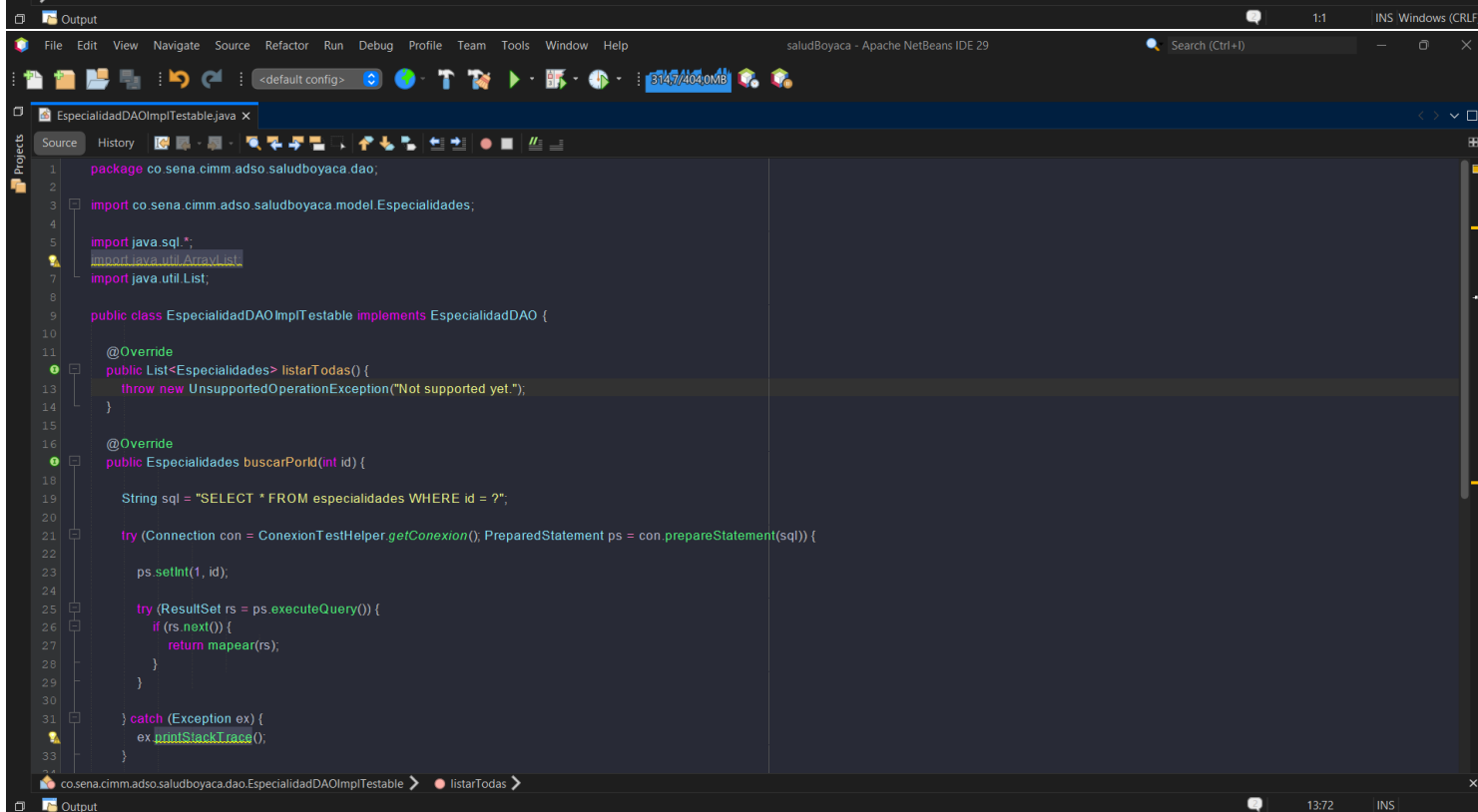
```
56     assertFalse(lista.isEmpty());
57 }
58
59 @Test
60 @DisplayName("listarPorMedico() debe filtrar horarios")
61 void listarPorMedico_debeFiltrar() {
62
63     List<Horarios> lista = dao.listarPorMedico(1);
64
65     assertEquals(1, lista.size());
66 }
67
68 @Test
69 @DisplayName("obtenerHorasDisponibles() debe retornar horas")
70 void obtenerHorasDisponibles_debeRetornarHoras() {
71
72     List<String> horas = dao.obtenerHorasDisponibles(1, "2026-05-13");
73
74     assertNotNull(horas);
75 }
76 }
77 }
```



```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Especialidades;
4 import org.junit.jupiter.api.*;
5
6 import java.sql.Connection;
7 import java.sql.PreparedStatement;
8
9 import static org.junit.jupiter.api.Assertions.*;
10
11 public class EspecialidadDAOImplTest {
12
13     private static EspecialidadDAOImplTestable dao;
14
15     @BeforeAll
16     static void setUp() throws Exception {
17
18         dao = new EspecialidadDAOImplTestable();
19
20         ConexionTestHelper.ejecutarSQL("""
21         CREATE TABLE especialidades (
22             id INT AUTO_INCREMENT PRIMARY KEY,
23             nombre VARCHAR(100),
24             descripcion VARCHAR(255)
25         )
26         """);
27
28         Connection con = ConexionTestHelper.getConnection();
29
30         PreparedStatement ps = con.prepareStatement("""
31         INSERT INTO especialidades (nombre, descripcion)
32         VALUES (?, ?)
33         """);
34     }
```



```
34     ps.setString(1, "Medicina General");
35     ps.setString(2, "Atención básica");
36     ps.executeUpdate();
37 }
38
39
40 @Test
41 @DisplayName("buscarPorId() debe retornar especialidad existente")
42 void buscarPorId_debeRetornarEspecialidad() {
43
44     Especialidades e = dao.buscarPorId(1);
45
46     assertNotNull(e);
47     assertEquals("Medicina General", e.getNombre());
48 }
49
50 @Test
51 @DisplayName("buscarPorId() con id inexistente debe retornar null")
52 void buscarPorId_inexistente_debeRetornarNull() {
53
54     Especialidades e = dao.buscarPorId(999);
55
56     assertNull(e);
57 }
58
59 }
```



```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Especialidades;
4
5 import java.sql.*;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class EspecialidadDAOImplTestable implements EspecialidadDAO {
10
11     @Override
12     public List<Especialidades> listarTodas() {
13         throw new UnsupportedOperationException("Not supported yet.");
14     }
15
16     @Override
17     public Especialidades buscarPorId(int id) {
18
19         String sql = "SELECT * FROM especialidades WHERE id = ?";
20
21         try (Connection con = ConexionTestHelper.getConexion(); PreparedStatement ps = con.prepareStatement(sql)) {
22             ps.setInt(1, id);
23
24             try (ResultSet rs = ps.executeQuery()) {
25                 if (rs.next()) {
26                     return mapear(rs);
27                 }
28             }
29         } catch (Exception ex) {
30             ex.printStackTrace();
31         }
32     }
33 }
```

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

SpecialidadDAOImplTestable.java X

Source History

```
28 }
29 }
30
31 } catch (Exception ex) {
32     ex.printStackTrace();
33 }
34
35 return null;
36 }
37
38 private Especialidades mapear(ResultSet rs) throws SQLException {
39
40     Especialidades e = new Especialidades();
41
42     e.setId(rs.getInt("id"));
43     e.setNombre(rs.getString("nombre"));
44     e.setDescripcion(rs.getString("descripcion"));
45
46     return e;
47 }
48
49 }
```

co.sena.cimm.adso.saludboyaca.dao.EspecialidadDAOImplTestable > listarTodas >

Output 13:72 INS

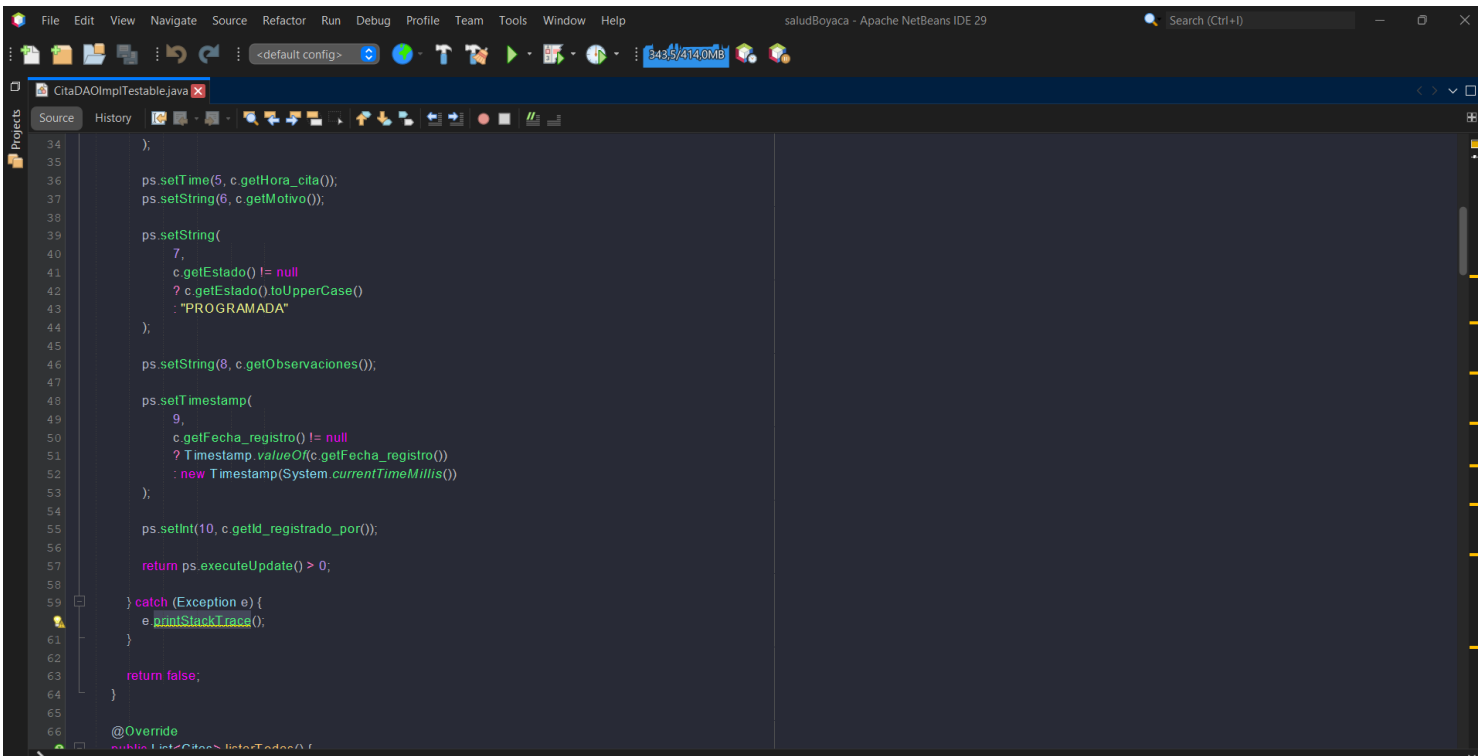
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)

CitaDAOImplTestable.java X

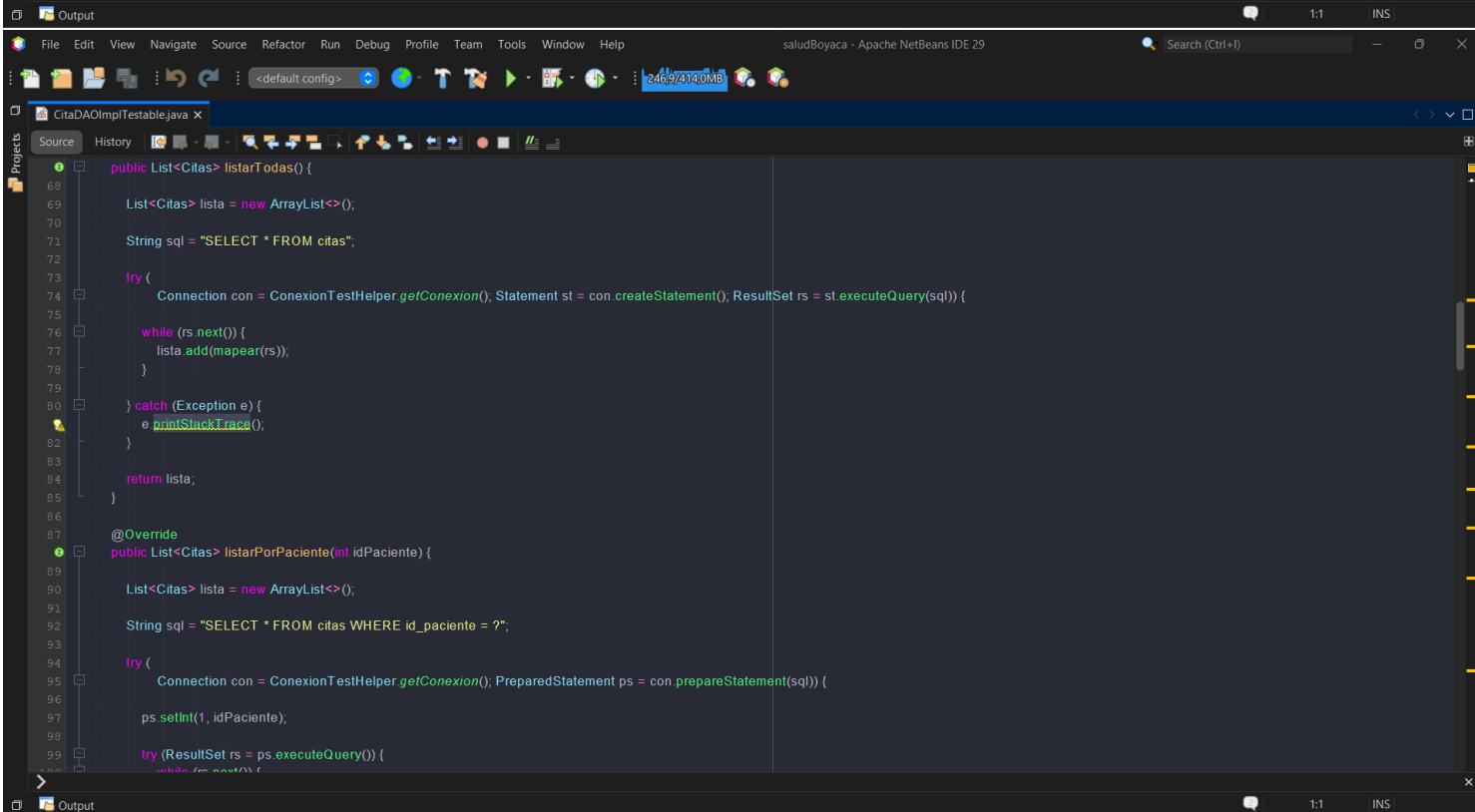
Source History

```
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Citas;
4
5 import java.sql.*;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class CitaDAOImplTestable implements CitaDAO {
10
11     @Override
12     public boolean insertar(Citas c) {
13
14         String sql = ""
15             INSERT INTO citas
16             (id_paciente, id_medico, id_especialidad,
17             fecha_cita, hora_cita, motivo, estado,
18             observaciones, fecha_registro, id_registro_por)
19             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
20         "";
21
22         try {
23             Connection con = ConexionTestHelper.getConnection();
24             PreparedStatement ps = con.prepareStatement(sql);
25
26             ps.setInt(1, c.getId_paciente());
27             ps.setInt(2, c.getId_medico());
28             ps.setInt(3, c.getId_especialidad());
29
30             ps.setTimestamp(
31                 4,
32                 c.getFecha_cita() != null
33                     ? Timestamp.valueOf(c.getFecha_cita())
34                     : null
35             );
36
37         } catch (SQLException ex) {
38             ex.printStackTrace();
39         }
40
41         return true;
42     }
43 }
```

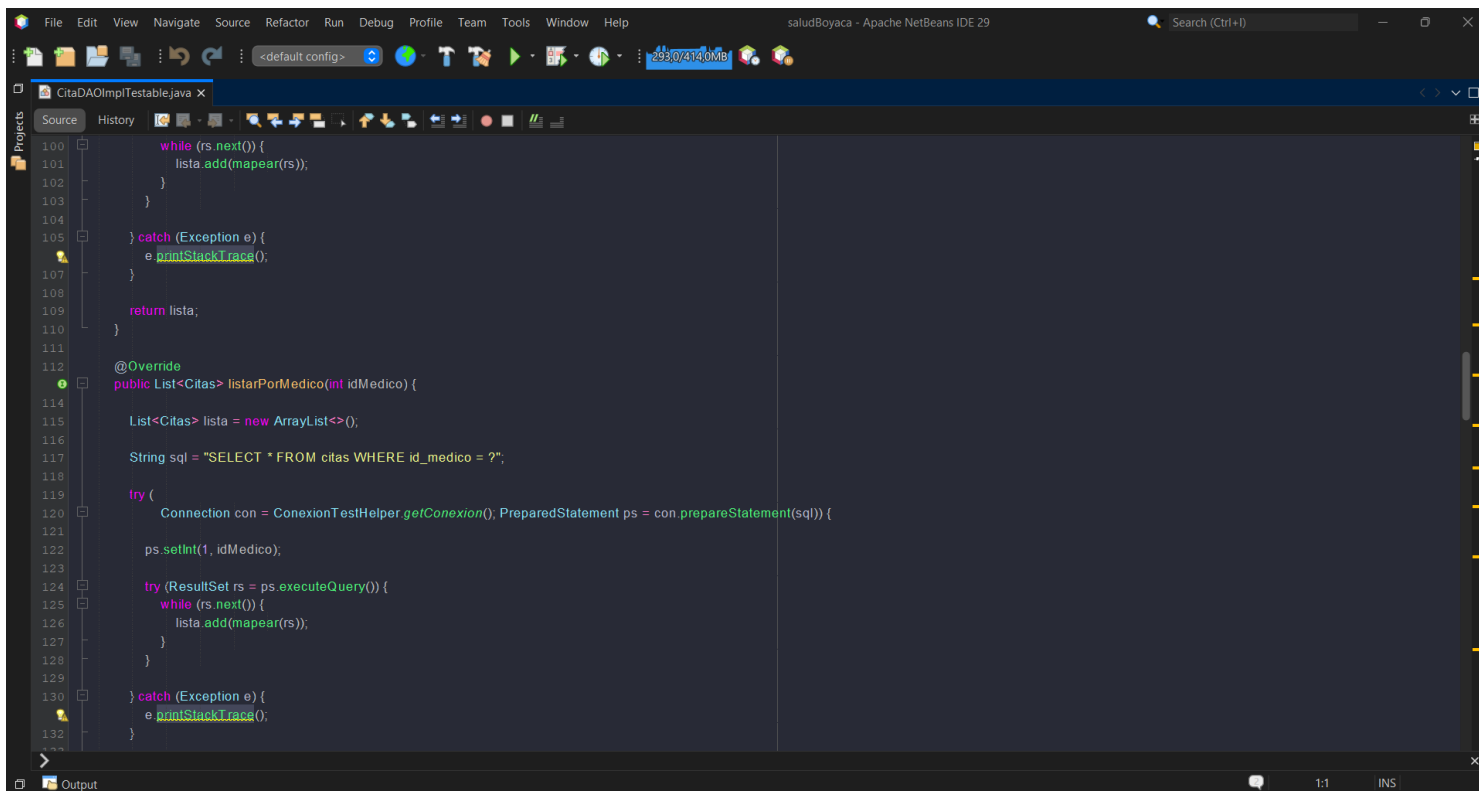
Output 1:1 INS



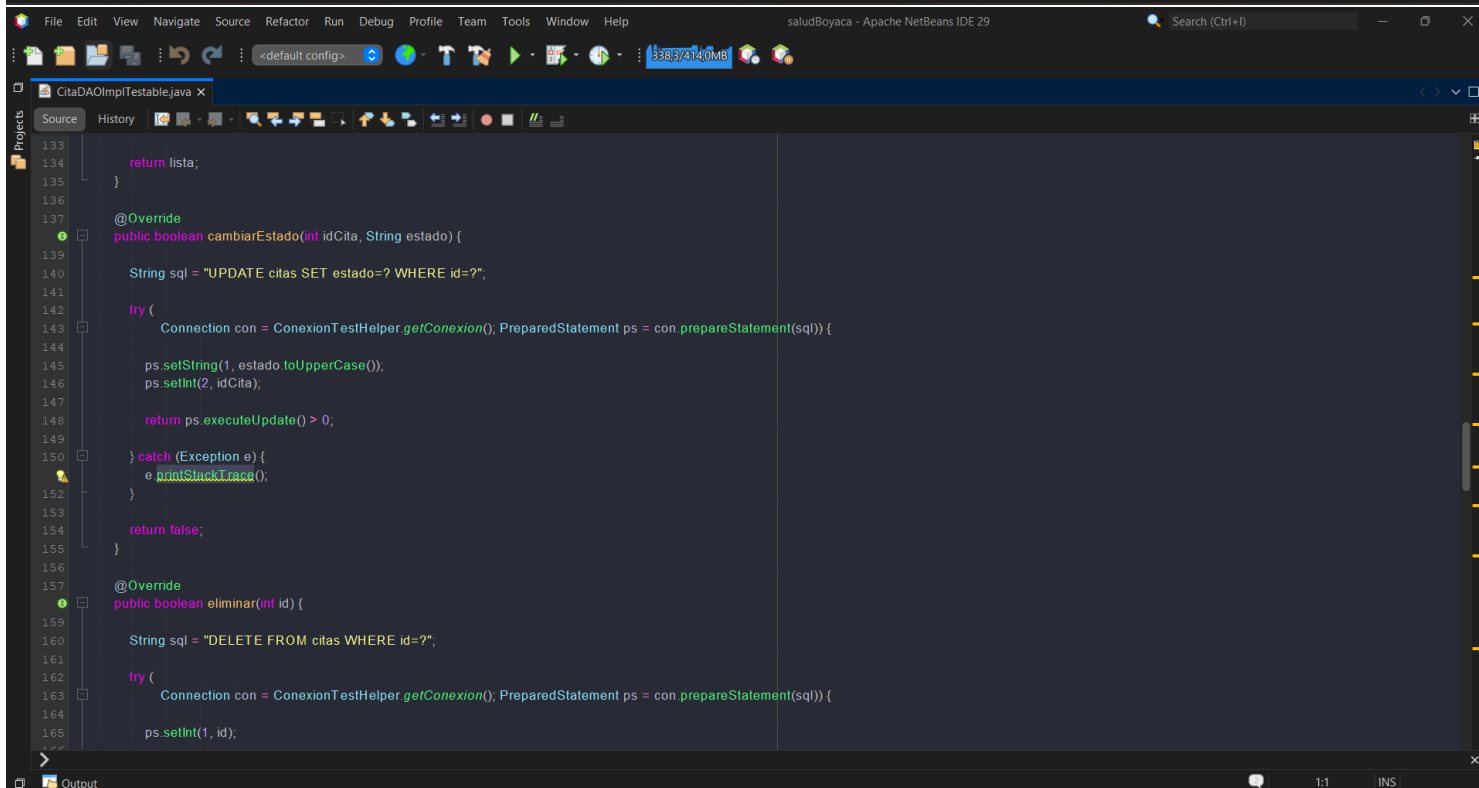
```
34  
35  
36 ps.setTime(5, c.getHora_cita());  
37 ps.setString(6, c.getMotivo());  
38  
39 ps.setString(  
40     7,  
41     c.getEstado() != null  
42     ? c.getEstado().toUpperCase()  
43     : "PROGRAMADA"  
44 );  
45  
46 ps.setString(8, c.getObservaciones());  
47  
48 ps.setTimestamp(  
49     9,  
50     c.getFecha_registro() != null  
51     ? Timestamp.valueOf(c.getFecha_registro())  
52     : new Timestamp(System.currentTimeMillis())  
53 );  
54  
55 ps.setInt(10, c.getId_registro_por());  
56  
57 return ps.executeUpdate() > 0;  
58  
59 } catch (Exception e) {  
60     e.printStackTrace();  
61 }  
62  
63 return false;  
64 }  
65  
66 @Override  
67 public List<Citas> listarTodas() {
```



```
68 public List<Citas> listarTodas() {  
69     List<Citas> lista = new ArrayList<>();  
70  
71     String sql = "SELECT * FROM citas";  
72  
73     try {  
74         Connection con = ConexionTestHelper.getConnection(); Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql); {  
75  
76         while (rs.next()) {  
77             lista.add(mapear(rs));  
78         }  
79     } catch (Exception e) {  
80         e.printStackTrace();  
81     }  
82  
83     return lista;  
84 }  
85  
86  
87 @Override  
88 public List<Citas> listarPorPaciente(int idPaciente) {  
89     List<Citas> lista = new ArrayList<>();  
90  
91     String sql = "SELECT * FROM citas WHERE id_paciente = ?";  
92  
93     try {  
94         Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql); {  
95  
96         ps.setInt(1, idPaciente);  
97  
98         try (ResultSet rs = ps.executeQuery()) {  
99             while (rs.next()) {
```



```
100     while (rs.next()) {
101         lista.add(mapear(rs));
102     }
103 }
104
105 } catch (Exception e) {
106     e.printStackTrace();
107 }
108
109 return lista;
110 }
111
112 @Override
113 public List<Citas> listarPorMedico(int idMedico) {
114     List<Citas> lista = new ArrayList<>();
115
116     String sql = "SELECT * FROM citas WHERE id_medico = ?";
117
118     try {
119         Connection con = ConexionTestHelper.getConexion();
120         PreparedStatement ps = con.prepareStatement(sql);
121
122         ps.setInt(1, idMedico);
123
124         try (ResultSet rs = ps.executeQuery()) {
125             while (rs.next()) {
126                 lista.add(mapear(rs));
127             }
128         }
129     } catch (Exception e) {
130         e.printStackTrace();
131     }
132 }
```

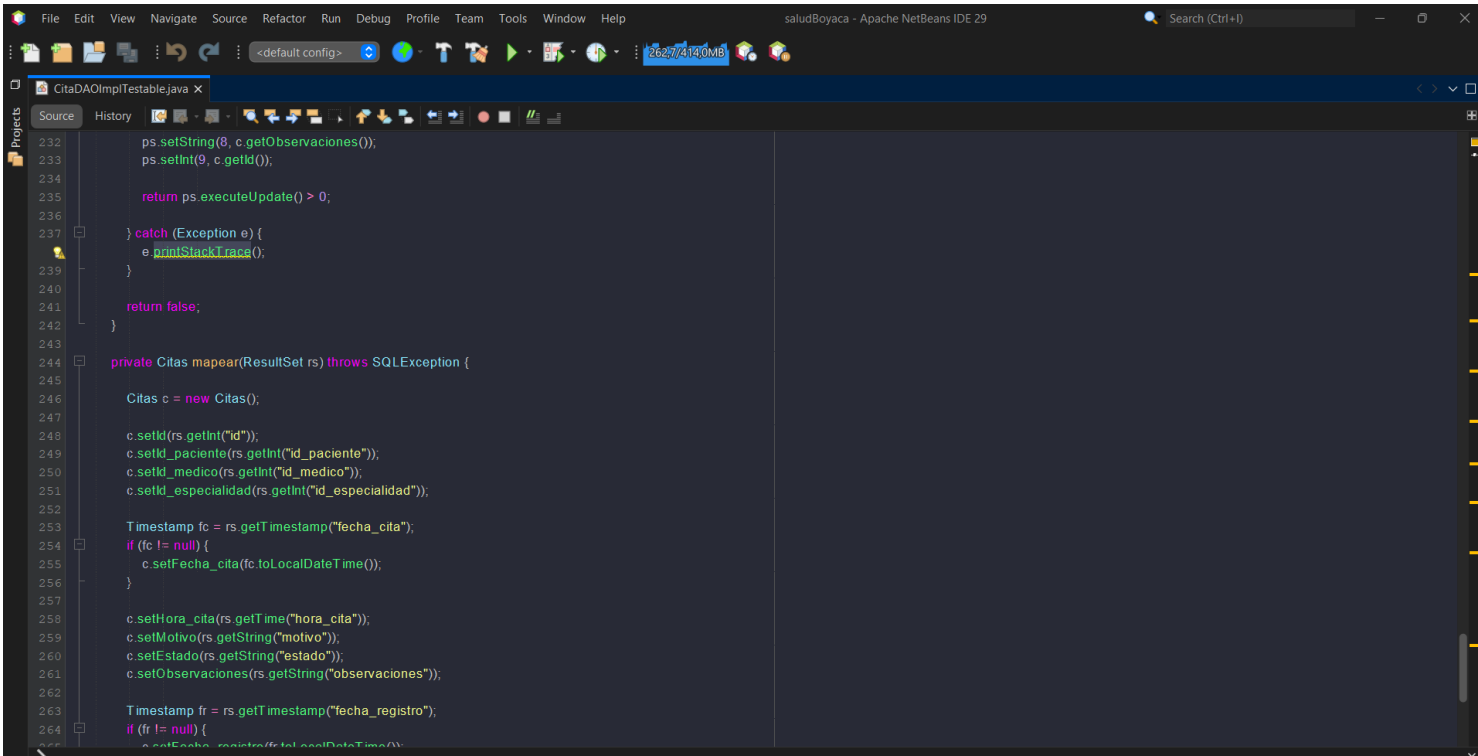


```
133     return lista;
134 }
135
136 @Override
137 public boolean cambiarEstado(int idCita, String estado) {
138     String sql = "UPDATE citas SET estado=? WHERE id=?";
139
140     try {
141         Connection con = ConexionTestHelper.getConexion();
142         PreparedStatement ps = con.prepareStatement(sql);
143
144         ps.setString(1, estado.toUpperCase());
145         ps.setInt(2, idCita);
146
147         return ps.executeUpdate() > 0;
148     } catch (Exception e) {
149         e.printStackTrace();
150     }
151
152     return false;
153 }
154
155 @Override
156 public boolean eliminar(int id) {
157     String sql = "DELETE FROM citas WHERE id=?";
158
159     try {
160         Connection con = ConexionTestHelper.getConexion();
161         PreparedStatement ps = con.prepareStatement(sql);
162
163         ps.setInt(1, id);
164     }
```

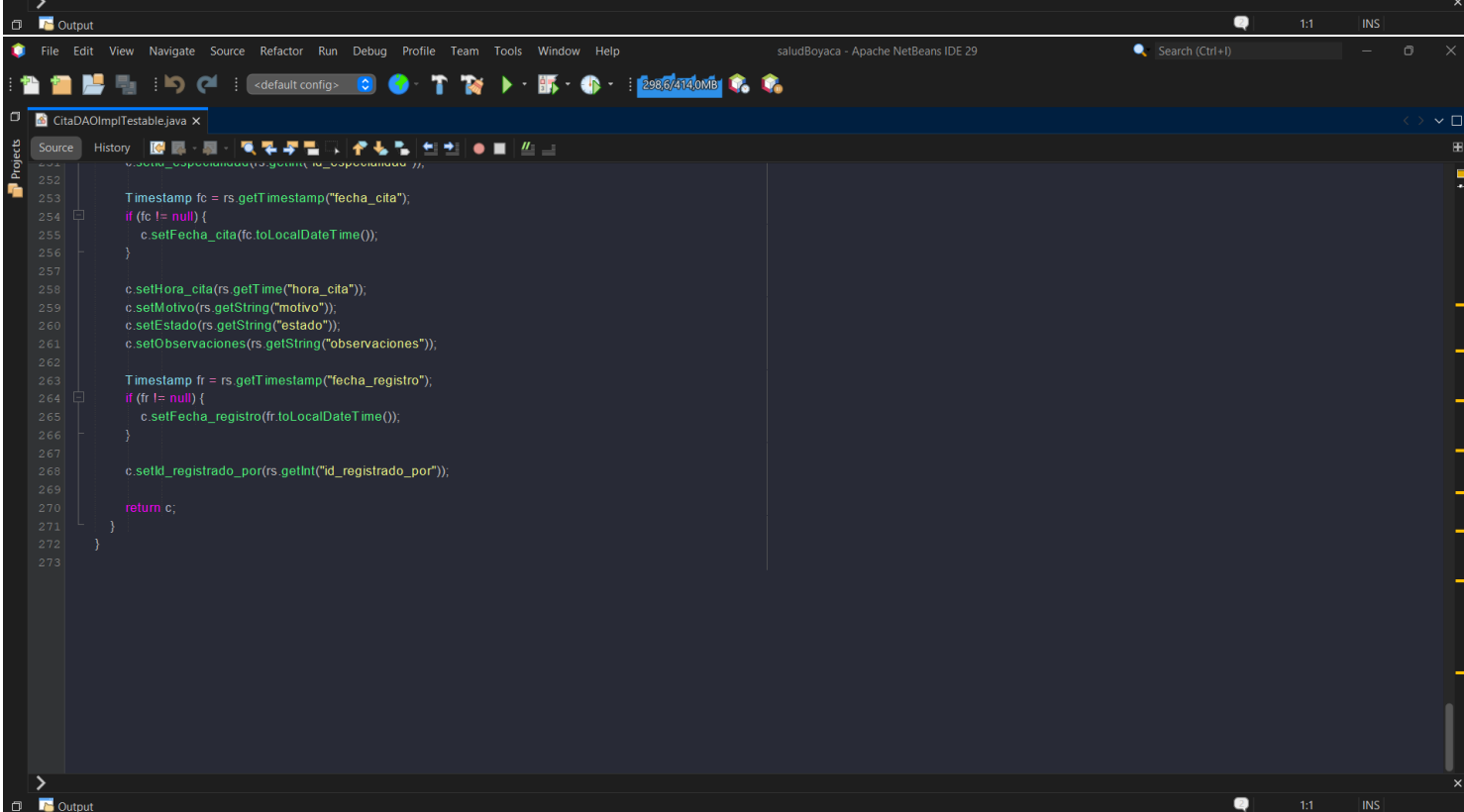


```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
CitaDAOImplTestable.java X
Source History
166
167     return ps.executeUpdate() > 0;
168
169     } catch (Exception e) {
170         e.printStackTrace();
171     }
172
173     return false;
174 }
175
176 @Override
177 public Citas buscarPorId(int id) {
178     String sql = "SELECT * FROM citas WHERE id=?";
179
180     try {
181         Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql);
182
183         ps.setInt(1, id);
184
185         try (ResultSet rs = ps.executeQuery()) {
186             if (rs.next()) {
187                 return mapear(rs);
188             }
189         }
190     } catch (Exception e) {
191         e.printStackTrace();
192     }
193
194     return null;
195 }
196
197 @Override
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
CitaDAOImplTestable.java X
Source History
199 @Override
200 public boolean actualizar(Citas c) {
201
202     String sql = ""
203     UPDATE citas
204     SET id_paciente=?,
205         id_medico=?,
206         id_especialidad=?,
207         fecha_cita=?,
208         hora_cita=?,
209         motivo=?,
210         estado=?,
211         observaciones=?
212     WHERE id=?
213     ,
214
215     try {
216         Connection con = ConexionTestHelper.getConnection(); PreparedStatement ps = con.prepareStatement(sql);
217
218         ps.setInt(1, c.getId_paciente());
219         ps.setInt(2, c.getId_medico());
220         ps.setInt(3, c.getId_especialidad());
221
222         ps.setTimestamp(
223             4,
224             c.getFecha_cita() != null
225             ? Timestamp.valueOf(c.getFecha_cita())
226             : null
227         );
228
229         ps.setTime(5, c.getHora_cita());
230         ps.setString(6, c.getMotivo());
231         ps.setString(7, c.getEstado());
232         ps.setString(8, c.getObservaciones());
233     }
```



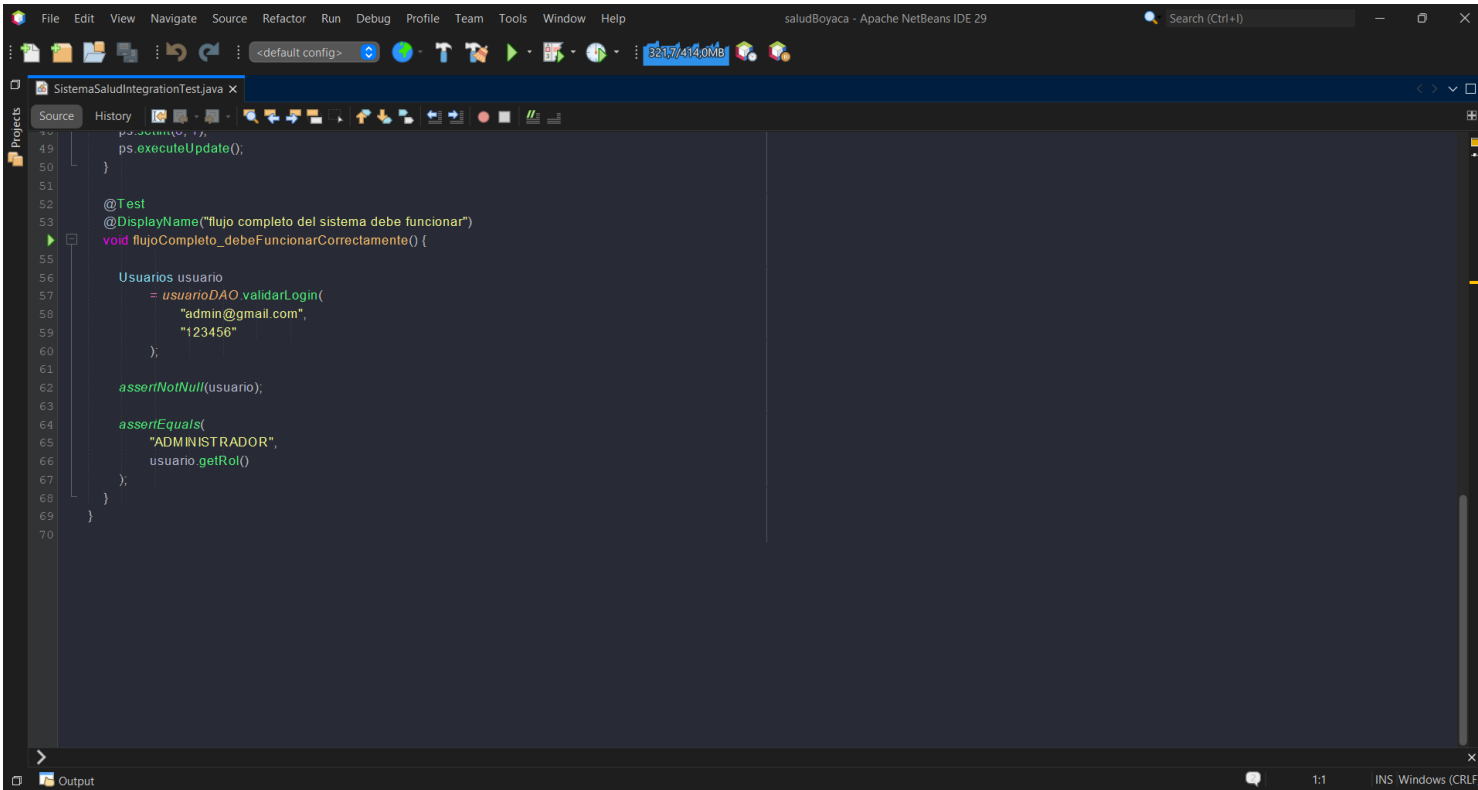
```
232 ps.setString(8, c.getObservaciones());
233 ps.setInt(9, c.getId());
234
235 return ps.executeUpdate() > 0;
236
237 } catch (Exception e) {
238     e.printStackTrace();
239 }
240
241 return false;
242 }
243
244 private Citas mappear(ResultSet rs) throws SQLException {
245
246     Citas c = new Citas();
247
248     c.setId(rs.getInt("id"));
249     c.setId_paciente(rs.getInt("id_paciente"));
250     c.setId_medico(rs.getInt("id_medico"));
251     c.setId_especialidad(rs.getInt("id_especialidad"));
252
253     Timestamp fc = rs.getTimestamp("fecha_cita");
254     if (fc != null) {
255         c.setFecha_cita(fc.toLocalDateTime());
256     }
257
258     c.setHora_cita(rs.getTime("hora_cita"));
259     c.setMotivo(rs.getString("motivo"));
260     c.setEstado(rs.getString("estado"));
261     c.setObservaciones(rs.getString("observaciones"));
262
263     Timestamp fr = rs.getTimestamp("fecha_registro");
264     if (fr != null) {
265         c.setFecha_registro(fr.toLocalDateTime());
266     }
267 }
```



```
252     c.setId_especialidad(rs.getInt("id_especialidad"));
253
254     Timestamp fc = rs.getTimestamp("fecha_cita");
255     if (fc != null) {
256         c.setFecha_cita(fc.toLocalDateTime());
257     }
258
259     c.setHora_cita(rs.getTime("hora_cita"));
260     c.setMotivo(rs.getString("motivo"));
261     c.setEstado(rs.getString("estado"));
262     c.setObservaciones(rs.getString("observaciones"));
263
264     Timestamp fr = rs.getTimestamp("fecha_registro");
265     if (fr != null) {
266         c.setFecha_registro(fr.toLocalDateTime());
267     }
268
269     c.setId_registrado_por(rs.getInt("id_registrado_por"));
270
271     return c;
272 }
273 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
SistemaSaludIntegrationTest.java x
Source History
1 package co.sena.cimm.adso.saludboyaca.dao;
2
3 import co.sena.cimm.adso.saludboyaca.model.Usuarios;
4
5 import org.junit.jupiter.api.BeforeAll;
6 import org.junit.jupiter.api.DisplayName;
7 import org.junit.jupiter.api.Test;
8
9 import java.sql.Connection;
10 import java.sql.PreparedStatement;
11
12 import static org.junit.jupiter.api.Assertions.*;
13
14 public class SistemaSaludIntegrationTest {
15
16     private static UsuarioDAOImplTestable usuarioDAO;
17
18     @BeforeAll
19     static void setUp() throws Exception {
20         usuarioDAO = new UsuarioDAOImplTestable();
21         ConexionTestHelper.ejecutarSQL("""
22 CREATE TABLE IF NOT EXISTS usuarios (
23     id INT AUTO_INCREMENT PRIMARY KEY,
24     nombres VARCHAR(100),
25     apellidos VARCHAR(100),
26     username VARCHAR(100),
27     email VARCHAR(100),
28     password VARCHAR(100),
29     rol VARCHAR(50),
30     lang_preferido VARCHAR(10),
31     activo INT
32 );
33 """);
34     }
35 }
```

```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help saludBoyaca - Apache NetBeans IDE 29 Search (Ctrl+I)
SistemaSaludIntegrationTest.java x
Source History
34 ConexionTestHelper.ejecutarSQL("DELETE FROM usuarios");
35 Connection con = ConexionTestHelper.getConexion();
36 PreparedStatement ps = con.prepareStatement("""
37 INSERT INTO usuarios
38 (nombres, apellidos, username, email, password, rol, lang_preferido, activo)
39 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
40 """);
41 ps.setString(1, "Admin");
42 ps.setString(2, "Principal");
43 ps.setString(3, "admin");
44 ps.setString(4, "admin@gmail.com");
45 ps.setString(5, "123456");
46 ps.setString(6, "ADMINISTRADOR");
47 ps.setString(7, "es");
48 ps.setInt(8, 1);
49 ps.executeUpdate();
50
51
52 @Test
53 @DisplayName("Flujo completo del sistema debe funcionar")
54 void flujoCompleto_debeFuncionarCorrectamente() {
55
56     Usuarios usuario
57         = usuarioDAO.validarLogin(
58             "admin@gmail.com",
59             "123456"
60         );
61
62     assertNotNull(usuario);
63
64     assertEquals(
65         "ADMINISTRADOR",
66         usuario.getRol()
67     );
68 }
```



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\julia\OneDrive\Documentos\NetBeansProjects\saludBoyaca> mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.mycompany:saludBoyaca >-----
[INFO] Building saludBoyaca 1.0
[INFO]   from pom.xml
[INFO]
[INFO] -----[ war ]-----
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ saludBoyaca ---
[INFO] Copying 3 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ saludBoyaca ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ saludBoyaca ---
[INFO] Copying 1 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ saludBoyaca ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 20 source files with javac [debug target 17] to target\test-classes
[WARNING] system modules path not set in conjunction with -source 17
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ saludBoyaca ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.CitaDAOImplTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.523 s -- in co.sena.cimm.adso.saludboyaca.dao.CitaDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.EspecialidadDAOImplTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in co.sena.cimm.adso.saludboyaca.dao.EspecialidadDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.HorarioDAOImplTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in co.sena.cimm.adso.saludboyaca.dao.HorarioDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.PacienteDAOImplTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.163 s -- in co.sena.cimm.adso.saludboyaca.dao.PacienteDAOImplTest
```

```
Windows PowerShell
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.523 s -- in co.sena.cimm.adso.saludboyaca.dao.CitaDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.EspecialidadDAOImplTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in co.sena.cimm.adso.saludboyaca.dao.EspecialidadDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.HorarioDAOImplTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in co.sena.cimm.adso.saludboyaca.dao.HorarioDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.PacienteDAOImplTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.163 s -- in co.sena.cimm.adso.saludboyaca.dao.PacienteDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.SistemaSaludIntegrationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in co.sena.cimm.adso.saludboyaca.dao.SistemaSaludIntegrationTest
[INFO] Running co.sena.cimm.adso.saludboyaca.dao.UsuarioDAOImplTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in co.sena.cimm.adso.saludboyaca.dao.UsuarioDAOImplTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.CitasTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s -- in co.sena.cimm.adso.saludboyaca.model.CitasTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.EspecialidadesTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in co.sena.cimm.adso.saludboyaca.model.EspecialidadesTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.HorariosTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in co.sena.cimm.adso.saludboyaca.model.HorariosTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.Log_accesosTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in co.sena.cimm.adso.saludboyaca.model.Log_accesosTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.OTP_TokensTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.019 s -- in co.sena.cimm.adso.saludboyaca.model.OTP_TokensTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.PacienteTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.PacienteTest$NombreCompletoTests
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.009 s -- in co.sena.cimm.adso.saludboyaca.model.PacienteTest$NombreCompletoTests
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.032 s -- in co.sena.cimm.adso.saludboyaca.model.PacienteTest
[INFO] Running co.sena.cimm.adso.saludboyaca.model.UsuarioTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.025 s -- in co.sena.cimm.adso.saludboyaca.model.UsuarioTest
[INFO] Running co.sena.cimm.adso.saludboyaca.util.ValidadorDocumentoTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 s -- in co.sena.cimm.adso.saludboyaca.util.ValidadorDocumentoTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.300 s
[INFO] Finished at: 2026-05-14T07:05:52-05:00
[INFO] -----
PS C:\Users\julia\OneDrive\Documentos\NetBeansProjects\saludBoyaca>
```